

2

Grid and reservoir modeling

This chapter describes grid and reservoir modeling in Eclipse. The creation of a grid which accurately describes the reservoir geology is fundamental to reservoir simulation, and in this chapter the underlying equations used to do this are explained for both simple and complex models.

- The [Transmissibility calculations](#) chapter explains the calculation of transmissibility between cells in the grid for the various grid types available. This is fundamental to all simulations.
- The [Local grid refinement and coarsening](#) chapter explains how the user can create areas of refinement where greater spatial accuracy is needed and how the user can reduce resolution in parts of the grid which have less impact on the simulation.
- The [Aquifer modeling facilities](#) chapter describes the wide range of facilities available to the user for modeling aquifers. These techniques may be numerical or analytical.
- The [Dual porosity](#) and [Multi porosity](#) chapters describe techniques available for modelling naturally fractured reservoirs, including shale and coal bed methane reservoirs.
- The [Independent reservoir regions](#) chapter describes a facility for simulating multiple reservoirs which are produced through a shared production facility and therefore share wells, either directly or through surface facilities.
- The [Vertical equilibrium](#) chapter describes a set of modifications to Eclipse suitable for situations where gravity dominates over viscous and capillary forces. The VE option has most impact when vertical resolution is not adequate to give an accurate model of the vertical segregation of phases such as when cells are large and the transition zone is small.

Transmissibility calculations

x	Eclipse 100
x	Eclipse 300

This chapter describes the expressions used by Eclipse to calculate transmissibility values, between cells in the grid, and between cells and numerical aquifers.

The transmissibility calculations for each type of geometry are detailed in the next section. In the corner point case, transmissibility values between cells that are not neighbors in the basic indexing grid may be calculated on the same basis as those for neighboring cells. In both cases, the mutual interface area is calculated, and this may be non-zero for non-neighboring cells at faults.

Cell ordering and geometry

The basic grid indexing system is by natural order. Cells may be identified by their (i ,j, k) indices in the Cartesian indexing grid, and in natural ordering, the i index changes most quickly, the k index most slowly. Internally, cells are held by active order, with any inactive cells (that is, those with zero pore volume) not requiring any storage.

Details of cell geometry may be entered using keywords such as [DX](#), [DY](#) and [DZ](#) to specify cell sizes. This is a block center description, and cells are assumed to have a simple, regular shape.

For example, the pore volume of each cell is then $DX \cdot DY \cdot DZ \cdot \varphi \cdot RNTG$

where

φ is the porosity

RNTG the net to gross ratio, and

DX, DY and DZ the cell dimensions.

An alternative is the entry of cell corner points using the [COORD](#) and [ZCORN](#) keywords. The data associated with these keywords is usually prepared using a pre-processor such as Petrel, and entered from an included file. The cell is assumed to have edges defined as straight lines between the corner points, and the cell faces may be bilinear surfaces. Pore volumes may be calculated exactly for such a shape.

It is possible to have more than one set of COORD data, using the [NUMRES](#) keyword in the RUNSPEC section, and [COORDSYS](#) to assign layers to these sets.

Transmissibility calculations

There are at most three types of transmissibility calculation available in Eclipse.

The first is the traditional block centered form, based upon the distances between the cell centers only. This is the default calculation when [DX](#), [DY](#), [DZ](#) are specified.

Note: For Eclipse 100, this corresponds to the use of the [OLDTRAN](#) keyword in the GRID section.

Eclipse 100

The second form also assumes a traditional block centered geometry but involves a different combination of cell cross-sectional areas and permeabilities from that assumed in the [OLDTRAN](#) formulation. The second form of transmissibility calculation is activated using the [OLDTRANR](#) keyword in the GRID section.

The third form is based upon the use of cell corner points, which are available to Eclipse when the COORD/ZCORN form of grid definition is used. In this case, it is possible to distinguish unambiguously between cell dip and fault displacement, and this option allows fault transmissibilities to be generated automatically.

Note: For Eclipse 100 use, this third type of transmissibility calculation is specified by the use of the **NEWTRAN** keyword in the GRID section. This is the default calculation when COORD/ZCORN are specified.

Eclipse 100

Although it is normal to use OLDTRAN with DX, DY, DZ, **TOPS** etc. input, and NEWTRAN with COORD/ZCORN, all types of transmissibility calculation are available with both types of grid specification, as Eclipse holds grid data internally in both forms. Corner points are obtained from DX/DY/DZ/TOPS data by assuming flat blocks, and DX/DY/DZ values are obtained from corner points by evaluation of the distances between cell face center points. However, use of NEWTRAN with DX/DY/DZ/TOPS input may well produce large numbers of spurious fault connections, as the assumed flat grid blocks in neighboring columns will generally overlap, and this combination of options is not recommended.

Note: If the grid has shales modeled as gaps in the grid, the keyword **MULTZ-** must be specified.

Cartesian transmissibility calculations

Block center transmissibility calculations

In this case, X- and Y-direction transmissibility values in the Cartesian case are obtained using cell center separations and cross sectional areas obtained from DX, DY and DZ, with a dip correction.

Eclipse 100

Note: For Eclipse 100, this type of calculation is also referred to as OLDTRAN.

X- transmissibility

$$\text{TRANX}_i = \frac{\text{CDARCY} \cdot \text{TMLTX}_i \cdot A \cdot \text{DIPC}}{B} \quad \text{Eq. 2.1}$$

where

TRANX_i Transmissibility between cell i and cell j, its neighbor in the positive X-direction

CDARCY Darcy's constant

= 0.00852702 (E300); 0.008527 (Eclipse 100) (METRIC)

= 0.00112712 (E300); 0.001127 (Eclipse 100) (FIELD)

= 3.6 (LAB)

= 0.00864 (PVT-M)

TMLTX_i Transmissibility multiplier for cell i

A Interface area between cell i and j

DIPC Dip correction.

A, DIPC and B are given by the expressions:

$$A = \frac{DX_j \cdot DY_i \cdot DZ_i \cdot RNTG_i + DX_i \cdot DY_j \cdot DZ_j \cdot RNTG_j}{DX_i + DX_j} \quad \text{Eq. 2.2}$$

$$B = \frac{\left(\frac{DX_i}{\text{PERMX}_i} + \frac{DX_j}{\text{PERMX}_j} \right)}{2} \quad \text{Eq. 2.3}$$

$$\text{DIPC} = \frac{\text{DHS}}{\text{DHS} + \text{DVS}} \quad \text{Eq. 2.4}$$

with

$$\text{DHS} = \left(\frac{DX_i + DX_j}{2} \right)^2$$

and

$$\text{DVS} = [\text{DEPTH}_i - \text{DEPTH}_j]^2.$$

RNTG is the net to gross ratio, which appears in the X- and Y- transmissibilities but not in the Z- transmissibility.

Y- transmissibility

The expression for the Y-transmissibility value is entirely analogous to the above, with the appropriate permutations of X, Y and Z.

Z- transmissibility

The Z-transmissibility is given by:

$$\text{TRANZ}_i = \frac{\text{CDARCY} \cdot \text{TMLTZ}_i \cdot A}{B} \quad \text{Eq. 2.5}$$

where

TRANZ_i Transmissibility between cell i and cell j, its neighbor in the positive Z-direction (that is below cell i)

TMLTZ_i The Z-transmissibility multiplier for cell i

and

$$A = \frac{DZ_j \cdot DX_i \cdot DY_i + DZ_i \cdot DX_j \cdot DY_j}{DZ_i + DZ_j} \quad \text{Eq. 2.6}$$

$$B = \frac{\left(\frac{DZ_i}{\text{PERMZ}_i} + \frac{DZ_j}{\text{PERMZ}_j} \right)}{2} \quad \text{Eq. 2.7}$$

Note that there is no dip correction in the Z-transmissibility case.

OLDTRANR transmissibility calculations

In this case, X- and Y-direction transmissibility values in the Cartesian case are also calculated using cell center separations and cross-sectional areas obtained from DX, DY and DZ, with a dip correction.

The expressions for the Cartesian case are given below:

X- transmissibility

$$\text{TRANX}_i = \frac{\text{CDARCY} \cdot \text{TMLTX}_i \cdot \text{DIPC}}{B} \quad \text{Eq. 2.8}$$

where

TRANX_i Transmissibility between cell i and cell j, its neighbor in the positive X-direction

CDARCY Darcy's constant (in the appropriate units)

TMLTX_i Transmissibility multiplier for cell i

DIPC Dip correction.

B and DIPC are given by the expressions:

$$B = \frac{\left(\frac{\text{DX}_i}{A_i \cdot \text{PERMX}_i} + \frac{\text{DX}_j}{A_j \cdot \text{PERMX}_j} \right)}{2} \quad \text{Eq. 2.9}$$

where

$$A_i = \text{DY}_i \cdot \text{DZ}_i \cdot \text{NTG}_i$$

and

with

$$\text{DIPC} = \frac{\text{DHS}}{\text{DHS} + \text{DVS}} \quad \text{Eq. 2.10}$$

$$\text{DHS} = \left(\frac{\text{DX}_i + \text{DX}_j}{2} \right)^2$$

and

$$\text{DVS} = [\text{DEPTH}_i - \text{DEPTH}_j]^2.$$

RNTG is the net to gross ratio, which appears in the X- and Y-transmissibilities but not in the Z-transmissibility.

Y- transmissibility

The expression for the Y-transmissibility value is entirely analogous to the above, with the appropriate permutations of X, Y and Z.

Z- transmissibility

The Z-transmissibility is given by:

$$\text{TRANZ}_i = \frac{\text{CDARCY} \cdot \text{TMLTZ}_i}{B} \quad \text{Eq. 2.11}$$

where

TRANZ_i Transmissibility between cell i and cell j, its neighbor in the positive Z-direction (that is below cell i)

TMLTZ_i The Z-transmissibility multiplier for cell i

$$B = \frac{\left(\frac{DZ_i}{A_i \cdot \text{PERMZ}_i} + \frac{DZ_j}{A_j \cdot \text{PERMZ}_j} \right)}{2} \quad \text{Eq. 2.12}$$

with

$$A_i = DX_i \cdot DY_i$$

$$A_j = DX_j \cdot DY_j$$

Note that there is no dip correction in the Z-transmissibility case.

Corner point transmissibility calculations

In this case, the transmissibility values are calculated from the X-,Y- and Z-projections of the mutual interface area of the two cells.

An inner product is then taken with the vector distance from the cell center to the center of the cell face, so that a dip correction is automatically incorporated.

Note: For Eclipse 100, this type of calculation is also referred to as NEWTRAN.

X- transmissibility

The X-transmissibility is given by the expression:

$$\text{TRANX}_i = \frac{\text{CDARCY} \cdot \text{TMLTX}_i}{\frac{1}{T_i} + \frac{1}{T_j}} \quad \text{Eq. 2.13}$$

where

$$T_i = \text{PERMX}_i \cdot \text{RNTG}_i \cdot \frac{A \cdot D_i}{D_i \cdot D_i}$$

with

$$(A \cdot D_i) = A_X \cdot D_{iX} + A_Y \cdot D_{iY} + A_Z \cdot D_{iZ}$$

and

$$(D_i \cdot D_i) = D_{iX}^2 + D_{iY}^2 + D_{iZ}^2.$$

A_X , A_Y and A_Z are the X-,Y- and Z- projections of the mutual interface area of cell i and cell j (which need not be neighbors in the Cartesian indexing grid), and

D_{iX} , D_{iY} and D_{iZ} are the X-, Y- and Z-components of the distance between the center of cell i and the center of the relevant face of cell i, these centers being obtained as the appropriate average.

The expression for T_j is analogous.

Y- and Z- transmissibility

The Y- and Z-transmissibility expressions are similar, the net to gross ratio being absent from the Z expression.

The calculation is illustrated by the following figure (where PERMX is represented by Kx):

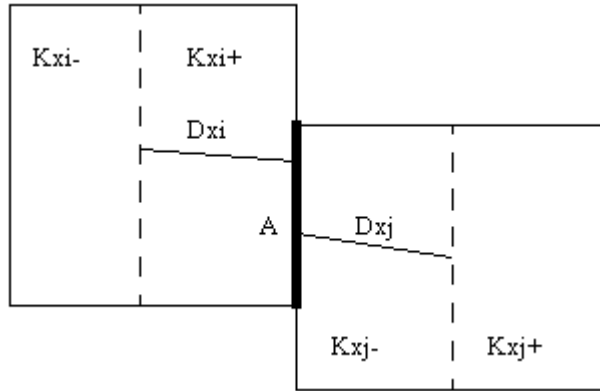


Figure 2.1. Y- and Z- transmissibility expressions

HALFTRAN transmissibility calculations (Eclipse 100)

By observing that the transmissibility between cells i and j in the NEWTRAN formulation is composed of a harmonic average between half block transmissibilities T_i and T_j , we have rather more freedom than just using a single permeability value for each cell.

In fact, there are six possible permeability values associated with each cell in the half block transmissibilities. For instance, in cell i, half block x-direction permeabilities K_{xil} , K_{xir} contribute to the two x-direction transmissibilities affecting cell i. Similarly in the Y- and Z-directions we can use K_{yil} , K_{yir} and K_{zil} , K_{zir} .

The use of extra half block permeabilities provides a more accurate upscaling of permeability while retaining the same computational grid.

Within Eclipse 100, the [HALFTRAN](#) keyword activates the half block transmissibility calculation.

By writing the half block transmissibilities as

$$\bar{T}_{ir} = K_{xir} \cdot RNTG_i \cdot \frac{A \cdot D_i}{D_i \cdot D_i} \quad \text{Eq. 2.14}$$

and

$$\bar{T}_{jl} = K_{xjl} \cdot RNTG_j \cdot \frac{A \cdot D_j}{D_j \cdot D_j} \quad \text{Eq. 2.15}$$

it is apparent that in terms of the normal NEWTRAN calculation, then

$$\overline{T}_{ir} = \lambda_{ir} T_i$$

and

$$\overline{T}_{jl} = \lambda_{jl} T_j$$

with

$$\lambda_{ir} = Kx_{ir}/Kx_i \text{ and } \lambda_{jl} = Kx_{jl}/Kx_j.$$

When HALFTRAN is used, Eclipse 100 requires the [MULTX](#) and [MULTX-](#) values to be supplied equal to λ_{ir} and λ_{jl} .

At present, the FloGrid program carries out the half-block upscaling and present the modified [MULTX](#) and [MULTX-](#) data to the simulator in order to effect the half block transmissibility calculation. The [PERMX](#) etc. values should be entered as normal. The half block permeabilities are then obtained by multiplying [PERMX](#) by [MULTX](#) or [MULTX-](#) etc. The connection factors are calculated as usual using Peaceman's formula for the block center permeability [PERMX](#) etc.

Radial transmissibility calculations

There is no distinction between the method used to calculate radial transmissibility values whether grid data is supplied in [DR](#) / [DTHETA](#) / [DZ](#) / [INRAD](#) / [TOPS](#) form or [COORD](#) / [ZCORN](#) form.

The expressions used are based upon true radial flow between the pressure equivalent radii, with a dip correction.

R-transmissibility

$$\text{TRANR}_i = \frac{\text{CDARCY} \cdot \text{TMLTR}_i \cdot \text{DIPC}}{\frac{1}{T_i} + \frac{1}{T_j}} \quad \text{Eq. 2.16}$$

with

$$T_i = \frac{\text{PERMR}_i \cdot \text{RNTG}_i \cdot D\theta_i \cdot \text{DZ}_i}{D_{1P}}$$

$$D_{1P} = \frac{R_1^2}{(R_2^2 - R_1^2)} \ln(R_1/R_2) + \frac{1}{2}$$

and

$$T_j = \frac{\text{PERMR}_j \cdot \text{RNTG}_j \cdot D\theta_j \cdot \text{DZ}_j}{D_{2M}}$$

$$D_{2M} = \frac{R_3^2}{(R_3^2 - R_2^2)} \ln(R_3/R_2) - \frac{1}{2}$$

where

R_1 is the inner radius of cell i,

R_2 is the outer radius of cell i,

R_3 is the outer radius of cell j .

Dip correction

The dip correction is given by:

$$DIPC = \frac{DHS}{DHS + DVS}$$

where

$$DHS = \left(\frac{1}{2}(R_3 - R_1) \right)^2$$

and

$$DVS = (\text{DEPTH}_i - \text{DEPTH}_j)^2.$$

Azimuthal transmissibility

The azimuthal transmissibility is given by:

$$\text{TRANT}_i = \frac{CDARCY \cdot TMLTT_i \cdot DIPC}{\frac{1}{T_i} + \frac{1}{T_j}}$$

Eq. 2.17

with

$$T_i = \frac{2 \cdot \text{PERMT}_i \cdot \text{RNTG}_i \cdot \text{DZ}_i \cdot \ln(R_2/R_1)}{D\theta_i}$$

where

R_1 is the inner radius,

R_2 is the outer radius.

T_j is similar.

It should be noted that $\ln()$ implies the natural logarithm throughout. This expression is also used in completing the circle in radial geometry.

Dip correction

The dip correction is given by:

$$DIPC = \frac{DHS}{DHS + DVS}$$

where

$$DHS = \left(\frac{1}{2}(D\theta_i + D\theta_j) \cdot R_{\text{mean}} \right)^2$$

and

$$DVS = (\text{DEPTH}_i - \text{DEPTH}_j)^2.$$

For Eclipse 100, R_{mean} takes a value of one, and for Eclipse 300, R_{mean} represents an average of the inner and outer radius.

Vertical transmissibility

The vertical transmissibility is given by:

$$\text{TRANZ}_i = \frac{\text{CDARCY} \cdot \text{TMLTZ}_i}{\frac{1}{T_i} + \frac{1}{T_j}} \quad \text{Eq. 2.18}$$

with

$$T_i = \frac{\text{PERMZ}_i \cdot D\theta_i \cdot (R_2^2 - R_1^2)}{DZ_i}$$

where

R_1 is the inner radius,

R_2 is the outer radius.

T_j is similar.

Note that the positions of the cells in a radial run may be entered by using [COORD](#) / [ZCORN](#) or [INRAD](#) / [DR](#) / [DTHETA](#) / [DZ](#).

In both cases the transmissibility expressions above may be used. There are some restrictions that exist on corner point positions for the above expressions to be sensible. Cells with the same j index should have a common $D\theta$, and cells with the same i index should have a common DR . If $\text{COORD}/\text{ZCORN}$ input is used, four corners of each cell should have a common inner radius, and four a common outer radius.

Similarly, there should be a single upper and lower theta value. (This implies vertical coordinate lines in the radial case). The depths of cell corners are, however, arbitrary, and may be used to obtain dipping radial geometry.

Variations on the transmissibility calculation (Eclipse 100)

The transmissibility calculation using `OLDTRAN`, `OLDTRANR` or `NEWTRAN` employs an effective average permeability for the face between two adjacent blocks, which Eclipse 100 calculates from the permeabilities and dimensions of these blocks.

Options are available, however, that allow you more control over the average permeability employed in these calculations. You may input the average permeabilities directly (see keyword [LINKPERM](#)), or request alternative averaging formulations (see keyword [PERMAVE](#)).

The options `LINKPERM` and `PERMAVE` may be useful for comparison with other simulators. It is also possible to override the transmissibilities calculated by Eclipse 100 by supplying transmissibilities directly in the `EDIT` section, for comparison with other simulators.

Use of linking permeabilities in the transmissibility calculation

The LINKPERM keyword may optionally be used to instruct Eclipse 100 to treat the grid block permeabilities (input with PERMX and related keywords) as applying to the cell faces instead of cell centers.

Eclipse 100 then uses the face permeabilities directly as the “linking permeabilities” in the transmissibility calculation, instead of calculating the linking permeabilities as an average of the adjacent grid block permeabilities.

For example, if $PERMX_l$ is chosen to represent the linking permeability in the X- direction between connecting cells I and J, then $PERMX_l$ simply replaces both $PERMX_i$ and $PERMX_j$ in the above expressions for the transmissibility in each of the OLDTRAN, OLDTRANR and NEWTRAN cases. The $PERMX_l$ values will be input as either $PERMX_i$ or $PERMX_j$ according to the option selected in LINKPERM.

Note: The grid block values of PERMX, PERMY and PERMZ are required to calculate well connection factors (see “The connection transmissibility factor”). If linking permeabilities are used to apply PERMX etc. to block faces, the input values may be inappropriate for calculating connection factors. In this case, connection factors could be set explicitly in COMPDAT or connection factor multipliers could be applied using WPIMULT.

Alternative permeability averaging schemes

The PERMAVE keyword may optionally be used to select alternative formulations for calculating the average permeability for the cell faces in the transmissibility calculation. The different formulations can best be compared by considering the formulae employed above in the default cases (when neither LINKPERM nor PERMAVE is specified).

Firstly, consider the OLDTRAN case. The X-transmissibility calculation outlined above can be expressed equivalently as:

$$TRANX_i = \frac{CDARCY \cdot TMLTX_i \cdot A \cdot DIPC}{B} \quad \text{Eq. 2.19}$$

with B rewritten as:

$$B = \frac{DX_i + DX_j}{2 \cdot PERMX_{av}} \quad \text{Eq. 2.20}$$

and the average permeability is given by a harmonic average:

$$PERMX_{av} = \frac{(W_i + W_j)}{\left(\frac{W_i}{PERMX_i} + \frac{W_j}{PERMX_j}\right)} \quad \text{Eq. 2.21}$$

with weights

$$\begin{aligned} W_i &= DX_i \\ W_j &= DX_j \end{aligned} \quad \text{Eq. 2.22}$$

This is just a special case of the power law average given by:

$$\text{PERMX}_{\text{av}} = \left[\frac{W_i \cdot \text{PERMX}_i^{\text{POW}} + W_j \cdot \text{PERMX}_j^{\text{POW}}}{W_i + W_j} \right]^{\frac{1}{\text{POW}}} \quad \text{Eq. 2.23}$$

where $\text{POW} = -1$.

Other forms of average may be obtained by varying the value of POW. A weighted arithmetic average is obtained if $\text{POW} = 1$, and a geometric average results in the limit as $\text{POW} \rightarrow 0$, that is

$$\text{PERMX}_{\text{av}} = \left[\text{PERMX}_i^{W_i} \cdot \text{PERMX}_j^{W_j} \right]^{1/(W_i + W_j)} \quad \text{Eq. 2.24}$$

Secondly, consider the `OLDTRANR` case. The formulation for `OLDTRANR` is analogous with `OLDTRAN`, but now the weights are given by

$$W_i = \frac{DX_i}{A_i} = \frac{DX_i}{DY_i \cdot DZ_i \cdot \text{RNTG}_i} \quad \text{Eq. 2.25}$$

$$W_j = \frac{DX_j}{A_j} = \frac{DX_j}{DY_j \cdot DZ_j \cdot \text{RNTG}_j}$$

Finally, consider the `NEWTRAN` case. The weights corresponding to the default harmonic average of block permeabilities are given by:

$$\frac{1}{W_i} = \text{RNTG}_i \cdot \frac{A \cdot D_i}{D_i \cdot D_i} \quad \text{Eq. 2.26}$$

$$\frac{1}{W_j} = \text{RNTG}_j \cdot \frac{A \cdot D_j}{D_j \cdot D_j}$$

and the alternative permeability averages are readily expressed in the same manner as above.

Nine-point schemes

The default discretization in Eclipse employs a five-point discretization in two dimensions or a seven point discretization in three dimensions. However, when the grid is non-orthogonal, or the flow is not aligned with the principal directions of the grid, then the solution accuracy may be compromised by this discretization. Eclipse uses a nine-point scheme to address this issue.

There are two nine-point schemes in Eclipse that address grid non-orthogonality errors and grid orientation effects only.

The `NINEPOIN` scheme reduces the effect of grid orientation effects for flows not aligned with the principal grid directions (such as diagonally across the grid), but does not account for non-orthogonality errors.

Eclipse 300

The Multi Point Flux Approximation (MPFA) scheme is a rigorous treatment of grid non-orthogonality and tensor permeability. This leads to a nine-point scheme when the grid is non-orthogonal or the permeability tensor is not aligned with the coordinate directions. The MPFA nine-point scheme is activated by the `MPFA` keyword. Refer to "[Tensor permeability](#)" for more details. The two nine-point schemes should not be used together.

The nine-point scheme based on diagonal NNC transmissibilities

The `NINEPOIN` keyword in the `RUNSPEC` section activates a nine-point scheme based on adding diagonal transmissibilities in the areal (XY) direction in order to reduce grid orientation effects when the flow is not aligned with the grid. Additional non-neighbor connections are added between diagonal neighbors. The

flows between each cell and its eight neighboring cells (in the areal dimension) are each computed using a two-point approximation to the pressure gradient. The scheme is a modified version of that given in [Yanosik and McCracken \(1979\)](#). The standard five-point transmissibilities are first obtained from data entered in the GRID and EDIT sections, and these are then modified to obtain the nine-point transmissibilities.

Deriving the nine-point scheme starts by splitting each transmissibility in the usual five-point scheme into two half-transmissibilities:

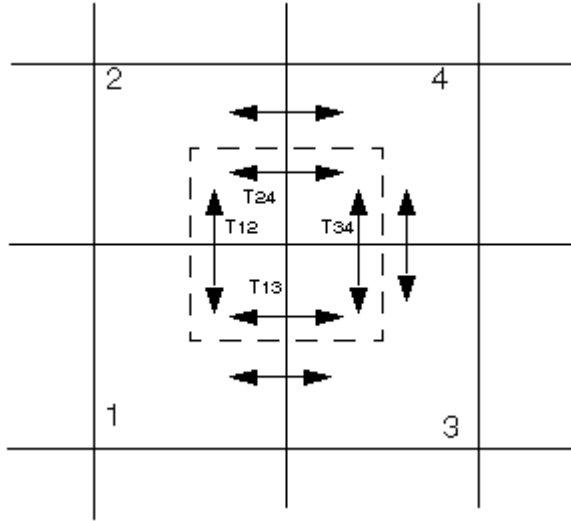


Figure 2.2. Splitting each five-point transmissibility into two half-transmissibilities

Considering just the transmissibility values ($T_{12}T_{24}T_{13}T_{34}$), replace these by the equivalent system ($t_{12}t_{13}t_{23}t_{24}t_{34}t_{14}$). This introduces a diagonal coupling between cells 1 and 4 and cells 2 and 3.

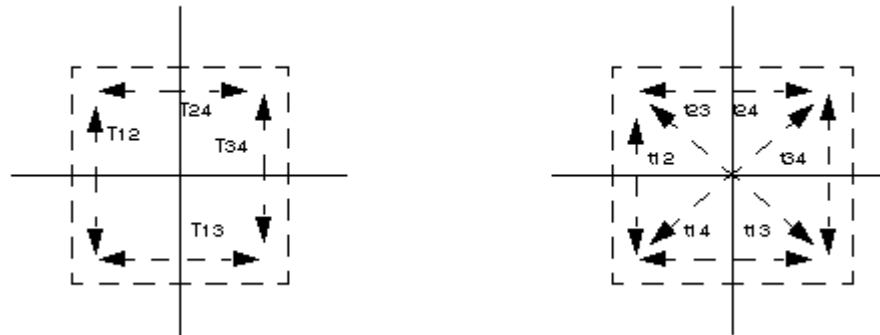


Figure 2.3. Deriving the nine-point transmissibilities using diagonal coupling between cells

The values of ($t_{12}t_{13}t_{23}t_{24}t_{34}t_{14}$) must preserve the total x and y transmissibilities for the pattern, so that

$$T_{13} + T_{24} = t_{13} + t_{24} + t_{23} + t_{14}$$

$$T_{12} + T_{34} = t_{12} + t_{34} + t_{23} + t_{14}$$

Tensor effects must be avoided, so that a pressure gradient in the x-direction does not produce a flow in the y-direction. This implies

$$t_{14} = t_{23}$$

A choice for t_{14} may now be made. This should not be so large that one of the other transmissibility values is forced to be negative. Following *Shiralkar and Stephenson (1991)*, use

$$t_{14} = t_{23} = \frac{1}{3} \left[\frac{(T_{13} + T_{24})(T_{12} + T_{34})}{(T_{13} + T_{24} + T_{12} + T_{34})} \right]$$

The total x and y transmissibility conditions above then define the values of the combinations $sx = t_{13} + t_{24}$ and $sy = t_{12} + t_{34}$, for example

$$sx = t_{13} + t_{24} = T_{13} + T_{24} - \frac{2}{3} \left[\frac{(T_{13} + T_{24})(T_{12} + T_{34})}{(T_{13} + T_{24} + T_{12} + T_{34})} \right]$$

Finally, the ratio of t_{13} and t_{24} is chosen to be the same as the ratio of the original values T_{13} and T_{24} ,

$$t_{13} = \frac{sxT_{13}}{(T_{13} + T_{24})}$$

$$t_{24} = \frac{sxT_{24}}{(T_{13} + T_{24})}$$

A similar relation defines t_{12} and t_{34} . This then defines all the new transmissibilities. This is repeated for each quadrant of active cells. The result is a set of nine-point transmissibilities that preserves the original total x- and y-direction flows but allows diagonal coupling between cells.

Within the code, the transmissibilities that fall outside the normal pattern are stored as non-neighbor connections. There is an extra cost associated with these terms, and the nine-point option is best used only for injection of highly adverse mobility fluids where the grid orientation effect can be a problem.

The option is invoked by using the `NINEPOIN` keyword in the `RUNSPEC` section. The nine-point transmissibilities will then by default be calculated across the entire grid. In Eclipse 100, by using the `NINENUM` keyword in the `GRID` section, you can choose to apply the scheme only in nominated regions of the reservoir. Note that the `NINENUM` keyword is not compatible with the Local Grid Refinement option.

It should be noted that the `NINEPOIN` option is specifically designed to account for grid orientation effects only and is not designed to deal with discretization errors arising from grid non-orthogonality.

This nine-point scheme based on diagonal NNC transmissibilities is invoked in the `RUNSPEC` section, as follows:

```

.....
RUNSPEC
..
NINEPOIN
.....

```

The run time will be increased due to the large number of additional NNCs created between diagonal neighbors.

The nine-point transmissibilities can be obtained from the 'TRANX', 'TRANY', 'TRANZ' and 'ALLNNC' switches in the `RPTGRID` keyword.

It is possible to apply the `NINEPOIN` scheme only in the XZ or YZ planes with the `NINEXZ` and `NINEYZ` keywords.

Tensor permeability (Eclipse 300)

Full tensor permeabilities are often required to describe complex reservoirs. Given a detailed heterogeneous reservoir description at the geological scale, upscaling is normally carried out to reduce the model size to one suitable for reservoir simulation. These upscaling procedures will generally create full tensor permeabilities, even if the permeabilities in the fine geological grid are not represented as tensors. Full tensor permeabilities also arise in complex crossbedded systems where the dip directions will not in general coincide with a local coordinate direction. A further case for full tensor permeabilities is in the modeling of fractured systems.

Conventionally, reservoir simulators ignore the full tensor nature of the absolute permeability and assume the permeability can be represented by a diagonal tensor whose principal axes coincide with the local coordinate axes. This allows the construction of the usual 5 point discretization in 2D or 7 point in 3D which can be solved efficiently in the simulator. Introduction of a full tensor permeability extends the stencil of the discretization scheme to 9 points in 2D and either 19 or 27 points in 3D.

Eclipse 300 allows a full tensor description of the absolute permeability. The permeability tensor may either be supplied by a pre-processor by upscaling the properties on the geological model, or else it may be entered by keywords `PERMXX`, `PERMY`, `PERMZ` (which describe the diagonal components) and `PERMXY`, `PERMYZ`, `PERMZ` to describe the off-diagonal components.

Only positive definite symmetric tensors are allowed in the simulator. The principal directions of the tensor must also be supplied. At present, the options are

- the directions for the components of the tensor are aligned with the Cartesian axes;
- aligned with the axes joining the midpoints of each pair of opposite faces in corner point cells; and
- aligned with the two axes formed by joining the midpoints of pairs of opposite faces in the I and J directions and with the third axis normal to the bedding plane.

The frame representing these directions is specified either by the `TBASIS` keyword or by the `MPFA` keyword.

The permeability tensor can be expressed as

$$\underline{K} = \begin{bmatrix} k_{xx} & k_{xy} & k_{xz} \\ k_{yx} & k_{yy} & k_{yz} \\ k_{zx} & k_{zy} & k_{zz} \end{bmatrix} \quad \text{Eq. 2.27}$$

where by symmetry

$$k_{yx} = k_{xy}, k_{zy} = k_{yz}, k_{zx} = k_{xz}. \quad \text{Eq. 2.28}$$

In Eclipse, the coefficients are entered as

$k_{xx} = \text{PERMXX}$, $k_{yy} = \text{PERMY}$, $k_{zz} = \text{PERMZ}$, $k_{xy} = \text{PERMXY}$, $k_{yz} = \text{PERMYZ}$ and $k_{zx} = \text{PERMZ}$.

The flow equations arising from the permeability tensor description are constructed using a Multipoint Flux Approximation (MPFA). The transmissibilities are calculated by the control volume discretization due to [Aavatsmark et al \(1998\)](#). The MPFA can only be used in corner point geometry, not block centered geometry.

The discretization scheme is selected by the `MPFA` keyword. The MPFA is a rigorous treatment of both cell non-orthogonality and tensor permeability. Even for a diagonal isotropic tensor, the MPFA will generally

lead to a 9 point discretization in 2D in order to remove the bias of grid non-orthogonality and grid orientation effects.

The MPFA discretization is applied by default across the entire grid. However, it can be applied in selected regions by using the `MPFANUM` keyword. The MPFA is only applied where the `MPFANUM` array equals 1. Further control may be achieved by applying the discretization only within nominated local grids using the MPFA keyword within a `CARFIN` and `ENDFIN` pair.

The MPFA can also be used on unstructured grids generated by FloGrid. The multipoint flux non-neighbor connections are exported to Eclipse through the `MPFNNC` keyword. Further details can be found in [Gunasekera et al \(1998\)](#).

The use of the MPFA option is not allowed with the WARP solver. Please use the pre-2002A solver (ECLSOL) instead by setting [item 66](#) of the `OPTIONS3` keyword to a non-zero value.

Discretization

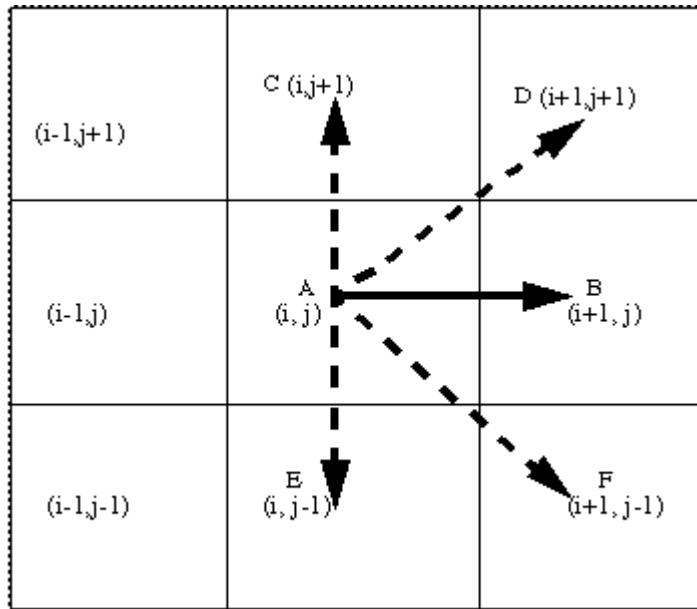


Figure 2.4. Stencil for MPFA flow in 2D between cells (i, j) and $(i + 1, j)$

Referring to "Formulation of the equations", the standard two-point flow between cells $A = (i, j)$ and $B = (i + 1, j)$ for the corner point grid in figure 2.4 can be written for a component c embedded in a phase p ($p = o, w, g$) as

$$F_{pAB}^c = T_{AB} M_{pAB}^c dP_{pAB} \quad \text{Eq. 2.29}$$

where

T_{AB} is the two-point transmissibility between cells A and B

M_{pAB}^c is the generalized mobility of component c in phase p , and is given by equation 10.37

dP_{pAB} is the potential difference of phase p between cells A and B, given by

$$dP_{pAB} = P_A - P_B + P_{cpA} - P_{cpB} - \rho_{pAB}g(D_A - D_B) \quad \text{Eq. 2.30}$$

and where

P_{cp} is the capillary pressure for phase p

ρ_p is the mass density of phase p

g is the acceleration due to gravity

D is the cell center depth.

Alternatively, the MPFA constructs the flow F_{pAB}^c which depends on steering terms from the neighboring cells C, D, E and F in figure 2.4 as

$$F_{pAB}^c = M_{pAB}^c \left(T_{AB} dP_{pAB} + \sum_S T_{AS} dP_{AS} \right) \quad \text{Eq. 2.31}$$

where the summation is over *steering* transmissibilities S between cells AC, AD, AE and SF.

The mobility M_{pAB}^c is evaluated in the upstream cell (A or B) according to the direction of flow in equation 2.31.

The steering potential differences dP_{AS} are evaluated using pressures and capillary pressures for cells A and S but using the density term ρ_{AB} for the primary flow coefficient obtained from equation 10.47, so that

$$dP_{pAS} = P_A - P_S + P_{cpA} - P_{cpS} - \rho_{pAB}G(D_A - D_S) \quad \text{Eq. 2.32}$$

The transmissibilities T_{AB} and T_{AS} are calculated following the procedure described in [Aavatsmark et al \(1997\)](#) when the permeability $\underline{K}$ is a full tensor.

Note that the flows between adjacent cells depend on steering terms from adjacent cells, but there are no flows created between cells which are diagonal neighbors as in the [NINEPOINT](#) scheme (see "[Nine-point schemes](#)").

Similarly in 3D, the flow between any two cells may depend on up to 16 steering transmissibilities and this leads to a 19 or 27 point scheme.

In the present version of Eclipse 300, the MPFA transmissibilities are not calculated across faults, but the two-point connections given by the [NEWTRAN](#) transmissibility calculation are used instead (see "[Transmissibility calculations](#)").

If the [MPFA](#) keyword is used without supplying off diagonal tensor permeabilities PERMXY , PERMYZ and PERMZX , these terms are taken to be zero and the MPFA scheme is applied with the diagonal tensor $\underline{K} = \text{diag}(\text{PERMX}, \text{PERMY}, \text{PERMZ})$ in the specified basis.

Orthogonality error

The MPFA or nine point discretization reduces to a standard two point flux scheme (that is, a five-point scheme in 2D or a seven-point scheme in 3D) if the grid is sufficiently orthogonal.

For example, the MPFA discretization reduces to 5- or 7-points when both

- the grid system is orthogonal and
- the permeability tensor $\underline{K}$ is diagonal

In general, when the grid is either non-orthogonal or $\underline{K}$ is a full tensor then a 9-point (2D) or 27-point (3D) discretization results. It is possible to control the application of the MPFA scheme based on a cell orthogonality error indicator. Essentially for grids which are nearly orthogonal it is desirable to apply the MPFA discretization only in regions of high non-orthogonality to reduce the CPU cost. In the context of a full tensor permeability, an orthogonality indicator can be defined as follows:

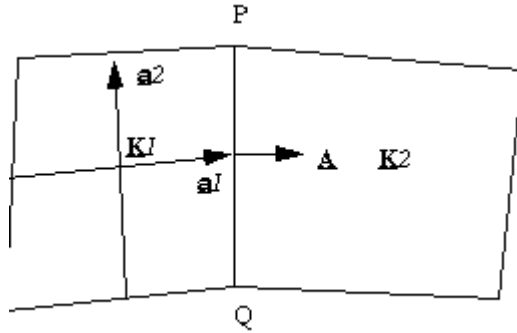


Figure 2.5. K -orthogonality error indicator

Consider an interface PQ between two cells 1 and 2. The permeabilities in each cells are $\underline{K}_1$ and $\underline{K}_2$. The directed area normal on the interface is denoted by $\underline{A}$. Let $\underline{a}_1$ and $\underline{a}_2$ be the vectors joining the midpoints of opposite faces of the cells. We call $\underline{a}_1$ and $\underline{a}_2$ the *covariant* basis vectors. Define the *contravariant* basis vectors $\underline{a}^1$ and $\underline{a}^2$ with the condition that

$$\underline{a}_i \cdot \underline{a}^j = \delta_{ij}$$

where δ_{ij} is the Kronecker delta.

Now consider a potential gradient in cell 1,

$$\underline{\nabla}\varphi = \sum_{j=1}^2 \alpha_j \underline{a}^j$$

The flux through PQ from cell 1 is therefore

$$f = \lambda \cdot \underline{A} \cdot \underline{K} \cdot \underline{\nabla}\varphi = \lambda \sum_{j=1}^2 \alpha_j \underline{A} \cdot \underline{K} \cdot \underline{a}^j$$

where λ accounts for the mobility.

Since this expression can be used for arbitrary values of α_j , it follows that if $\underline{A} \cdot \underline{K}$ is parallel to $\underline{a}_1$, such that $\underline{A} \cdot \underline{K} \cdot \underline{a}^2 = 0$, then the flux depends only on the gradient in the covariant direction $\underline{a}_1$ and implies that a two point discretization can be employed.

The grid is said to be *K-orthogonal* (see [Gunasekera et al \(1998\)](#) and [Aavatsmark et al \(1998\)](#)) when $\underline{A} \cdot \underline{K}$ is parallel to $\underline{a}_1$.

A convenient measure of orthogonality error for an interface is therefore

$$\varepsilon = \left\| \underline{A} \cdot \underline{K} - \frac{[(\underline{A} \cdot \underline{K}) \cdot \underline{a}_1] \underline{a}_1}{\|\underline{a}_1 \cdot \underline{a}_1\|} \right\| / \|\underline{A} \cdot \underline{K}\|$$

This is evaluated separately for each of the two cells adjoining the interface $\underline{A}$. The maximum of these two values provides a measure of the departure from orthogonality for the interface, that is, the departure from a two point flux approximation for this interface.

Keyword `ORTHERRO` may be used to specify two point fluxes for all interfaces with $\varepsilon < \text{ORTHERRO}$. The familiar 5-point (2D) or 7-point (3D) scheme then applies in regions when all interfaces have negligible orthogonality error. Alternatively using `ORTHERRX`, `ORTHERRY` and `ORTHERRZ` controls calculation of MPF fluxes for interfaces in the grid I, J and K directions.

Application of MPFA can also be controlled by a minimum pore volume limit using the `MINPORV9` keyword, so that cells with $PORV < MINPORV9$ use the familiar 5- point or 9-point discretization. The 5- or 9-point discretization (using `NEWTRAN` transmissibilities) is also used in regions for which `MPFANUM` is set equal to 0.

The keywords `ORTHERRO`, `ORTHERRX`, `ORTHERRZ` and `MINPORV9` can be individually applied to local grids if they are specified between the corresponding `CARFIN` and `ENDFIN` pairs.

Restrictions

The MPFA discretizations cannot at present be used with:

- The parallel option (keyword `PARALLEL`)
- Velocity dependent relative permeabilities (keyword `VELDEP`)
- Cell coarsening (keyword `COARSEN`)
- The Diffusion option (keyword `DIFFUSE`)
- The Thermal option (keyword `THERMAL`)
- Radial geometry (keyword `RADIAL`)
- The WARP solver (keyword `WARP`)

Local grid refinement and coarsening

x	Eclipse 100
x	Eclipse 300

The local grid refinement option allows enhanced grid definition near wells. The local models may be 2D radial, 3D radial or 3D Cartesian. Local models may have more layers than the global model. Transmissibilities between the local models and the global model are computed automatically by Eclipse. The properties of the cells in the local grid can be inherited from the global grid or specified explicitly for the refined cells.

Grid coarsening may be used to amalgamate cells in regions of the reservoir where accuracy is unimportant.

Eclipse 100

The auto-refinement facility may be used to refine an existing global model as the reverse process to grid coarsening.

Local grid refinement

Radial local grid refinements may be defined either in a single column of cells with the `RADFIN` keyword, or in a box of four columns with the `RADFIN4` keyword.

Radial refinement in a single column

To refine column (I=7, J=8) in layers 3 to 5 inclusive with a 3D radial grid with 6 radial segments, 4 angular segments and 18 vertical segments use:

```
RADFIN
-- NAME      I  J K1  K2 NR  NTHETA  NZ  NWMAX
'SOUTH'      7  8  3   5  6    4      18  /
INRAD
0.2 /
ENDFIN
```

in the GRID section. This inserts a local grid with $6 \times 4 \times 18 = 432$ cells. The `INRAD` keyword specifies the inner radius of the innermost block of the radial refinement.

This creates a refinement that looks like the following figure.

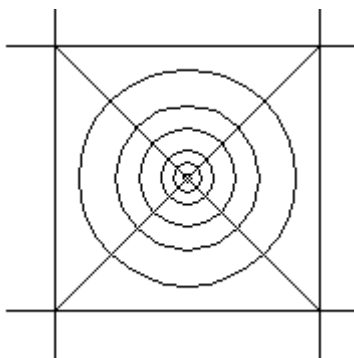


Figure 2.6. Single column radial grid refinement

Alternatively it is possible to use a two-dimensional model with `NTHETA` =1 that looks like the following figure.

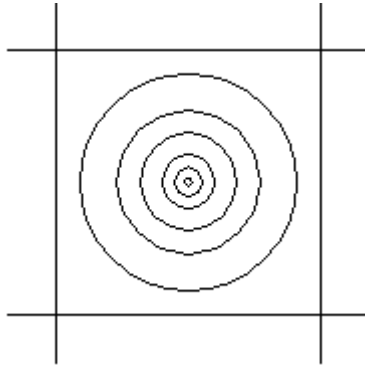


Figure 2.7. Single column radial grid refinement with $NTHETA=1$

By default, non-neighbor connections are generated automatically within the local grid to complete the circle connecting angular segment $ITHETA = 1$ to $ITHETA = 4$, unless circle completion is specifically disallowed by specifying 'INCOMP' in item 3 of the **COORDSYS** keyword entered between the **RADFIN** and **ENDFIN** keywords. The **ENDFIN** keyword tells Eclipse to finish reading GRID data for the local grid and to continue reading GRID data for the global grid. The permitted values of $NTHETA$ (the number of angular segments) are 1 for 2D radial refinements and 4 for 3D radial refinements. **NWMAX** is the maximum number of wells in the refinement. If **NWMAX** is not specified it is defaulted to 1, which is the normal value in a radial refinement.

To reserve memory for the refinement, set **MAXLGR=1** and **MAXCLS=432** in the keyword **LGR** in the **RUNSPEC** section, for example:

```
LGR
--MAXLGR  MAXCLS
      4      432 /
```

where **MAXLGR** is the maximum number of local grid refinements and **MAXCLS** is the maximum number of cells in any local grid.

Radial refinement in a box of four columns (Eclipse 100)

To refine the box of four columns ($I=7,8$; $J=8,9$) in layers 3 to 5 inclusive with a radial grid having 6 radial segments, 8 angular segments and 18 vertical segments use:

```
RADFIN4
-- NAME      I1  I2  J1  J2  K1  K2  NR  NTHETA  NZ      NWMAX
  'SOUTH'    7   8   8   9   3   5   6   8       18   /

INRAD
  0.2 /

ENDFIN
```

in the GRID data section. This inserts a local grid with $6 \times 8 \times 18 = 864$ cells. The **INRAD** keyword specifies the inner radius of the innermost block of the radial refinement.

This creates a refinement which looks like the following figure.

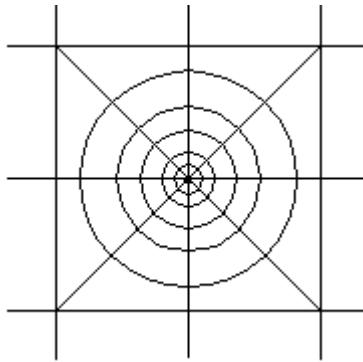


Figure 2.8. Radial refinement in a four column box

Alternatively, setting `NTHETA=4` creates a refinement which looks like the following figure.

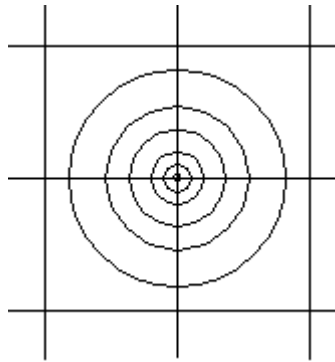


Figure 2.9. Radial refinement in a four cell box with `NTHETA=4`

By default, non-neighbor connections are generated automatically within the local grid to complete the circle connecting angular segment `ITHETA=1` to `ITHETA=8`, unless circle completion is specifically disallowed by specifying 'INCOMP' in item 3 of the `COORDSYS` keyword entered between the `RADFIN4` and `ENDFIN` keywords. The `ENDFIN` keyword tells Eclipse to finish reading `GRID` data for the local grid and to continue reading `GRID` data for the global grid. The permitted values of `NTHETA` (the number of angular segments) are 4 or 8. `NWMAX` is the maximum number of wells in the refinement. If `NWMAX` is not specified it is defaulted to 1, which is the normal value in a radial refinement.

To reserve memory for the refinement, set `MAXLGR=1` and `MAXCLS=864` in the keyword `LGR` in the `RUNSPEC` section, for example:

```
LGR
--MAXLGR  MAXCLS
      4      864 /
```

Placing wells in local grids

To place a well in a local grid, use keywords `WELSPECL` and `COMPDATL` instead of `WELSPECS` and `COMPDAT`. `WELSPECL` is similar to `WELSPECS` except that there is an additional item - the name of the local grid - between the group name and the I-J location. `COMPDATL` has exactly the same items as `COMPDAT`. In these keywords the I, J and K locations of the wellhead and the connecting cells refer to the cell coordinates in the local grid system. The I and J locations of the connecting cells must be entered explicitly in `COMPDATL`, and should not be defaulted as they can be in `COMPDAT`.

To complete well PRODUCER in layers 9 to 11 of the refined grid named 'SOUTH', in all four theta sections of the innermost block:

```

WELSPECL
-- WELL    -- GROUP -- LOCAL GRID -- LOCATION -- BHP -- PREF
-- NAME    -- NAME  -- NAME    -- I    J    -- DEPTH -- PHASE
'PRODUCER' 'G'      'SOUTH'    1    1    8060  'OIL'  /
/
COMPDATL
-- WELL    -- LOCAL  -- LOCATION  -- OPEN  -- SAT  -- CONN -- WELL
-- NAME    -- GRID   -- IX JY  K1 K2 -- SHUT  -- TAB  -- FACT -- DIAM
'PRODUCER' 'SOUTH'  1  1  9  11 'OPEN'  0    -1    0.4  /
'PRODUCER' 'SOUTH'  1  2  9  11 'OPEN'  0    -1    0.4  /
'PRODUCER' 'SOUTH'  1  3  9  11 'OPEN'  0    -1    0.4  /
'PRODUCER' 'SOUTH'  1  4  9  11 'OPEN'  0    -1    0.4  /
/

```

Eclipse 300 allows wells to be completed in any combination of local grids. With Eclipse 100 a well cannot be completed in a cell outside its local grid. However, it is possible to complete a single well in two or more Cartesian local grids by amalgamating these grids into a single entity (see ["Amalgamating Cartesian local grids \(Eclipse 100\)"](#)).

Wells not completed in local grid systems should have their specification and connection data set with the keywords WELSPECS and COMPDAT as normal. Eclipse checks that these wells are not completed in a cell that is host to a refined grid system.

Cartesian refinements

Cartesian refinements are specified by keyword [CARFIN](#). For example, to replace layers 3 and 4 in column (2,3) of the global model with a 5*5*8 Cartesian refinement with new porosities and permeabilities:

```

CARFIN
-- NAME    I1-I2 J1-J2 K1-K2 NX NY NZ NWMAX--
'LGR1'    2  2  3  3  3  4  5  5  8  5    /
PORO
200*0.2 /
EQUALS
'PERMX' 500 /
'PERMY' 500 /
'PERMZ' 50 /
/
ENDFIN

```

This creates a refinement that looks like the following figure (in the areal direction).

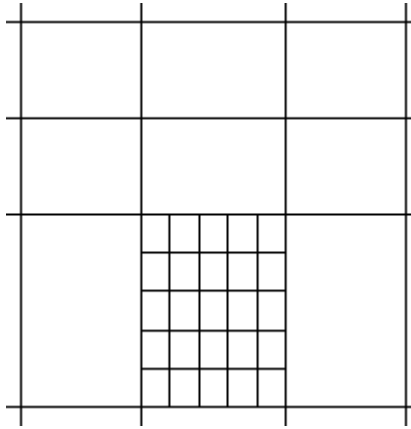


Figure 2.10. Cartesian grid refinement

It is recommended that you insert `ENDFIN` between successive `RADFIN`, `RADFIN4` and `CARFIN` keywords. The primary purpose of `ENDFIN` is to revert Eclipse to reading data for the global grid system.

Note that the `CARFIN` keyword is designed to refine a general box, rather than just a single column of cells as in `RADFIN` or a box of four columns of as in `RADFIN4`. The maximum number of wells within the Cartesian refinement is specified by `NWMAX` (=5 in the example above).

To place wells in a Cartesian refined grid, follow the same procedure as described above for radial grids. For example, to place a horizontal well that fully penetrates the refined grid from the x-direction, use the following procedure to complete the well in five horizontally-adjacent cells in the refined grid:

```
WELSPECL
-- WELL -- GROUP -- LOCAL GRID -- LOCATION -- BHP -- PREF
-- NAME -- NAME -- NAME -- I J -- DEPTH -- PHASE
  'HORIZ'  'G'      'LGR1'      1 3      8060  'OIL'  /
/
COMPDATL
-- WELL -- LOCAL -- LOCATION --- OPEN - SAT - CONN - WELL - KH - S - D - PEN
-- NAME -- GRID -- IX JY K1 K2 -- SHUT - TAB - FACT - DIAM - - - - DRN
'HORIZ'  'LGR1'  1 3 3 3 'OPEN' 0 -1 0.4 3* 'X' /
'HORIZ'  'LGR1'  2 3 3 3 'OPEN' 0 -1 0.4 3* 'X' /
'HORIZ'  'LGR1'  3 3 3 3 'OPEN' 0 -1 0.4 3* 'X' /
'HORIZ'  'LGR1'  4 3 3 3 'OPEN' 0 -1 0.4 3* 'X' /
'HORIZ'  'LGR1'  5 3 3 3 'OPEN' 0 -1 0.4 3* 'X' /
/
```

Defining local grid block sizes

Most `GRID` section keywords may be inserted between `CARFIN`, `RADFIN` or `RADFIN4` and `ENDFIN` to alter refined grid block properties from those of the host cells. Alternatively, grid data for local grids may be input using the `REFINE` keyword if the local grid has previously been defined using `CARFIN`, `RADFIN` or `RADFIN4`.

If [DX](#), [DY](#) or [DZ](#) are specified for the local grid then Eclipse checks for consistency with the dimensions of the global host cells. Local grid block sizes can also be defined in terms of fractions of the host grid using the following keywords, which are especially useful if the global grid has been defined in corner geometry:

[NXFIN](#), [NYFIN](#), [NZFIN](#) Define number of local cells in each host cell

[HXFIN](#), [HYFIN](#), [HZFIN](#) Define ratios for splitting the host cell into local blocks.

For example, to define a local grid of 5 layers in a host of 2 layers:

```
CARFIN
'LGR2' 3 5 1 2 4 5 6 4 5 /
NZFIN
3 2 /
HZFIN
1.0 2.0 3.0
2.0 1.0 /
```

This gives 3 local layers in the top host layer, with [DZ](#) values in the ratio 1:2:3 and 2 local layers in the bottom host layer, with [DZ](#) values in the ratio 2:1. For example, if the host layer thicknesses are 60 ft. and 30 ft., local [DZ](#) values are 10 ft., 20 ft., 30 ft., 20 ft., 10 ft.

This option could be used to set up a logarithmic refinement, for example for a Cartesian refinement:

```
HXFIN
1.0 10.0 100.0 /
```

Note that the numbers used in the [HXFIN](#), [HYFIN](#), [HZFIN](#) data could correspond to actual [DX](#), [DY](#), [DZ](#) values, but these need not fit exactly in the host cell, as they are adjusted automatically. If no ratios are given, the default is to split the host cell in equal proportions. Data may be defaulted for some or all host cells, but data must be complete if given for a particular host cell. For example, to divide a host grid of 3 layers into 7 local cells, with splitting ratios defined for the middle layer:

```
NZFIN
3 2 2 /
HZFIN
3* 1.0 2.0 2* /
```

A logarithmic Cartesian refinement can also be set up using any combination of the [LXFIN](#), [LYFIN](#) and [LZFIN](#) keywords. These will automatically generate suitable equivalent values for the corresponding [N*FIN](#) and [H*FIN](#) keywords, based on the user's specified central (fracture) cell thickness and preferred geometry type. For example, to generate a logarithmic grid in the y-direction with a central cell width of 1.5:

```
LYFIN
1.5 LINEAR /
```

The equivalent [N*FIN](#) and [H*FIN](#) keywords will be output to the PRT file, and can be modified and used to replace the [L*FIN](#) if adjustment is required.

The keywords [HZFIN](#) and [NZFIN](#) can also be used in radial refinements declared by [RADFIN](#) or [RADFIN4](#), to define the vertical splitting. [INRAD](#) should always be specified for radial refinements. If [OUTRAD](#) is specified after [RADFIN](#) or [RADFIN4](#) then its value applies to the inner radius of the outermost cell of the local grid, because the outermost cell assumes the odd shape required to fit into the Cartesian host cell. If [DR](#) or [DRV](#) values are specified after [RADFIN](#) or [RADFIN4](#) then the sum of [INRAD](#) and the [DR](#) values must be less than the [DX](#) and [DY](#) values of the host cells so that the refinement fits within the host cell dimensions. An incomplete set of [DR](#) or [DRV](#) values may be provided for a local grid and any missing data for the outer rings is completed automatically as a geometric series (Eclipse 100), or logarithmic series

(Eclipse 300) to fit within the host block. Rather than supplying the DR values directly, you can instead define relative sizes for the DR values using the [HRFIN](#) keyword:

```
RADFIN
--NAME  I1 I2 J1 J2 K1 K2 NR NT NZ
'LGR3'   6  6  4  4  1  5  5  4  20 /
INRAD
0.25 /
OUTRAD
3000.0 /
HRFIN
1.0 2.0 3.0 4.0 /
```

This gives $NR - 1$ linearly increasing DR values scaled such that their sum is equal to OUTRAD. HRFIN could be useful when the default logarithmic distribution creates tiny cells near the well bore, which can lead to convergence problems.

If the global grid is corner-point, then the [ZCORN](#) and [COORD](#) keywords may be used between the [CARFIN](#) / [ENDFIN](#) and [RADFIN](#) / [ENDFIN](#) pairs for example to provide finer control over the local grid geometry. If multiple reservoirs are used (see RUNSPEC section keyword [NUMRES](#)) then local grids cannot straddle two or more reservoirs; only one reservoir is allowed within each local grid. In order to model a local grid straddling two reservoirs, you should specify several adjacent constituent local grids stacked vertically which can then be amalgamated using the [AMALGAM](#) keyword (see below). If the local grid geometry is input in this way, the local corner points should be consistent with those of the host cells. Eclipse issues an error message if this is not so.

Defining local grid rock properties

By default, values of porosity, permeability and transmissibility multipliers, for example, are copied to local grid cells from the global host cells. In most cases this is adequate, but it is possible to define properties directly for some or all of the cells within a local grid by specifying grid arrays between the [CARFIN](#) / [ENDFIN](#), [RADFIN](#) / [ENDFIN](#), [RADFIN4](#) / [ENDFIN](#) or [REFINE](#) / [ENDFIN](#) pairs. When the end of the GRID section is reached, any values that have not been explicitly defined for local cells are then copied from the global host cells. For example:

```
CARFIN
-- NAME      I1-I2  J1-J2  K1-K2  NX  NY  NZ  NWMAX
'NORTH'     5 10    5 10    1  4  18  18  8   5 /
EQUALS
'PORO'      0.3   1 18  1 18  1  4 /
'PERMZ'     50.0  1 18  1 18  1  4 /
'NTG'       0.9   1 18  1 18  1  8 /
/
MULTIPLY
'PORO'      0.5   6 13  6 13  1  2 /
'NTG'       0.2   6 13  6 13  1  2 /
/
ENDFIN
```

This defines the porosities only in the top four layers on the local grid, and these are subsequently modified for the top two layers with the [MULTIPLY](#) keyword. The porosities for cells in the bottom four layers are copied from their host cells in the global grid.

Since by default Eclipse does not copy any properties from host cells until the end of the GRID section is reached, it is not permitted to modify any property for cells within the local grid (by using the [ADD](#), [COPY](#), [MULTIPLY](#) or [OPERATE](#) keywords) without previously defining the property for those cells.

This behavior can be changed by specifying the [LGRCOPY](#) keyword in the RUNSPEC, GRID, EDIT, PROPS or REGIONS section, depending on the simulator used. This will cause global values to be copied

‘just in time’ to the corresponding local grid cells. There are limitations on the use of this associated with the order of the keywords in the data file - for instance, global values of `DX` must have been set before the LGR operations are carried out in order for the correct global parent cell to be allocated to each LGR cell. If `DX` is redefined for the global cells after an operation requiring the `LGRCOPY` functionality to be used has been invoked, this will trigger an error. Similar limitations apply to all keywords which may affect which global cell a particular LGR cell is physically located in.

Special care should be taken with the `MINPV` keyword. It is likely that a suitable minimum pore volume for the global grid will not be suitable for much smaller local grid cells. A more suitable value should be specified for them using `MINPVV`. In Eclipse 100, if this is set for the host global cells it will be copied to the LGR cells. In Eclipse 300, this is not the case and it must be specified directly for the local grid cells.

Amalgamating Cartesian local grids (Eclipse 100)

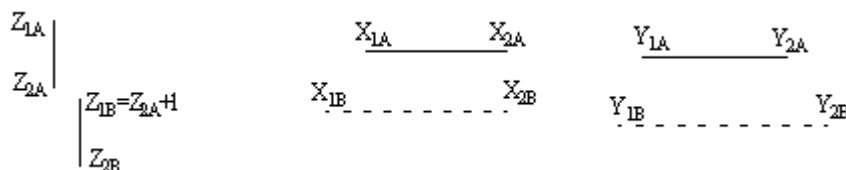
Eclipse 300 allows local grids to be placed anywhere in the reservoir provided they do not overlap. In Eclipse 100 a local grid cannot be placed next to another local grid.

It is however possible to amalgamate two or more Cartesian local grids into a single entity, by using the keyword `AMALGAM`. This facility provides a means of completing a well in more than one local grid. Thus the grid may be refined more efficiently around a deviated well by employing a number of smaller refinements in a zigzag pattern instead of a single large refinement.

Local grids within the same amalgamation are allowed to touch, but must not overlap. Several geometrical configurations allow amalgamation. Using the notation of the drawing below:

- Z-direction (A above B): If $Z_{1B} = Z_{2A} + 1$ then an amalgamation is defined if $X_{1B} \leq X_{2A}$ and $X_{1A} \leq X_{2B}$ and $Y_{1B} \leq Y_{2A}$ and $Y_{1A} \leq Y_{2B}$. This case corresponds to the drawing below. Swap indices A and B for the case B above A.
- X-direction (B on the right side of A): $X_{1B} = X_{2A} + 1$ then an amalgamation is defined if $Y_{1B} \leq Y_{2A}$ and $Y_{1A} \leq Y_{2B}$ in the case where NNCs and NEWTR are set. If no NNCs or NEWTR is set, then the following condition on the Z-coordinates is used: $Z_{1B} \leq Z_{2A}$ and $Z_{1A} \leq Z_{2B}$. For B on the left of A swap indices A and B.
- Y-direction: same as X-direction.

Note that this test is done for all pairs of active LGRs where a Flux Boundary (`USEFLUX`) is not used. This includes active and inactive cells.



Any valid Cartesian grids may be amalgamated, even if they do not touch. Eclipse treats an amalgamation as a single entity by lumping together all its constituent local grids, with non-neighbor connections between any adjacent grids in the amalgamation. The amalgamation is handled in the same way that Eclipse treats a single local grid; thus the timestep is synchronized across all the local grids within an amalgamation. Use of this facility to mimic nested refinements is not encouraged because this leads to a large number of non-neighbor connections between local grids. A well may be completed in any local grid within the same amalgamation, but may not be completed outside its amalgamation. Wells within local grid amalgamations should be specified in the usual way with keyword `WELSPECL`, naming any of the constituent local grids in

item 3. The keyword `COMPDATL` should be used instead of `COMPDAT` to specify connection data for wells completed in local grids.

For a well in an amalgamation, keyword `COMPLMPL` should be used instead of `COMPLUMP` to lump connections for automatic workovers. Similarly, `WELOPENL` should be used instead of `WELOPEN` to open or shut wells or well connections; and `WFRICTNL` must be used instead of `WFRICTN` for the Eclipse 100 Wellbore Friction option. The '-L' versions of the keywords each contain an extra data item to specify which local grid the following connection belongs to. Either keyword from each pair may be used for wells completed in a single local grid.

For example, consider defining a deviated well in neighboring local grids 'WEST' and 'EAST'. The `GRID` section might look like:

```

CARFIN
-- NAME      I1-I2  J1-J2  K1-K2  NX  NY  NZ  NWMAX--
'WEST'      3  4    2  3    1  2    8  4  4  1  /
CARFIN
-- NAME      I1-I2  J1-J2  K1-K2  NX  NY  NZ  NWMAX--
'EAST'      4  5    4  5    1  2    8  4  4  1  /
AMALGAM
'EAST' 'WEST' /
/

```

This generates a composite refinement which looks like the following figure (in the areal direction).

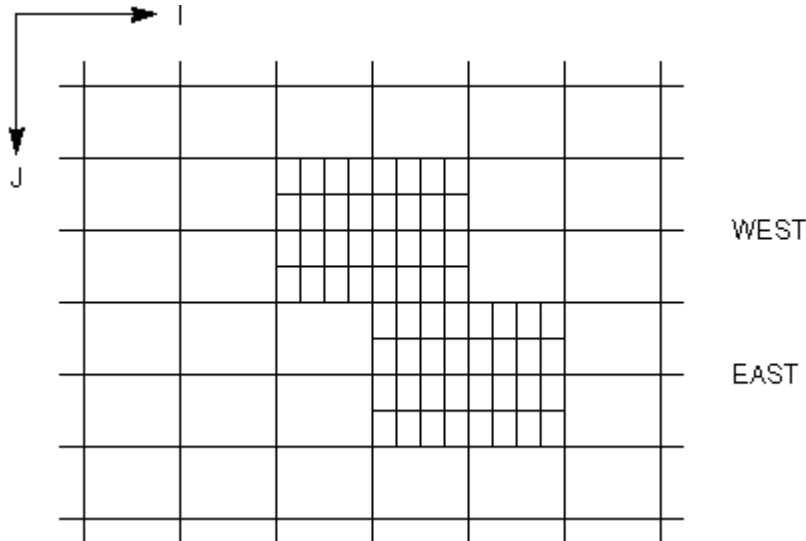


Figure 2.11. A composite refinement

A well may be specified in the `SCHEDULE` section in the following way:

```

WELSPECL
-- WELL  -- GROUP -- LOCAL GRID -- LOCATION -- BHP  -- PREF
-- NAME  -- NAME  -- NAME    -- I  J  -- DEPTH -- PHASE
'DEV'    'G'      'EAST'    2  2    8000  'OIL' /
/
COMPDATM
-- WELL - LOCAL - LOCATION - OPEN - SAT - CONN - WELL - KH - S - D - PEN -
-- NAME - GRID  - I  J  K1  K2 - SHUT - TAB - FACT - DIAM          - DRN -

```

```
'DEV'   'WEST'  6  3  3  3  'OPEN'  0   -1   0.3   3*           'X' /
'DEV'   'WEST'  6  4  2  2  'OPEN'  0   -1   0.3   3*           'X' /
'DEV'   'EAST'  2  1  2  2  'OPEN'  0   -1   0.3   3*           'X' /
'DEV'   'EAST'  2  2  1  1  'OPEN'  0   -1   0.3   3*           'X' /
/
```

Editing local grids

It is possible to modify data for local grids in the `EDIT` section by inserting modifications between a `REFINE` / `ENDFIN` pair, just as for the global grid, in order to modify pore volume, depth and transmissibility data. This is not recommended for general use and should be used with care. For example:

```
REFINE
'NORTH' /
EQUALS
'PORV' 1000.0 1 18 1 18 1 8 /
'TRANZ' 0.0 1 18 1 18 4 4 /
/
ENDFIN
```

edits `PORV` and `TRANZ` within the local grid `NORTH`.

If the pore volume is edited for any cells in the global grid that have been refined, then Eclipse carries over the modification to local cells by applying an additional pore volume multiplier to the local grid, which is defined as the ratio of edited to unedited pore volumes for the global host cells. Similarly, when transmissibilities are edited on the global grid, Eclipse imposes an additional transmissibility multiplier to the local grids which is determined in the same manner. This additional transmissibility multiplier applies to regular as well as to non-neighbor connections within the local grids, and also to connections between the local and global grids.

Grid and solution reports

Just as `WELSPECL` and `COMPDATL` (or `COMPDATM`) replace `WELSPECS` and `COMPDAT` for refined wells, so `RPTGRIDL` replaces `RPTGRID` in the `GRID` data section grid reports for all local grids. `RPTGRID` may also be used in the normal way to produce global grid reports.

Eclipse 100

Refined cells have the decimal point replaced by `R` in reports output to the `PRT` file using the `RPTGRID`, `RPTSOL`, and `RPTSCHED` keywords.

Tuning

Eclipse 100

Eclipse 100 allows the local grid refinements to be run with a completely different timestepping scheme from the global grid. If separate tuning parameters are required for local grids, use `TUNINGL` and/or `TUNINGS` in the `SCHEDULE` section. `TUNINGL` supplies a common set of tuning parameters for all local grids. In addition, specific sets of tuning parameters can be specified for individual local grids with the `TUNINGS` keyword. The `TUNING` keyword may be used in the usual way for the global system. `NEXTSTPL` can be used in place of `TUNINGL` to restrict the length of the next timestep in all local grids.

Timing reports

Eclipse 100

Extra timing arguments have been added to the timing reports. These reports are initiated by using argument `CPU` (12th control) in `RPTSCHED`.

- CPSU Calculation of the local component of the global matrix.
- TLGR Solving the local grid systems.

Vertical equilibrium (Eclipse 100)

Vertical equilibrium may be used for the global grid system, with either dispersed flow or VE in the local grid systems. The vertical equilibrium model is activated in a local grid system by using the `VEFIN` keyword. The relative fraction of the VE and the rock relative permeability curves can be modified on a local cell by cell basis using the `REFINE` and `VEFRACV` keywords in the `PROPS` section.

The global host cells are described by tables of distinct depths versus volumes, that correspond to the distinct corner depths in the local grid. This accurate description of the global cell prevents inconsistencies between the global and local systems that could lead to jumps in the global contact depths. However, if the number of local cells in a global host cell is large, the cost of setting up a large table of distinct depths can be high. To keep the cost down, you can specify a smaller number of distinct depths for the table. The number of distinct depths in the table for each global host cell can be controlled by setting `NVEPT` in the `VEFIN` keyword. Defaulting this item causes the original description of the global host cells to be used (with up to 8 distinct depths in the table).

The compressed VE option may be used to run a global 3D model as a 2D VE areal run, with dispersed flow in the local 3D grids. To do this, use the keyword `VE` in the `RUNSPEC` section.

Within an LGR amalgamation, all the constituent local grids should employ the same physical model (dispersed flow or VE).

Good practice in designing LGRs

When considering a grid refinement it is good practice to conduct sensitivity studies comparing alternative grids. This allows the engineer to understand the choices made between execution speed and accuracy of results. It is particularly important to ensure adequate resolution of areas where solution variables are likely to vary rapidly, such as oil/water or gas/water contacts in the region of a well. When determining these areas consider the connectivity between cells around an LGR; for example, transmissibility between cells in the vertical direction can be larger than other directions due to the aspect ratio of cells and consequently full depth LGRs may be more important than wider areal extent.

Dual, multiple and triple porosity

In dual, multiple and triple porosity runs, the global coordinates specified in `CARFIN` and `RADFIN` refer to the (outer, in the case of multi-porosity) matrix blocks only. For example, in dual porosity, in an 8*8*6 global grid, to refine the matrix cell at (8, 8, 2) together with its corresponding fracture cell at (8, 8, 5) use;

```
CARFIN
--          I1 I2 J1 J2 K1 K2 NX NY NZ NW
'CARF882'  8  8  8  8  2  2  5  5 10 3 /
ENDFIN
```

The refinement has 5*5*5 matrix cells and 5*5*5 fracture cells. For multiple and triple porosity, the same principle applies, namely that the global grid z- or k-extents (K1 and K2, data items 6 and 7) are always defined for the matrix grid only, but the size of the local grid in the z- or k-direction (NZ, data item 10) covers all porosities. To find its size in one porosity only (for example in the matrix) it must be divided by `NPOROS`, which is `NMATRIX+1`, where `NMATRIX` is the number of matrix porosities (for multi porosity, it is data item 1 of keyword `NMATRIX`, or it is 1 for dual porosity), or the number of matrix and vug porosities

(for triple porosity, it is data item 1 of keyword [TRPLPORO](#)). Conversely, NZ must be divisible exactly by NPOROS.

Eclipse 100

If the keyword [DPNUM](#) has been used to specify regions of the reservoir for which a conventional single porosity model is to be used in a dual porosity run, then the local grid may not straddle different [DPNUM](#) regions. A local grid in a dual porosity run must either lie entirely within a dual porosity region or entirely within a single porosity region. Within an LGR amalgamation, all the constituent local grids must be in the same [DPNUM](#) region.

Unstructured local grid refinements

Unstructured (PEBI) grids can now be run as local grid refinements. The [EXTFIN](#) keyword can be used to specify an unstructured local grid refinement. The [EXTHOST](#) keyword can be used to specify the global host cells for an unstructured local grid refinement, or similarly, the [EXTREPGL](#) keyword can be used to specify the replaced global cells for an unstructured local grid refinement. Finally, the [TRANGL](#) keyword can be used to specify the global-local connections and associated transmissibilities for an unstructured local grid refinement.

Between any [EXTFIN](#)/[ENDFIN](#) bracket, either [EXTHOST](#) or [EXTREPGL](#) should be used, but not both. An external program generally sets these keywords and you do not need to change them.

Region data and end-point scaling

By default, cells in the local grid systems are assigned [REGIONS](#) section data values ([SATNUM](#) and [PVTNUM](#) for example) and end-point scaling data (if the option is active) from their corresponding global host cells. In many cases this will prove adequate.

It is possible to modify selected data quantities from the [REGIONS](#) and [PROPS](#) sections within local grids by inserting modifications between a [REFINE](#) / [ENDFIN](#) pair. Any data following a [REFINE](#) keyword is applied to the named local grid refinement until either another [REFINE](#) keyword or an [ENDFIN](#) keyword is encountered. Local grid quantities that can be set in this way include end point scaling data and vertical equilibrium option data. An example that sets [SATNUM](#) values in a local grid containing 250 cells is:

```
REFINE
'CARF882' /
SATNUM
125*1
125*2 /
ENDFIN
```

Switching refinements on and off (Eclipse 100)

Refinements may be switched on or off at any time during a simulation using keywords [LGRON](#) and [LGROFF](#) in the [SCHEDULE](#) data section. For example,

```
LGROFF
'SOUTH' /
```

switches off the refinement called [SOUTH](#), while

```
LGRON
  'NORTH' /
```

switches on the refinement called NORTH.

Refinements can be made to switch on and off automatically depending on the number of active (that is, open or stopped, but not shut) wells they contain. For example, using either

```
LGROFF
--NAME      NWELLS
  'SOUTH'   2      /
```

or

```
LGRON
--NAME      NWELLS
  'SOUTH'   2      /
```

causes the refinement called SOUTH to be on whenever it contains two or more active wells, and off whenever it contains less than two.

If LGROFF and LGRON are not used then all refinements defined in the GRID data section are activated throughout the entire simulation.

Switching refinements on part way through a run is not advised, in general, since there is no valid procedure for allocating material to refined cells except at initial conditions when equilibration is used. If a refinement is switched on during a simulation, material is distributed uniformly to the refined cells within each host cell.

It is possible to add extra local grids when restarting a run. This is analogous to switching on an LGR part way through a run, but allows an LGR to be added without re-running the base case. The new LGRs should be added to the GRID section after all the existing LGRs have been specified. The new LGRs are turned on at the next timestep, unless instructed otherwise with the LGROFF keyword. This facility allows you to add new LGRs into the grid where new wells are about to be opened. It is generally not permitted to add a new LGR around a well that already exists. If this is required, however, the existing well and all its connections must be shut before the first [TIME](#), [TSTEP](#) or [DATES](#) keyword is read in the restart run. In order to continue to model the well, a new counterpart well should be defined in the local grid (with a different name) using [WELSPECL](#) and [COMPDATL](#) or [COMPDATM](#), which has all the attributes of the existing shut well. Eclipse then ignores the existing global well, and an error condition results if the well is ever to be reopened. This procedure is not recommended in general.

Eclipse 300

Flexible restarts can be used in Eclipse 300 to add LGRs on a restart run, given that no LGRs existed in the base run. The new LGRs should be added to the GRID section (as normal). No other keywords are required. The new LGRs are turned on at the next timestep. It is important to note that when adding an LGR using a restart, refinement around an existing well is not possible. If this is the case, an error message is output.

Eclipse 300

Flexible restarts can also be used in Eclipse 300 to turn LGRs off in a restart run. Any LGRs found in the RESTART file of the base run are ignored by Eclipse 300. See also the [GETGLOB](#) and [GETSOL](#) keywords.

Summary files

In LGR runs, two sets of summary data are output. The usual SUMMARY files contain summary data produced at the global timesteps. In addition SUMMARY file quantities are output at the local timesteps; this local data is output to a separate LGR file. The [RPTONLY](#) keyword to limit the frequency of SUMMARY file output does not apply to the LGR file.

To save space, the creation of LGR files can be prevented by using the NOSUMLGR keyword in the SUMMARY section. When running LGR in-place (see "[Timesteps](#)"), there is little point in dumping SUMMARY files for the local timesteps since the global and local timesteps are all synchronized.

The local timestep summary data is stored in a file with the following name:

ROOT.FLGR (or ROOT.FLG)	Formatted
ROOT.LGR	Unformatted

See the [SUMMARY](#) section overview for information on how to write local timestep data to this file.

Transmissibility calculations

For radial local grid refinements, the standard method based on pressure equivalent radii is used for flow between local radial blocks. Flow between the outer local block and the adjoining host block is treated as linear, but with the 'transmissibility center' taken at the pressure equivalent radius of the local block, rather than the geometric center.

For Cartesian refinements, transmissibilities between the local and global grid are calculated from the usual formulae (described in "[Transmissibility calculations](#)") when the [NEWTRAN](#) method is used. However, when [OLDTRAN](#) or [OLDTRANR](#) is used for block-centered grids, then the representative area chosen for the transmissibility between local and global cells is the face area of the local grid cells. For example, for the transmissibility in the X-direction between a local cell *l* and a global cell *g*, when [OLDTRAN](#) is used

$$\text{TRANN}_{lg} = \frac{\text{CDARCY} \cdot \text{TMLTX}_l \cdot \text{DIPC}}{B} \quad \text{Eq. 2.33}$$

where

CDARCY is Darcy's constant in the appropriate units

TMLTX_{*l*} is the X+ transmissibility multiplier for global cell *g*

DIPC is the dip correction

and

$$B = \frac{1}{2A_l} \cdot \left(\frac{DX_g}{\text{PERMX}_g} + \frac{DX_l}{\text{PERMX}_l} \right) \quad \text{Eq. 2.34}$$

The area is given by

$$A_l = DY_l \cdot DZ_l \cdot \text{NTG}_l \quad \text{Eq. 2.35}$$

The dip correction is given by

$$\text{DIPC} = \frac{\text{DHS}}{\text{DHS} + \text{DVS}} \quad \text{Eq. 2.36}$$

with

$$DHS = \left(\frac{DX_l + DX_g}{2} \right)^2 \quad \text{Eq. 2.37}$$

and

$$DVS = (DEPTH_l - DEPTH_g)^2 \quad \text{Eq. 2.38}$$

Note: The convention used for transmissibility multipliers: For connections in the X-, Y-, Z- directions, the global value for the adjoining block is used. For connections in the X+, Y+, Z+ directions, the local grid block value is used.

Transmissibilities between local cells in adjacent amalgamated local grids are calculated from the usual formulae when the NEWTRAN method is used. When OLDTRAN or OLDTRANR is used, connections are possible between local cells on faces of adjacent refined cells in the global grid. The interface areas used for the calculation account for partial overlap. These overlap areas are obtained by projecting the refinements on the faces of adjacent refined cells in the global grid onto the unit square; then the proportion of cell areas in each overlap is applied as a multiplier to the actual local face areas in the OLDTRAN formula.

A report on transmissibilities between the local and global grids and between any amalgamated local grids can be obtained by using the mnemonic ALLNNC (switch 24) in [RPTGRIDL](#).

Treatment of transmissibility multipliers in local grids

It is possible to enter transmissibility multipliers [MULTX](#), [MULTY](#) and [MULTZ](#) for example for local grids between the [CARFIN](#) / [ENDFIN](#), [RADFIN](#) / [ENDFIN](#), [RADFIN4](#) / [ENDFIN](#) and [REFINE](#) / [ENDFIN](#) pairs in the GRID section. If any of these multipliers are not defined for the local grid, they are inherited from the host cells of the global grid according to the following rule: The multiplier on the transmissibility between two adjacent local grid cells is set to the global multiplier in the appropriate direction at a cell boundary of the global grid, and is set to 1.0 otherwise. When the local [MULTR](#) and [MULTTHT](#) are undefined within a radial local grid, these are inherited from the appropriate global [MULTX](#) and [MULTY](#) respectively.

Timestepping

Local timestepping (Eclipse 100)

Efficiency is achieved by solving each local grid (or amalgamation of local grids) individually using “local timestepping”. This allows the local grids to have shorter timesteps than the global grid, in order to resolve coning effects for example, without holding up the progress of the global grid simulation. Each local grid may require several timesteps to reach the end of a global timestep. This is very efficient in cases where the local grids naturally require much shorter timesteps than the global grid.

Local timestepping has some disadvantages, however. The global grid and each of the local grids may individually be solved fully implicitly, but since both the global and local grid solutions provide boundary conditions for each other, a semi-explicit coupling is required. Thus the coupled solution is not fully implicit and may even exhibit instabilities in some cases. Also, local timestepping can cause group flow rates to drift significantly away from their group control targets. The approach of local timestepping is the default method for solving local grids and was the only method available in versions of Eclipse 100 prior to 97A.

When local timestepping is used, the sequence for calculating each global timestep is as follows:

1. The global model is advanced through a new timestep in the usual way. At this stage the local grids are represented by their host cells in the global grid. Wells in refined grids are solved using the grid block properties of the refined system. Properties of host cells are computed using local grid information.
2. Each local simulation is then advanced through a series of local timesteps that are often smaller than the global timestep. The local timesteps are chosen to synchronize with the end of each global timestep.
3. Flows between the global and local systems are computed dynamically as boundary conditions to each local simulation using currently updated variables in the local grid and fixed variables in the global grid (linearly interpolated in time over the global timestep). Spatial interpolation of the pressure field may also be performed for Cartesian refinements to enhance accuracy.
4. The process is completed by updating well rates and material in the host cells of the global model to be consistent with the local models. The global model is then ready to advance to the next global timestep.
5. After the completion of a global timestep the material for each phase in each host cell is compared with the material sum for the corresponding local cells. Subsequent global timesteps are often selected to keep these material calculations within 2.5%. The convergence report summary uses the mnemonic LGRC followed by the name of the constraining LGR when this criterion is used. For example:

```
STEP 16 TIME= 116.32 DAYS (+3.5 DAYS LGRC/LGRWEST 4 ITS)
PAV= 3185.2
```

This timestep selection criterion may be altered using keyword [TUNING](#).

6. The first local grid specified within an LGR amalgamation is chosen to be the representative of the amalgamation. The convergence reports for each amalgamation are identified with this name.

In-place LGRs

In Eclipse 100, an alternative fully implicit solution method, “LGR in-place”, is available to overcome the shortcomings of local timestepping. The global and local grids are solved simultaneously and fully implicitly as a coupled system. This avoids any problems with stability or meeting group control targets associated with local timestepping. However, the coupled system chooses a timestep that is the minimum of the natural timestep over the global and all local grids. In cases where one LGR requires a timestep much smaller than the rest of the grid (say for a horizontal well in a Vertical Equilibrium model, or for a radial LGR), running the coupled system at this timestep may severely limit the efficiency of the simulation. Thus it is possible to nominate local grids individually to be solved in-place (keyword [LGRLOCK](#)). Any local grids not nominated as in-place are advanced using local timestepping. If all local grids are nominated as in-place, then a fully coupled implicit solution is obtained. This procedure is commonly adopted in Eclipse 300.

Eclipse 300

Eclipse 300 solves LGRs using the same timestep as the global grid. There is no local timestepping option.

Restrictions on the use of local grids

Eclipse 100

- Separate local grid refinements must not be placed in adjacent cells of the global grid, unless the refinements belong to the same amalgamation.
- Partially overlapping LGRs are not allowed.
- Keyword [NINENUM](#) is not compatible with the local grid refinement option.

- Eclipse 100*
- It is possible to define a refinement within another local grid (nested refinements), provided the parent grid is a Cartesian local grid. Nested refinements can only be solved “in place” and do not work with the parallel option. They are also not allowed if there are amalgamated local grids in the data set or with the vertical equilibrium option.
 - It is not currently possible to coarsen within a local grid.
- Eclipse 100*
- Radial local grids may not be amalgamated. Radial LGRs need to be separated by a minimum of non-LGR containing global cell.
- Eclipse 300*
- Adjoining radial LGRs are allowed.
 - Each local cell must be identified with a single global cell. It cannot be split between two global cells. The number of local cells in the X, Y or Z direction must be an integral multiplier of the global grid dimensions, unless sufficient local data are provided to allocate local cells to host cells according to size (for example: DX, COORD) or to define how the host cells are split up (for example: NZFIN, HZFIN).
 - Local grids may not straddle different reservoirs (if the number of reservoirs is set > 1 in RUNSPEC keyword [NUMRES](#)). To achieve this, separate local grids can be stacked vertically, one in each reservoir, and then amalgamated using the [AMALGAM](#) keyword.
 - Local grids may not straddle different flux regions (specified by keyword [FLUXNUM](#) for the Eclipse Flux Boundary facility). In dual porosity runs, local grids may not straddle the boundary between single porosity and dual porosity regions (specified using keyword [DPNUM](#)).
 - Numerical aquifers cannot be attached to local grids.
 - The diffusion options (molecular diffusion and environmental tracers) cannot be combined in local grid runs.
 - The [NNC](#) or [EDITNNC](#) keywords for the global grid should not refer to non-neighbor connections connecting refined cells in the global grid.
 - It is not possible to use the grid coarsening option when restarting a run.
 - Local grids cannot be added to a radial global grid.

Treatment of wells in local grids

Wells are placed in local grids by using keywords [WELSPECL](#) and [COMPDATL](#) instead of [WELSPECS](#) and [COMPDAT](#). The I, J and K locations of the wellhead and connections refer to the cell coordinates within the well’s refined grid system. In Eclipse 100 a well cannot be completed in any other cells outside its local refined grid or outside its amalgamation.

Eclipse 100

In other keywords relating to well connections, apart from [COMPLUMP](#), [COMPLMPL](#), [WELOPEN](#), [WELOPENL](#), [WPIMULT](#), [WPIMULTL](#), [COMPRPL](#), [COMPVEL](#), [WFRICTN](#) and [WFRICTNL](#), the coordinates of the host cells in the global grid must be used. The data specified for each connection in the global grid will be applied to all the well’s connections in the refined grid cells contained within the global grid connection cell.

Eclipse 300

In contrast to the above, Eclipse 300 permits specialized LGR keywords (ending in ‘L’) to be used when well connections need to be specified. The exception to this is when all the connection values are defaulted and the action of the keyword applies to all the well connections.

For wells in local grids, keywords [COMPLMPL](#), [WELOPENL](#), [WPIMULTL](#) and [WFRICTNL](#) should be used respectively to lump connections, open and close connections, apply PI multipliers to connections, and define friction wells. (In fact, the keywords [COMPLUMP](#), [WELOPEN](#), [WPIMULT](#) and [WFRICTN](#) may also be used for wells in single local grids, but they cannot be used for wells in amalgamated local grids.) In runs

employing the Vertical Equilibrium option, well connection depth data should be set, if required, using keyword `COMPVEL` rather than `COMPVE`. In each of these keywords, I, J and K coordinates within the respective local grid systems should be used to identify the connections.

Group control is applied to wells in the usual way when the global grid is advanced through each global timestep. When using local timestepping, the target rates allocated to the wells under group control are then held fixed while each local grid system is advanced to the end of the global timestep. If the flow rate of any local grid well changes over the length of the global timestep, the target rates of the other wells cannot be adjusted to accommodate the change. Thus the group flow target is not met exactly. The group flow discrepancy may be significant if a local grid well changes its flow rate significantly over the length of the global timestep. If this occurs, possible remedies are to limit the size of the global timesteps, use Coarsening instead of Local Grid Refinement, or run LGR in-place for those local grids containing wells that contribute to the group flow discrepancy.

Eclipse 100

When local timestepping is employed, economic limits applied to local wells and their connections (`WECON` and `CECON`) are checked at the end of each local timestep in the local grid system, and workovers for example, are performed as required. Group economic limits (`GECON`) and rate limits involving workovers (`GCONPROD`) are checked at the end of each global timestep. These may result in more workovers to local wells.

In automatic workovers, connections are closed one at a time. This may be on too fine a scale for a well in a refined grid. In particular, a well in a 3-D radial refinement could have one of its connections on a given layer closed while the other connections on that layer remain open - not a realistic situation. It is therefore advisable to lump the connections together into sets (or completions), which are the smallest unit that can be closed in an automatic workover. When a workover limit is exceeded, the program searches for the worst-offending completion, and close all its connections together. Keyword `COMPLMPL` may be used to lump the connections into completions. For example, consider the well 'PRODUCER' that is completed in layers 9 - 11 in the 3-D radial refined grid. To lump the four connections on each layer into one completion (making three completions in total), use:

```

COMPLMPL
-- WELL ---- LOCAL GRID ---- LOCATION ---- COMPLETION
-- NAME ---- NAME      --- I  J  K1 K2 -- NUMBER
' PRODUCER '   1*         2*   9  9       1      /
' PRODUCER '   1*         2*  10 10       2      /

' PRODUCER '   1*         2*  11 11       3      /

/

```

Current restrictions with local grid wells

Eclipse 100

- The keyword `GCONTROL` cannot be used in local grid refinement runs.

Eclipse 100

- Keyword `WLIMTOL` only works for economic limit checks on wells in active local grids. It is ignored in all other limit violation checks.

Eclipse 100

- In parallel runs, LGRs cannot be switched on (using the `LGRON` keyword) after they have been switched off.

Eclipse 100

- In parallel runs, wells cannot be declared inside of LGRs (using the `WELSPECL` keyword) after LGRs have been switched off.

Other problems with local grid wells

- When a well is under rate control, its potential flow rate in a local grid system can be much larger than it would be in the unrefined global grid. This is because the connecting grid blocks, whose pressures are used in the potential calculation, are much smaller in the local grid system. In calculating the potential,

the assumption is made that the pressures in the connecting grid blocks does not change if the well flow rate increases from its current value to its potential value. While this may be a reasonable assumption in the global grid, it is not true for highly refined local grids. The smaller the connecting grid blocks are, the more optimistic will be the potential.

Eclipse 100

- When group control targets are applied, there may be significant discrepancies between the actual group flow rate and the imposed rate target when local timestepping is employed. This is because group control is applied while solving the global grid, but not while solving the local grids. Group control is applied to wells in the usual way while solving the global grid. The target rates allocated to the wells under group control are then held fixed while each local grid system is advanced to the end of the global timestep. If the flow rate of any local grid well changes over the length of the global timestep, the target rates of the other wells cannot be adjusted to accommodate the change. Thus the group flow target is not met exactly. The group flow discrepancy may be significant if a local grid well changes its flow rate significantly over the length of the global timestep. If this occurs, possible remedies are to limit the size of the global timesteps, use Coarsening instead of Local Grid Refinement, or run all those local grids containing wells in the offending group(s) as in-place LGRs using keyword [LGRLOCK](#).

Parallel load balancing with LGRs

The strategy used to distribute local grids across processors is fundamentally different between Eclipse 100 and Eclipse 300. In Eclipse 300 refined cells are placed on the same processor as the corresponding host cell. An LGR could therefore be distributed across any number of processors. Eclipse 100 assigns complete LGRs to processes without regard to the ownership of the host cell. Host and fringe cell information is swapped between processors as required. Eclipse 100 instead attempts to balance the number of refined cells on each processor.

Gravity and grid effects

For grid cells in an LGR that connect to global grid cells at very different depths, gravity effects might cause the solution to look unusual. For example if the local grid is defined through all the layers, gravity effects might cause water to accumulate at the bottom layer of the LGR, while this is not seen if the LGR is removed. The reason is that there is a grid effect due to the difference in depth of the bottom layer of the local grid and the depth of the connecting global grid cells.

Geometry and grid data in LGRs

Transmissibility data

In general, if grid data are not specified for a local refinement (by inserting keywords between [CARFIN](#), [RADFIN](#) or [RADFIN4](#) and [ENDFIN](#)) then they are derived from the properties of the host cells. You should note the following conventions used for default local permeability and transmissibility data.

Permeabilities

Unless permeabilities are explicitly defined for a local radial grid, global [PERMX](#) data are copied to [PERMR](#) and global [PERMY](#) data are copied to [PERMTH](#). If the [PERMX](#) and [PERMY](#) values are different, a warning is issued by Eclipse.

Transmissibility multipliers

Transmissibility multipliers ([MULTX](#), [MULTY](#), [MULTZ](#)) are applied to the X+, Y+ and Z+ faces of each grid block. For a local refinement, multipliers can be copied from the host grid only if the positive face of the local cell coincides with a host cell face; otherwise, a default value of 1.0 is used. If this is inappropriate, transmissibility multipliers should be explicitly defined for the local grid.

Note: For connections between local and global cells, the transmissibility multipliers for the adjoining global block are used for flows in the X-, Y- or Z- directions. The local grid block multipliers are used for flows in the X+, Y+ or Z+ directions.

Pore volumes

The pore volume of a refined global cell may differ from the sum of the pore volumes of the local cells which it contains, either because the local porosities and net-to-gross ratios differ from the values for the host cell or because of discrepancies in geometry.

To avoid significant volumetric errors for radial grids, the geometric volumes of the outer refined blocks are adjusted to fit the host cell. However, local porosities and NTG values are not modified, so differences in global and local pore volumes may still occur.

After computing the local pore volumes, Eclipse replaces the pore volume of the host cell with the sum of the refined pore volumes. If a difference of more than 1% is detected, a warning message is issued. Pore volume reports are produced after any modifications have been made.

Active cells

The local cells identification, specified using the `ACTNUM` keyword, is inherited from the parent global cell. This property can be overwritten by inserting the `ACTNUM` keyword between `CARFIN`, `RADFIN`, or `RADFIN4` and `ENDFIN` only if the value for the global cell is 1.

Radial refinements

1. A local radial grid may be defined in either a single column of cells (using `RADFIN`), or in a box consisting of four columns of cells (using `RADFIN4`). The geometry of the local grid may be defined using one of the following methods:
 - a. Refinements in the R-direction may be defined by user-given `DR` values. In the Z-direction, refinements can be specified by `NZFIN` / `HZFIN` data. Missing data are calculated automatically, so that a radial grid can be defined by a minimum set of one `RADFIN` or `RADFIN4` keyword and one `INRAD` keyword.
 - b. In a block-centered host, `DZ` values may be input directly, with radii defined as for the case (a). Use of user-defined `DTHETA` values is not recommended, as the local angles are usually restricted by the host cell geometry.
 - c. In a corner-point host, a local radial grid may be defined by corner-point coordinates. Note that the local `COORD` data are expected in cylindrical coordinates, relative to an origin at the center of the refinement. (Note: Cartesian local `COORD` data are defined relative to the global grid origin.)
2. A local radial refinement defined through `RADFIN` can only be inserted in a single column of host cells, with `NTHETA` = 1 or `NTHETA` = 4. Such a radial grid defined by user-specified corner points (in a corner-point host) must be 3-dimensional (that is, `NTHETA` = 4).

A local radial refinement defined through `RADFIN4` can only be inserted in a box of four columns of host cells, with `NTHETA` = 4 or `NTHETA` = 8. Such a radial grid defined by user-specified corner points (in a corner-point host) must be 3-dimensional (that is, `NTHETA` = 8).

A local radial grid cell must fit inside a single host cell.

3. The center of a radial refinement defined through `RADFIN` is assumed to be at the geometric center of the top host block.

The center of a radial refinement defined through RADFIN4 is assumed to lie at the central node of the top of the refined box.

4. The sides of the inner radial cells are assumed to be vertical.
5. The innermost radius must be defined by INRAD.
6. The radius for the inside of the outer local block can be defined by OUTRAD. Otherwise it is calculated as $R_{MAX} / 3.0$ where R_{MAX} is the maximum radius of a circle contained within the radial refinement.
7. You may provide some or all DR values. Any missing radii are completed as a geometric series (Eclipse 100), as follows:

No. of radii = NR

No. of user-defined radii = NU

Radius of inside of outer local block = RREF = R_{NR-1}

Last user-defined radius = RU

Then for $i = NU + 1$ to $NR - 2$

$R_{i+1} = FAC \cdot R_i$

where $FAC = \left(\frac{RREF}{RU} \right)^{1/(NR-NU-1)}$

8. For local transmissibility calculations using RADFIN, the outer radius is taken as the area equivalent radius of the host grid block, based on the top area of the host cell. When RADFIN4 is used, the equivalent radius is based on the top area of the four host cells. For volumetrics and connections to the global grid, the sides of the local grid are taken from the coordinate lines for the host.

Note that the equivalent radii are shown in the output reports for the local COORD values.

9. The figure below shows the numbering convention used for connecting local radial grids to the host when RADFIN is used:

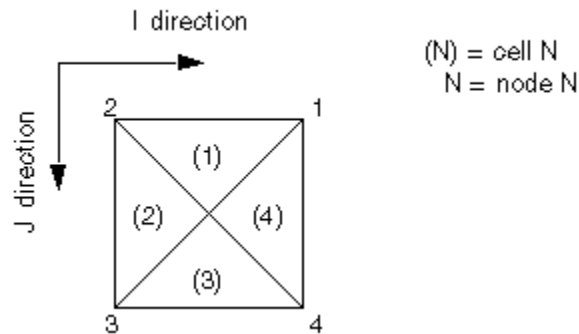


Figure 2.12. The numbering convention used by RADFIN for connecting radial LGRs to host cells

The radial cells are numbered so that Theta increases in an anti-clockwise direction, starting from the quadrant corresponding to the Y- face of the global cell. If necessary, the local coordinates are rotated to align the radial cell with its host.

Reported COORD data will show the corrected Theta values.

For a local radial refinement defined with RADFIN4, the topology is also arranged with theta increasing anti-clockwise, but starting from the middle of the X+ (I+) face of the refinement as shown in the following figure.

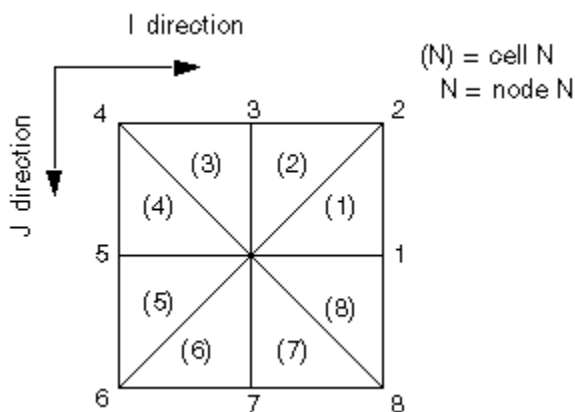


Figure 2.13. The numbering conventions used by RADFIN4 for connecting radial LGRs to host cells

In the above case, Theta increases anti-clockwise in (I,J) space, but there is of course no reason why the coordinate system should not be right handed, in which case theta increases clockwise.

10. Transmissibility calculations

For flow between local radial blocks, the standard Eclipse method based on pressure equivalent radii is used to compute transmissibilities.

Flow between the outer local block and the adjoining host block is treated as linear, but with the transmissibility center for the local block taken at the pressure equivalent radius. With radii computed as described earlier, and host cells which are approximately square in cross-section, this method corresponds to the approach described by [Pedrosa and Aziz \(1986\)](#) and [Ewing et al \(1989\)](#).

Alphabetical list of GRID section keywords allowed within a local grid

Refer to the *Eclipse Reference Manual* for a further description of each keyword.

Keyword	Description
ACTNUM	Identifies active grid blocks.
ADD	Adds specified constants to specified arrays in the current box.
ADDZCORN	Adds a constant to the cell corner point depths array ZCORN.
BOX	Redefines the current box.
COORD	Defines the lines that contain all grid block corner points for each (i, j) and for each reservoir in the grid.
COORDSYS	Specifies completion of the circle in both corner point and block-centered geometries. Allocates grid block layers to reservoirs if there is more than one reservoir in the grid.
COPY	Copies data in the current box from one specified array to another.
COPYBOX	Copies data from one box of cells to another box of the same size.

Keyword	Description
DIFFMMF	Matrix-fracture diffusivity multipliers for current input box.
DIFFMR	R direction diffusivity multipliers for current input box.
DIFFMTHT	Theta direction diffusivity multipliers for current input box.
DIFFMX	X+ direction diffusivity multipliers for current input box.
DIFFMY	Y+ direction diffusivity multipliers for current input box.
DIFFMZ	Z+ direction diffusivity multipliers for current input box.
DR	R direction grid block sizes for the current box.
DRV	Vector of R direction grid block sizes.
DTHETA	Theta direction grid block sizes for the current box.
DTHETAV	Vector of theta direction grid block sizes.
DX	X direction grid block sizes for the current box.
DXV	Vector of X direction grid block sizes.
DY	Y direction grid block sizes for the current box.
DYV	Vector of Y direction grid block sizes.
DZ	Z direction (vertical) grid block sizes for the current box.
DZMTRX	Vertical dimension of a typical block of matrix material, for the entire grid.
DZMTRXV	Vertical dimension of a typical block of matrix material, for the current box.
DZNET	Grid block net thicknesses for the current box.
ENDBOX	Redefines the current box to encompass the whole grid.
EQLZCORN	Resets part of the corner point depth array.
EQUALS	Sets specified arrays to specified constants in the current box.
HXFIN, HYFIN, HZFIN	Local grid cell size ratios.
HRFIN	Radial grid DR ratios.
IMPORT	Imports a binary grid file.
INRAD	Sets inner radius of radial grid.
MAXVALUE	Applies a maximum value to specified arrays within the current box.
MINPV	Sets a minimum pore volume that each cell must have to be active.
MINPVV	Sets a vector of minimum pore volumes that the cells in the current box must have to be active.
MINVALUE	Applies a minimum value to specified arrays within the current box.
MULTIPLY	Multiplies specified arrays by constant values in the current box.
MULTPV	Pore volume multipliers for the current box.
MULTR	R+ direction transmissibility multipliers for the current box.
MULTR-	R- direction transmissibility multipliers for the current box.
MULTTHT	Theta+ direction transmissibility multipliers for the current box.
MULTTHT-	Theta- direction transmissibility multipliers for the current box.

Keyword	Description
MULTX	X+ direction transmissibility multipliers for the current box.
MULTY	Y+ direction transmissibility multipliers for the current box.
MULTZ	Z+ direction transmissibility multipliers for the current box.
MULTX-	X- direction transmissibility multipliers for the current box.
MULTY-	Y- direction transmissibility multipliers for the current box.
MULTZ-	Z- direction transmissibility multipliers for the current box.
NNC	Explicit entry of non-neighbor connections.
NTG	Grid block net-to-gross ratios for the current box.
NXF _{IN} , NYF _{IN} and NZF _{IN}	Number of local grid cells in each host cell.
PERMR	R direction permeabilities for the current box
PERMTHT	Theta direction permeabilities for the current box.
PERMX	X direction permeabilities for the current box.
PERMY	Y direction permeabilities for the current box.
PERMZ	Z direction permeabilities for the current box.
PORO	Grid block porosities for the current box.
QMOBIL	Controls the mobile end-point correction for a local grid.
SIGMA	Matrix-to-fracture coupling factor for the entire grid
SIGMAGD	Matrix-fracture coupling for oil-gas gravity drainage (whole grid).
SIGMAGDV	Matrix-fracture coupling for oil-gas gravity drainage (current box).
SIGMAV	Matrix-to-fracture coupling factor for the current box.
SOLVDIRS	Overrides principal solver directions within a local grid.
THCGAS	Gas phase thermal conductivities for the current box.
THCOIL	Oil phase thermal conductivities for the current box.
THCONR	Specifies rock and fluid thermal conductivities for the current box.
THCROCK	Rock thermal conductivities for the current box.
THCSOLID	Solid phase thermal conductivities for the current box.
THCWATER	Water phase thermal conductivities for the current box.
TOPS	Depths of top faces of grid blocks for the current box.
VEFIN	Controls the Vertical Equilibrium model within a local grid.
ZCORN	Depths of grid block corners.

Local grid coarsening

Grid Coarsening is the complement of local grid refinement. Cells may be amalgamated to reduce the total number of active cells in the global grid.

Eclipse 100

To reserve memory for grid coarsening, in keyword `LGR` in the `RUNSPEC` section set `MCOARS` equal to the number of coarsening cells (amalgamations), for example:

```
LGR
--MAXLGR  MAXCLS  MCOARS
      0      0      48  /
```

Because grid coarsening generates non-neighbor connections, the keyword **NONNC** must not be specified in the RUNSPEC section. Also, care must be taken not to specify grid coarsening where new NNCs generated will be invalid, for instance at the boundary of two PVT regions with different constant R_s values.

Coarsenings are specified in the GRID data section using keyword **COARSEN**. For example, with **NDIVIX**=10, **NDIVIY**=10 and **NDIVIZ**=6:

```
COARSEN
-- I1 I2 J1 J2 K1 K2 NX NY NZ --
      1 10  1  4  1  6  5  2  3  /
      1  4  5 10  1  6  2  3  3  /
/
```

The first coarsening reduces the number of active cells from $10*4*6 = 240$ to $5*2*3 = 30$. The eight cells at (1,1,1), (2,1,1), (1,2,1), (2,2,1), (1,1,2), (2,1,2), (1,2,2), (2,2,2) are amalgamated into a single cell for simulation. The second coarsening reduces the number of active cells from $4*6*6 = 144$ to $2*3*3 = 18$, a reduction in the number of active cells by a factor of 8. Thus the number of amalgamated cells is $MCOARS = 30+18 = 48$.

NX, **NY**, and **NZ** are the number of coarsened cells in each direction within each coarsening set. If values are not provided then default values are set to 1. Specified values of **NX** must be an exact factor of $I2 - I1 + 1$, and similarly for **NY** and **NZ**.

Coarsening sets must not overlap one another, but they may join together as in the above example. Thus the coarsening facility is extremely flexible, allowing the engineer to build a highly complex grid. Local grid refinements cannot be placed within or adjacent to a grid coarsening, that is the local grids may only be applied to non-coarsened cells.

Implementation

Grid coarsenings are set for the duration of a run and may not be switched on or off at different times during a run. The amalgamations are computed automatically by Eclipse which also computes the appropriate non-neighbor connections between different coarsenings and between normal cells and coarsened cells.

Within each amalgamation, a “representative cell” is chosen in the center of the amalgamation. All cells within the amalgamation apart from the representative cell are made inactive. The representative cell is given the properties (dimensions, permeability and pore volume for example) of the complete amalgamation.

Note: Even if this chosen representative cell is supposed to be deemed inactive (due to the use of the **ACTNUM** array, or having a zero pore volume, for instance), it is reactivated because Eclipse uses it as the storage cell for the upscaled quantities.

If a well is completed in an amalgamation, Eclipse moves its connections into the representative cell. Thus, if it was completed in several layers of a multi-layer amalgamation, it ends up with only one connection that is situated in the representative cell. The representative cell of course has the **DX**, **DY** and **DZ** of the complete amalgamation, and the connection transmissibility factor is calculated accordingly.

Furthermore, if a cell connects with an aquifer, then the appropriate connection is set up by moving the connection to the representative cell of the coarsening group involved.

Just as refined cells have the decimal point replaced by R in solution reports (specified using `RPTGRID`, `RPTSOL`, or `RPTSCHED`), so coarsened cells have the decimal point replaced by C.

For a run involving dual porosity, the coarsenings should only be defined for fracture blocks. Eclipse copies these coarsenings into the matrix blocks.

Upscaling

Within each coarse block the properties are simply upscaled from fine (f) to coarse (c) in a single coarse cell amalgamation $(I1, I2) \times (J1, J2) \times (K1, K2)$ as defined in the following sections.

Pore Volume

$$PV_c = \sum_f PV_f$$

Porosity

$$PORO_c = \frac{PV_c}{\sum_f \frac{PV_f}{PORO_f}}$$

Depth

$$D_c = \frac{\sum_f PV_f D_f}{PV_c}$$

NTG

$$NTG_c = \frac{\sum_f PV_f NTG_f}{PV_c}$$

DX, DY, DZ

$$DX_c = \frac{\sum_f DX_f}{(J2 - J1 + 1)(K2 - K1 + 1)},$$

with similar expressions for DY_c and DZ_c .

Permx, Permy, Permz

$$Kx_c = \frac{\sum_f PV_f Kx_f}{PV_c}.$$

Permy, and Permz are calculated in a similar way.

Tops

$$TOPS_c = \frac{\sum_f PV_f \cdot \overline{TOPS}_f}{PV_c}$$

where $\overline{TOPS}$ is the depth corrected to the top surface of the coarse cell amalgamation.

Rock Volume (E300 Thermal)

$$ROCKV_c = \sum_f ROCKV_f$$

Tranx, Trany, Tranz

The upscaling for transmissibilities is achieved as a simple average. Between the centers of two neighboring coarse cells, $TRANX_c$ is obtained from a harmonic average of $TRANX_f$ in the X-direction and by summing $TRANX_f$ in the Y- and Z- directions so that

$$TRANX_c = \sum_{JK} \left[\frac{1}{\sum_I \left[\frac{1}{TRANX_f} \right]} \right]$$

where the summations in J and K are over the X-direction face between the two cells, and the summation $\sum_I \left(\frac{1}{TRANX_f} \right)$ is over all I-indexed fine cells between the cell centers of the coarse cells.

So for two coarse cells next to each other in the X-direction, say $(IA, IB) \times (J1, J2) \times (K1, K2)$ and $(IB + 1, IC) \times (J1, J2) \times (K1, K2)$,

then the summations in the J- and K-directions are $\sum_{J=J1}^{J2} \sum_{K=K1}^{K2}$, and in the I-direction $\sum_{I=IL}^{IU}$, where (assuming an odd number of fine cells in each direction in each coarse cell), $IL = (IA + IB)/2$ and $IU = (IB + 1 + IC)/2 - 1$.

For one-dimensional single-phase flow governed by Darcy's law, this argument produces a continuous flux across each interface.

Note: Because of the summing algorithm, it is possible to end up with a coarse cell which has flow in a direction where the non-coarsened grid would have had a barrier. If transmissibility or permeability multipliers are being used to block flow, they should be applied to all the cells in the coarse cell, not just a single slice of the fine cells in it.

Within each coarse cell $(I1, I2) \times (J1, J2) \times (K1, K2)$, an active representative cell is chosen to be located at $((I1 + I2)/2, (J1 + J2)/2, (K1 + K2)/2)$. The values of upscaled grid quantities (described below and above) are nominally associated with this cell. After processing the grid data, Eclipse retains just the active uncoarsened cells in the fine grid and the active representative cells in coarse groups.

Any region quantities such as SATNUM, ROCKNUM, PVTNUM for the coarse cell are therefore associated with the representative cell only. Similarly, any end-point values in a coarse cell, such as SWL, SWCR and SOWCR, are taken from the values in the representative cell. No relative permeability upscaling is done here; take care not to define coarse cells too large, in order to encompass wide relative permeability variations between different rock types.

Any well completed in a coarse cell has its completions nominally moved to the representative cell (for reporting purposes) and the connection factor and drawdown are calculated using the upscaled quantities $DX_c, DY_c, DZ_c, Kx_c, Ky_c, Kz_c$ and $DEPTH_c$. This means that in each coarse cell with a completion on the fine grid, the well is assumed to penetrate fully through the coarse cell in the appropriate direction of penetration.

Note: If the connection factor, Kh or r_o (pressure equivalent radius) is given in [COMPDAT](#), then this is not upscaled from fine to coarse grids.

Faults in coarsened regions

Where x- or y-faults exist in coarsened regions, these are represented by non-neighbor connections between cells which are physically adjacent despite having different z-coordinates. When coarsening is defined such that the fault lies between two original cells which will subsequently be in the same column of coarse cells, these connections are no longer sensible, as they connect cells in the z-direction where the geometry of the situation requires a connection in the x- or y-direction.

In this case, the ends of the non-neighbor connection are recalculated so that the connection is still made across the fault in the correct direction. This is geometrically equivalent to moving the fault to the edge of the new coarsened cell.

The transmissibility of the new non-neighbor connection is calculated exactly as for the upscaling example above, except that one of the transmissibilities in the sum will be the old non-neighbor connection transmissibility. Diffusivity and thermal transmissibility, if appropriate, are recalculated in exactly the same way.

Note: There can be only one fault per direction in any one coarsened cell. So a coarsened cell can contain both an x-fault and a y-fault, but not two x-faults or two y-faults.

Automatic refinement (Eclipse 100)

The Automatic Refinement facility permits the grid to be refined as a reverse process to coarsening, with minimal change to the input data. This is useful for the automatic scaling of existing data sets for massively parallel computations, and for performing grid refinement sensitivity studies. There are several caveats on its use. The facility should not be used to create 'in-place' LGRS.

You specify refinement factors (NRX, NRY, NRZ) for each grid dimension by adding the RUNSPEC section keyword **AUTOREF** to an existing base data set. For a base data set with grid dimensions NX, NY, NZ, Eclipse converts the input data to the refined grid, obtained by refining each base cell into NRX*NRY*NRZ cells. Each grid cell is simply refined uniformly by the factors NRX, NRY, NRZ in each dimension. It is possible to coarsen an auto-refined grid so as to achieve auto-refinement only in a part of the grid.

The base data set should first be run without auto-refinement to check for any errors. Error and consistency checking of the data set with auto-refinement is performed with respect to the base grid, but any output files such as GRID, RESTART or FLUX files, refer to the auto-refined grid.

Treatment of GRID section data in automatically refined grids

Data for the auto-refined grid is inherited from host cells in the base grid. Most data is simply copied to the automatically refined cells from the host cells in the base grid. The exceptions to this rule are the geometric grid data DX, DY, DZ, TOPS, DZNET, DZMTRXV, COORD and ZCORN. These are determined such that the obvious refinement rules are observed. Note that the PINCH and PINCHOUT values are not altered from the base grid and that the MINPVV/MINPV data for the auto-refined grid is simply inherited from host cells in the base grid.

By default, data for the multiplier keywords, for example **MULTX**, is also inherited from host cells in the base grid. Therefore in the auto-refined grid, **MULTX**(IB, JB, KB) applies to all auto-refined cells (IA, JA, KA) contained within the base grid cell (IB, JB, KB) rather than just on the X+ direction interface between cells (IB, JB, KB) and (IB+1, JB, KB) in the base grid.

If item 4 of the **AUTOREF** keyword is set to 1, then **MULTX**, for example, will be applied only between auto-refined cells on the X+ interface between these base grid cells.

If item 4 of the `AUTOREF` keyword is set to 2, then `MULTX`, for example, assumes its default behavior in the auto-refined grid, and additional multipliers, for example `SMULTX`, may also be entered, which only apply between auto-refined cells on the X+ interface between base grid cells.

Any numerical aquifers will be represented equivalently in the auto-refined grid by the central auto-refined cell within the host cell in the base grid. The specified aquifer length and cross-sectional area are copied from the base grid to retain the initial volumes.

Any coarsening boxes specified in the base grid using the `COARSEN` keyword are auto-refined. The keyword `AUTOCOAR` may be used to specify coarsening boxes in the base grid which are not auto-refined; this allows regions of the auto-refined grid to be coarsened back to the base grid if required.

Treatment of EDIT section data in automatically refined grids

Data entered in the `EDIT` section is upscaled from the base grid to the auto-refined grid in an appropriate manner. Edited pore volumes in the base grid are applied to auto-refined cells by multiplying by the ratio of new (edited) to old pore volumes for host cells in the base grid. In like fashion, any transmissibilities entered in the `EDIT` section are accounted for by applying a multiplier which is the ratio of new (edited) transmissibility for the host cells in the base grid to the value of transmissibility on the host cells in the base grid obtained by upscaling the calculated transmissibilities on the auto-refined grid to the base grid.

Treatment of REGIONS section data in automatically refined grids

Any region data specified in the `PROPS`, `REGIONS` or `SCHEDULE` sections is inherited by the auto-refined grid from the host cells in the base grid. This applies also to end-points specified in the `PROPS` section.

Treatment of SUMMARY section data in automatically refined grids

Any grid cell numbers or completion location indices for the base grid entered in the `SUMMARY` section are converted to the central auto-refined cell within the base cell. The `SUMMARY` and `RSM` files exhibit this conversion.

Treatment of wells in automatically refined grids

A vertical well connection in an automatically refined base cell is moved to the central column of refined cells within the base cell. Thus consider a single connection in the base grid for a well penetrating in the Z direction. Within the automatically refined grid there are NRZ connections, corresponding to each layer of the automatically refined grid within the base connected cell. The connections are located in the central column of refined cells, with location $((NRX + 1)/2, (NRY + 1)/2)$. Eclipse treats these as independent connections (for example, they will be closed one by one in automatic workovers). If you wish these connections to be lumped together for simultaneous closure in workovers, you should use the `COMPLUMP` keyword, specifying the base cell coordinates in the lumped connection range. Note that all the well and connection data input in the `SCHEDULE` section should refer to the cell coordinates in the base grid (thus requiring no change to the data), but the results reported in the output files will refer to the refined grid cell coordinates.

This is illustrated in the following week in the case $NRX = NRY = 3$:

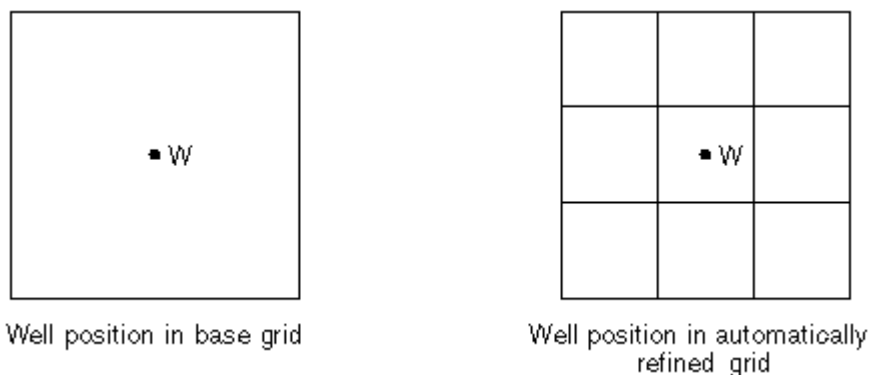


Figure 2.14. Well positioning in automatically refined grids

Similarly, a horizontal connection (`COMPDAT` item 13 = 'X' or 'Y') in the base grid is replaced by NRX or NRY connections in the central X or Y direction row of the base cell.

Note: The above rule is inappropriate for a well located at a corner or an edge of a base grid cell; Eclipse still places it in the central column or row of refined cells.

If the connection factor and Kh are not explicitly defined using keyword `COMPDAT` in the base data set, then Eclipse calculates the connection factors within the auto-refined grid in the usual way.

If Kh is defined in keyword `COMPDAT` then its value in the auto-refined grid is set equal to the specified Kh divided by the number of divisions in the direction of penetration of the well. If the connection factor T_{wj} is defined in `COMPDAT` then an additional skin factor ΔS is computed such that T_{wj} equals the value calculated by the default formula on the base grid when the skin factor is incremented by ΔS . The connection factor on the auto-refined grid is then computed from the default formula on the auto-refined grid again with the increment ΔS added to the skin factor.

If the well's drainage radius is defaulted in item 7 of keyword `WEL SPECS`, it takes the value of the pressure equivalent radius of the refined cell, rather than that of the base cell. This results in an increase in the reported productivity index.

Current restrictions

- Local grid refinements may not be used with automatic refinement.
- The `NNC` and `EDITNNC` keywords are not available with automatic refinement.
- Wellbore friction and multisegment wells may not be used with automatic refinement.
- The nine-point schemes are incompatible with auto-refinement, and it is not possible to re-specify transmissibility multipliers within the `SCHEDULE` section when using auto-refinement.
- Radial local grid refinements are assumed vertical: see item 7 of "[Radial refinements](#)". Non-vertical refinements are not forbidden, as historically this has been an acceptable workflow. However, the code does assume that the central axis of the refinement is vertical, and, if the refinement is highly slanted, this may cause the axis to be outside of the refinement at its base, and so invert some cells. Heed the warnings in this case, and treat them as errors where possible.

Summary of keywords

Common keywords

Section	Keyword	Description
EDIT, GRID, PROPS, REGIONS, SOLUTION, SCHEDULE	ENDFIN	Terminates data for a local grid.
EDIT, GRID, PROPS, REGIONS, SOLUTION, SCHEDULE	REFINE	Indicates that subsequent data refers to a named local grid.

RUNSPEC

Restriction	Keyword	Description
Eclipse 100	AUTOREF	Sets options for auto-refinement.
	LGR	Sets options and dimensions for local grid refinement and coarsening.
Eclipse 300	NOAIMLGR	Suppresses making all LGR cells implicit in AIM runs.

GRID section

Restriction	Keyword	Description
Eclipse 100	AUTOCOAR	Is analogous to the COARSEN keyword and may be used in auto-refined grids.
Eclipse 100	AMALGAM	Defines groups of LGRs which are to be amalgamated.
	CARFIN	Sets up a Cartesian local grid refinement.
	COARSEN	Defines a box of cells for grid coarsening.
	EXTFIN	Specifies an unstructured local grid refinement.
	EXTHOST	Used to specify the global host cells for an unstructured local grid refinement.
	EXTREPGL	Specifies the replaced global cells for an unstructured local grid refinement.
	GRIDFILE	Controls the contents of the Grid Geometry file.
	HREFIN	Can be used to dictate the size ratios of DRV values within radial local grids.
	HXFIN	Can be used to dictate the size ratios of DXV values within Cartesian local grids.
	HYFIN	Can be used to dictate the size ratios of DYV values within Cartesian local grids.
	HZFIN	Can be used to dictate the size ratios of DZV values within Cartesian local grids.
	INRAD	Specifies the inner radius of the radial refinement.
	NXFIN	Sets the number of local cells in each global cell in the x-direction.

Restriction	Keyword	Description
	NYFIN	Sets the number of local cells in each global cell in the y-direction.
	NZFIN	Sets the number of local cells in each global cell in the z-direction.
Eclipse 100	OUTRAD	Inner radius of the outer annulus in a radial LGR.
Eclipse 100	QMOBIL	Controls the mobile end point correction in an LGR.
	RADFIN	Sets up a radial LGR.
Eclipse 100	RADFIN4	Sets up a 4 column radial LGR.
	RPTGRIDL	controls grid section output for LGRs
	TRANGL	Specifies the global-local connections and associated transmissibilities for an unstructured local grid refinement.
Eclipse 100	SMULTX , SMULTY , SMULTZ	Alternative transmissibility multipliers for auto-refinement.
Eclipse 100	VEFIN	May be used in LGR runs with Vertical Equilibrium in the global grid, to control the Vertical Equilibrium model in the local grid systems.

SOLUTION section

- [GETGLOB](#) Read only the global restart data.

SUMMARY section

Well, block and connection data for an LGR can be obtained by prefixing the appropriate keywords with an L. Each subsequent data item to which the keyword refers is then prefixed by and LGR name. (See the [SUMMARY](#) section overview for further details.)

SCHEDULE

Restriction	Keyword	Description
Eclipse 300	COMPAGHL	Sets the gravity head in local grid completion cells.
	COMPDATL	Defines well completions in an LGR.
Eclipse 300	COMPKRIL	Sets injection relative permeabilities in local grid completion cells.
	COMPLMPL	Lumps well completions for automatic workovers.
Eclipse 300	COMPMBIL	Sets injection mobilities in local grid completion cells.
Eclipse 100	COMPRPL	Scales saturations for local grid well connections.
Eclipse 100	COMPVEL	Sets well connection depth data in local grids.
Eclipse 100	LGRFREE	Switches to local timestepping.
Eclipse 100	LGRLOCK	Switches to global timestepping.
Eclipse 100	LGROFF	Switches a local grid refinement off
Eclipse 100	LGRON	Switches a global grid on.
Eclipse 100	LSCRITS	Sets WARP linear solver convergence criteria for sub-grids.
Eclipse 100	NEXTSTPL	Sets the next local timestep.

Restriction	Keyword	Description
Eclipse 100	TUNINGL	Sets simulator control parameters for all local grids.
Eclipse 100	TUNINGS	Sets simulator control parameters for specific local grids.
	WELMOVEL	Moves a well into an LGR at a restart.
	WELOPENL	Shuts or reopens wells or well connections in an LGR.
	WELSPECL	Defines a well in an LGR.
Eclipse 100	WFRICTNL	Designates a well in an LGR to be a wellbore friction well
	WPIMULTL	Multiplies well connection factors for a well in an LGR.

Options keyword (Eclipse 100)

Switch	Description
32	controls timestep chops due to wells dying under THP control
37	replaces local grid solution states with those of the global host cell.
46	controls the interpolation of MINPV or MINPVV to LGRs.
51	controls the first local grid solution following a global timestep.
52	controls the use of corner point depths in VE runs.
56	controls the application of LGRON / LGROFF .

Example LGR problem

This data set describes the seventh SPE Comparative Solution Project (1991) - a horizontal well with friction. The data set describes CASE 3A, having a 900 ft. horizontal well completed across 3 global cells in the x-direction. These 3 cells, plus an additional cell at each end (making 5 in all), are refined as follows:

Z-direction: 3 layers with a 2: 1: 2 thickness ratio.

Y-direction: 3 rows with a 2: 1: 2 thickness ratio.

X-direction: The two global cells at each end were refined into 4, with equal DX values. The central global cell was not refined in the x-direction.

RUNSPEC

```

RUNSPEC
TITLE
  7th SPE Benchmark, Case 3a

DIMENS
  9    9    6  /

OIL
WATER
GAS

DISGAS

FIELD

TABDIMS
  1    1   15   15   1   15  /

```

```

WELLDIMS
 2   9   1   2 /

LGR
 1 150 /

FRICTION
 1 /

START
 1 'JAN' 1990 /

NSTACK
 25 /

FMTOUT
FMTIN
UNIFOUT
UNIFIN

```

GRID

```

GRID
=====
DXV
 9*300 /

DYV
 620 400 200 100 60 100 200 400 620 /

DZ
 81*20 81*20 81*20 81*20 81*30 81*50 /

PERMX
 486*300 /

PERMY
 486*300 /

PERMZ
 486*30 /

PORO
 486*0.2 /

EQUALS
'TOPS' 3590 1 9 1 9 1 1 /
/

```

Specify the refined local grid. The global cells I=5-9, J=5, K=1 are refined into a 9 x 3 x 3 system.

```

CARFIN
-- name I1-I2 J1-J2 K1-K2 nx ny nz
'LGR1' 5 9 5 5 1 1 9 3 3 /

```

Specify the x-direction partition of the 5 global cells into 9. The cells at either end are partitioned into 4 equal-sized cells. The middle cell is not partitioned in the x-direction.

```

NXFIN
 2 2 1 2 2 /

```

Specify the size ratio of the local grid cells in the y-direction. The global cells are divided into 3 rows in the y-direction, with a 2:1:2 thickness ratio.

```

HYFIN
 2.0 1.0 2.0 /

```

Specify the size ratio of the local grid cells in the z-direction. The global cells are divided into 3 layers in the z-direction, with a 2:1:2 thickness ratio.

```

HZFIN
  2.0  1.0  2.0  /

```

End of local grid specification.

```

ENDFIN

```

PROPS

Use Stone's method 2 for three-phase oil relative permeability.

```

PROPS  =====
STONE2

SWOF
-- Sw      Krw      Krow      Pcow
  0.22     0.0      1.0000     6.30
  0.3      0.07     0.4000     3.60
  0.4      0.15     0.1250     2.70
  0.5      0.24     0.0649     2.25
  0.6      0.33     0.0048     1.80
  0.8      0.65     0.0        0.90
  0.9      0.83     0.0        0.45
  1.0      1.0      0.0        0.0    /

SGOF
-- Sg      Krg      Krog      Pcgo
  0.0      0.0      1.00      0.0
  0.04     0.0      0.60      0.2
  0.1      0.022    0.33      0.5
  0.2      0.1      0.10      1.0
  0.3      0.24     0.02      1.5
  0.4      0.34     0.0       2.0
  0.5      0.42     0.0       2.5
  0.6      0.5      0.0       3.0
  0.7      0.8125   0.0       3.5
  0.78     1.0      0.0       3.9    /

PVTW
-- Pref     Bw      Cw      Vw      Viscosity
  3600     1.00329   3.0E-6   0.96      0    /

ROCK
-- Pref     Cr
  3600     4.0D-6    /

DENSITY
-- oil      water    gas
  45.0     62.14     0.0702    /

PVDG
-- Pgas     Bg      Viscg
  400      5.9      0.013
  800      2.95     0.0135
  1200     1.96     0.014
  1600     1.47     0.0145
  2000     1.18     0.015
  2400     0.98     0.0155
  2800     0.84     0.016
  3200     0.74     0.0165
  3600     0.65     0.017
  4000     0.59     0.0175
  4400     0.54     0.018
  4800     0.49     0.0185
  5200     0.45     0.019
  5600     0.42     0.0195    /

```

```

PVCO
-- Ppub   Rs      Bo      Visco   Co      CVo
   400    0.165  1.012  1.17    1.0E-5  0.0
   800    0.335  1.0255 1.14    2*
  1200    0.500  1.038  1.11    2*
  1600    0.665  1.051  1.08    2*
  2000    0.828  1.063  1.06    2*
  2400    0.985  1.075  1.03    2*
  2800    1.130  1.087  1.00    2*
  3200    1.270  1.0985 0.98    2*
  3600    1.390  1.11   0.95    2*
  4000    1.500  1.12   0.94    2*
  4400    1.600  1.13   0.92    2*
  4800    1.676  1.14   0.91    2*
  5200    1.750  1.148  0.9     2*
  5600    1.810  1.155  0.89    2*
/

PMAX
6000 /

```

SOLUTION

The specified initial conditions can be obtained by equilibration.

```

SOLUTION =====
EQUIL
-- Dep    Pres   Dwoc   Pcwoc   Dgoc   Pcgoc   Rsvd   Rvvd   Fipc
   3600   3600   3700    0.0   3590    0.0     1     0     0 /

RSVD
-- Dep    Rs
   3500   1.41
   3750   1.41 /

RPTSOL
'PRES'  'SOIL'  'SWAT'  'SGAS'  'RS' /

```

SUMMARY

```

SUMMARY =====
FOPR
FGPR
FWPR
FOPT
FGPT
FWPT
FWCT
FGOR
LWBHP
'LGR1'  'PROD' /
/
LCPR
'LGR1'  'PROD' 3 2 2 /
/
FPR
FWIR
FWIT
RUNSUM

```

SCHEDULE

```

SCHEDULE =====
RPTSCHED
'WELLS=2'  'SUMMARY=2'  'WELSPICS' /

```

Specify producer in local grid LGR1:

```
WELSPECL
-- Name Group LGR I J Dbh Phase
  'PROD' 'G' 'LGR1' 8 5 3600 'OIL' /
/
```

Specify injector in global grid:

```
WELSPECS
-- Name Group I J Dbh Phase
  'INJ' 'G' 9 5 3725 'WAT' /
/
```

Specify connections of producer in local grid:

```
COMPDATL
-- Name LOCAL GRID I J K1-K2 Op/Sh SatTab ConFact Diam KH S D Dirn
  'PROD' 'LGR1' 3 2 2 2 'OPEN' 2* 0.375 3* 'X' /
  'PROD' 'LGR1' 4 2 2 2 'OPEN' 2* 0.375 3* 'X' /
  'PROD' 'LGR1' 5 2 2 2 'OPEN' 2* 0.375 3* 'X' /
  'PROD' 'LGR1' 6 2 2 2 'OPEN' 2* 0.375 3* 'X' /
  'PROD' 'LGR1' 7 2 2 2 'OPEN' 2* 0.375 3* 'X' /
/
```

Specify connections of injector in global grid:

```
COMPDAT
-- Name I J K1-K2 Op/Sh SatTab ConFact
  'INJ' 1 5 6 6 'OPEN' 1* 24.34 /
  'INJ' 2 5 6 6 'OPEN' 1* 24.34 /
  'INJ' 3 5 6 6 'OPEN' 1* 24.34 /
  'INJ' 4 5 6 6 'OPEN' 1* 24.34 /
  'INJ' 5 5 6 6 'OPEN' 1* 24.34 /
  'INJ' 6 5 6 6 'OPEN' 1* 24.34 /
  'INJ' 7 5 6 6 'OPEN' 1* 24.34 /
  'INJ' 8 5 6 6 'OPEN' 1* 24.34 /
  'INJ' 9 5 6 6 'OPEN' 1* 24.34 /
/
```

Specify the friction data for the producer. The grid block locations refer to the local grid.

```
WFRICTNL
-- Name Diam Roughness
  'PROD' 0.375 3.75E-4 /
-- Local Grid I J K Tlen1 Tlen2
  1* 7 2 2 0.0 150 /
  1* 6 2 2 150 300 /
  1* 5 2 2 300 600 /
  1* 4 2 2 600 750 /
  1* 3 2 2 750 900 /
/
```

Specify the production well constraints:

```
WCONPROD
-- Name Op/Sh CtlMode Orat Wrat Grat Lrat Resv Bhp
  'PROD' 'OPEN' 'LRAT' 3* 9000 1* 1500 /
/
```

Specify the injection well constraints:

```
WCONINJE
-- Name Phase Op/Sh CtlMode Srat Resv Bhp
```



```
'INJ' 'WAT' 'OPEN' 'BHP' 2* 3700 /
/
```

Specify timesteps and reports:

```
TSTEP
4*100 /
RPTSCHED
'PRES' 'SWAT' 'SGAS' 'FIP' 'WELLS=2'
'SUMMARY=2' 'CPU=2' 'WELSPECS' /
TSTEP
100 /
RPTSCHED
'WELLS=2' 'SUMMARY=2' 'WELSPECS' /
TSTEP
4*100 /
RPTSCHED
'PRES' 'SWAT' 'SGAS' 'FIP' 'WELLS=2'
'SUMMARY=2' 'CPU=2' 'WELSPECS' /
TSTEP
100 /
RPTSCHED
'WELLS=2' 'SUMMARY=2' 'WELSPECS' /
TSTEP
4*100 /
RPTSCHED
'PRES' 'SWAT' 'SGAS' 'FIP' 'WELLS=2'
'SUMMARY=2' 'CPU=2' 'WELSPECS' /
TSTEP
100 /
END
```

Aquifer modeling facilities

x	Eclipse 100
x	Eclipse 300

This chapter describes the aquifer models available in both Eclipse 100 and Eclipse 300.

The models available are:

- Numerical aquifers
- Carter-Tracy aquifers
- Fetkovich aquifers
- Constant flux aquifers
- Constant head aquifers
- Multi-component aquifers

Numerical aquifers are represented by a one-dimensional row of cells within the simulation grid.

The other types of aquifer, classed as ‘analytic aquifers’, are represented by computed source terms in the reservoir grid cells with which they connect.

A fuller description of these numerical and analytical aquifers is provided in the following sections.

Numerical aquifer

A numerical aquifer is modeled by a one-dimensional row of cells. A set of cells in the simulation grid is nominated to represent the aquifer, which may then connect to specified faces of the reservoir (using the [AQUCON](#) keyword).

The properties of the aquifer grid blocks (length, cross-sectional area, porosity, permeability, initial pressure, depth, and PVT and saturation table numbers) may be specified explicitly using keyword [AQUNUM](#) in the `GRID` section, or if the properties of the aquifer cells are defaulted, they are taken from the grid block values using data entered in the `GRID` and `EDIT` sections. This enables the aquifer properties to be independent of its actual position within the grid. The only information concerning the shape of the numerical aquifer is given by the cross-sectional area, length and depth.

The aquifer should be connected to a face of the reservoir using the keyword [AQUCON](#). The first cell of the aquifer declared in the keyword [AQUNUM](#) connects to the specified faces of the reservoir by non-neighbor connections. The set of cells defining the aquifer are connected together (possibly by non-neighbor connections) in the order specified in [AQUNUM](#). The aquifer cells are isolated from the grid except for these connections. In dual porosity runs, the aquifer should be placed in the lower half (that is the fracture part) of the grid.

The aquifer **pore volume**, for simulation grid cell i , is initially calculated from the data entered in the [AQUNUM](#) keyword using the equation:

$$\text{PORV}_i = C \cdot \text{PORO}_i \cdot \text{LENGTH}_i \cdot \text{AREA}_i \quad \text{Eq. 2.39}$$

where

PORV_i is the pore volume of the aquifer cell i

PORO_i is the porosity of the aquifer cell i

$LENGTH_i$ is the length of the aquifer cell i

$AREA_i$ is the cross-sectional area of the aquifer cell i as input in item 5 of AQUNUM.

C Only used to convert between barrels and cubic feet in the FIELD unit system.

The porosity of the aquifer cell is taken from the grid block if it is defaulted in the AQUNUM keyword. Any values of [MULTPV](#) entered in the GRID or EDIT sections will not apply to the numerical aquifer cells, and any edits to [PORV](#) in the EDIT section for numerical aquifer cells are ignored.

The **transmissibility** between two aquifer cells is given by:

After initialization numerical aquifer cells are subject to rock compression during the simulation as other cells so the pore volume may subsequently change.

$$TR = \frac{CDARCY}{1/T_i + 1/T_j} \quad \text{Eq. 2.40}$$

with

$$T_i = \frac{2 \cdot PERMX_i \cdot XSECT_i}{LENGTH_i}$$

where

$PERMX_i$ is the permeability of aquifer cell i

$XSECT_i$ is the cross-sectional area

$LENGTH_i$ is the length of the aquifer cell i .

This expression is used for transmissibilities between aquifer cells in both radial and Cartesian geometries. However, in connecting the first aquifer cell to a grid block, the appropriate radial or Cartesian transmissibility from the block edge to the center is used (see the [AQUCON](#) keyword).

No multiplier is applied to the transmissibility of the inter-block connections within the one-dimensional aquifer. The values of the transmissibility multipliers for the simulation grid entered in the GRID section ([MULTX](#) and related keywords) do not apply to any numerical aquifer connections.

Eclipse 100 If required, a transmissibility multiplier may be applied to the calculated transmissibility between the first aquifer block and the nominated connecting face of the reservoir, using keyword [AQUCON](#). The use of a multiplier may be considered in order to aid history matching, for instance.

Eclipse 100 Transmissibility values are calculated using the [OLDTRAN](#) block centered method (see "[Transmissibility calculations](#)").

Eclipse 100 If the initial aquifer pressure is not entered in keyword [AQUNUM](#), then the initial pressure is calculated to be in equilibrium with the reservoir by using the initial pressures for the underlying grid blocks obtained by equilibration. Note that, when initializing the solution in numerical aquifer blocks, Eclipse sets the water saturation to unity, so the initial state is not then quiescent if the representative cells do not lie completely in the water zone.

Eclipse 300 For the [AQUNUM](#) keyword, if the initial aquifer pressure is not entered (by choosing the default value) or even if a negative value is specified, then the initial pressure will be calculated to place the aquifer as near as possible in equilibrium with the reservoir.

Summary quantities for a numerical aquifer may be obtained by defining each aquifer as a fluid-in-place region. It is then possible to examine the aquifer inflow performance and pressure support supplied to the reservoir.

In order to use the facility, the maximum number of numerical aquifer blocks and the maximum number of lines of connection data in AQUCON should be set using the [AQUDIMS](#) keyword in the RUNSPEC section.

Carter-Tracy aquifer

The Carter-Tracy aquifer model is a simplified approximation to a fully transient model, which avoids the need for superposition.

The method uses a table that supplies a constant terminal rate influence function. Although the theory has been developed for a radially symmetric reservoir surrounded by an annular aquifer, the method is applicable to arbitrarily-shaped reservoirs.

The two main parameters that govern the behavior of the aquifer are the **time constant** (with the dimension of time):

$$T_c = \frac{\mu_w \varphi C_t r_o^2}{k_a c_1} \quad \text{Eq. 2.41}$$

where

μ_w is the viscosity of water in the aquifer

φ is the aquifer porosity

C_t is the total (rock + water) compressibility

r_o is the outer radius of the reservoir (or inner radius of the aquifer)

k_a is the aquifer permeability

c_1 is 0.008527 (METRIC, PVT-M); 0.006328 (FIELD); 3.6 (LAB)

and the **aquifer influx constant** (with the dimension of total influx per unit pressure drop):

$$\beta = c_2 h \theta \varphi C_t r_o^2 \quad \text{Eq. 2.42}$$

where

h is the aquifer thickness

θ is the angle subtended by the aquifer boundary from the center of the reservoir, in degrees, divided by 360

φ is the aquifer porosity

C_t is the total (rock + water) compressibility

r_o is the outer radius of the reservoir (or inner radius of the aquifer)

c_2 is 6.283 (METRIC, PVT-M); 1.1191 (FIELD); 6.283 (LAB).

The time constant T_c is used to convert time t into dimensionless time t_D using the following expression:

$$t_D = \frac{t}{T_c} \quad \text{Eq. 2.43}$$

The Carter-Tracy model uses tables of dimensionless time versus a dimensionless pressure influence function. The default table is for an infinite aquifer with constant terminal rate as given by van Everdingen and Hurst, but you may supply alternative tables using the keyword **AQUTAB** in the PROPS section.

The Carter-Tracy model expresses the pressure drop at the aquifer boundary in terms of the dimensionless pressure influence function PI_D by

$$p_{a0} - \bar{p} = \frac{Q_a}{\beta} PI_D(t_D) \quad \text{Eq. 2.44}$$

where

Q_a is the aquifer inflow rate

p_{a0} is the initial pressure of water in the aquifer

$\bar{p}$ is the average water pressure on the aquifer/reservoir boundary.

The average inflow rate from the aquifer to a grid block i over a simulator time interval $[t, t + \Delta t]$ is calculated as

$$\overline{Q_{ai}} = \alpha_i \{a - b[p_i(t + \Delta t) - p_i(t)]\} \quad \text{Eq. 2.45}$$

where

$$a = \frac{1}{T_c} \left\{ \frac{\beta \Delta p_{ai} - W_a(t)(PI_D')(t + \Delta t)_D}{PI_D(t + \Delta t)_D - t_D PI_D'(t + \Delta t)_D} \right\} \quad \text{Eq. 2.46}$$

and

$$b = \frac{\beta}{T_c [PI_D(t + \Delta t)_D - t_D PI_D'(t + \Delta t)_D]} \quad \text{Eq. 2.47}$$

where

Δp_{ai} is the pressure drop $p_{a0} + \rho g(d_i - d_a) - p_i(t)$

PI_D' is the derivative of PI_D with respect to t_D

α_i is the area fraction for each connection.

Here, the area fraction for each connection is given by:

$$\alpha_i = \frac{m_i A_i}{\sum m_i A_i}$$

where

A_i is the area of the block face communicating with the aquifer.

m_i is an aquifer influx coefficient multiplier.

The aquifer influx rates calculated from equation 2.45 contribute to the residual for the implicit equations solved by Eclipse at time t . The cumulative aquifer influx $W_a(t)$ used in Equation 2.46 is updated explicitly at the end of the timestep.

The aquifer properties (compressibility, porosity, initial pressure, depth and radius for example) should be defined using the keyword `AQUCT`, and the aquifer connections to one or more faces of the reservoir should be made using the keywords `AQUANCON` and `AQANCONL`. The latter two keywords help to distinguish between connections to a global grid cell or a local grid cell.

Eclipse 100

If the initial aquifer pressure is defaulted, it is calculated from equation 2.50 such that the aquifer is in initial equilibrium with the reservoir.

The aquifer should also be given a table number for the influence function table that it will use. Influence function tables may be supplied with the `AQUTAB` keyword in the `PROPS` section. Eclipse has a built-in default influence function table (table number 1), which represents the constant terminal rate solution for an infinite aquifer as given by van Everdingen and Hurst.

In order to use the facility, the maximum number of analytic aquifers and the maximum number of grid block connected to any single analytic aquifer should be set using the `RUNSPEC` section keyword `AQUUDIMS`.

Summary file quantities for the aquifer may be obtained using `AAQR` (influx rate) and `AAQT` (cumulative influx) in the `SUMMARY` section.

For Tracer Tracking runs, the initial concentrations of water phase tracers in the aquifer may be supplied using the keyword `AQANTRC` in the `SOLUTION` section.

For the Brine Tracking option, the initial salt concentration may be supplied using the `AQUCT` keyword in the `SOLUTION` section.

Fetkovich aquifer

The Fetkovich aquifer model uses a simplified approach based on a pseudo steady-state productivity index and a material balance relationship between the aquifer pressure and the cumulative influx.

The Fetkovich model is best suited for smaller aquifers that may approach a pseudo steady-state condition quickly.

This model assumes that the pressure response is felt uniformly throughout the entire aquifer.

The aquifer inflow is modeled by the equation:

$$Q_{ai} = \frac{d}{dt}(W_{ai}) = J\alpha_i[p_a + p_c - p_i + \rho g(d_i - d_a)] \quad \text{Eq. 2.48}$$

where

Q_{ai} is the inflow rate from the aquifer to the connecting grid block i

W_{ai} is the cumulative influx from the aquifer to grid block i

J is the specified Productivity Index of the aquifer

α_i is the area fraction for the connection to grid block i

p_a is the pressure in the aquifer at time t

p_c is the capillary pressure

p_i is the water pressure in a connecting grid block i

ρ is water density in the aquifer

d_i is the depth of grid block i

d_a is the datum depth of the aquifer.

Eclipse 100

If the Polymer model is in use, similar equations are applied for the polymer and brine components.

The area fraction for each grid block connection is given by:

$$\alpha_i = \frac{m_i A_i}{\sum m_i A_i} \quad \text{Eq. 2.49}$$

where

A_i is the area of the block face communicating with the aquifer

m_i is an aquifer influx coefficient multiplier.

For vertical faces, the areas A_i are multiplied by the net-to-gross ratio of the connecting grid blocks. You may specify the influx multiplier m_i to help with the history matching process.

The pressure response in the aquifer is determined by the material balance equation:

$$W_a = C_t V_{w0} (p_{a0} - p_a) \quad \text{Eq. 2.50}$$

where

W_a is the cumulative total influx from the aquifer

C_t is the total (rock + water) compressibility of the aquifer

V_{w0} is the initial volume of water in the aquifer

p_{a0} is the initial pressure of water in the aquifer.

Eclipse 100

If the brine tracking or passive tracer options are used, Eclipse 100 also accounts for the inflow of the tracer or brine from the aquifer.

The aquifer performance essentially depends on two parameters, the **aquifer time constant** and the **productivity index**. The aquifer time constant, which has the dimension of time, is given by:

$$T_c = \frac{C_t V_{w0}}{J} \quad \text{Eq. 2.51}$$

where

C_t is the total (rock + water) compressibility of the aquifer

V_{w0} is the initial volume of water in the aquifer

J is the specified Productivity Index of the aquifer.

Under the assumption of uniform reservoir pressure in the connecting grid blocks, and by integrating equations 2.48 and 2.50, the average influx rate over the time interval Δt is expressed as

$$\overline{Q_{ai}} = \alpha_i J (p_a - p_i + \rho g(d_i - d_a)) \left(\frac{1 - e^{(-\Delta t/T_c)}}{\Delta t/T_c} \right) \quad \text{Eq. 2.52}$$

which is the form used by the simulator to compute the influx rates.

At the end of each timestep, the aquifer's cumulative total influx is incremented and its pressure is updated using equation 2.50.

By varying the aquifer volume and the productivity index, the Fetkovich model can encompass a range of aquifer behavior from steady state to the 'pot' aquifer. If the product $C_t V_{w0}$ is given a large value, so that the time constant is very large, the behavior approaches that of a steady state aquifer, in which the pressure on the external boundary does not change. Alternatively, if the productivity index is given a large value so that the time constant is small, the behavior approaches that of a 'pot' aquifer, in which the pressure is in approximate equilibrium with the reservoir at all times.

The aquifer properties (compressibility, porosity, initial pressure, depth and productivity index for example) should be defined using the keyword [AQUFETP](#), and the aquifer connections to one or more faces of the reservoir should be made through the keywords [AQUANCON](#) and [AQANCONL](#). The latter two keywords help to distinguish between connections to a global grid cell or a local grid cell.

Eclipse 100

Alternatively, both the aquifer properties and the connection data may be defined with a single keyword [AQUFET](#), but this is less versatile as it allows the aquifer to connect to only one face of the reservoir.

In order to use the facility, the maximum number of analytic aquifers and the maximum number of grid block connected to any single analytic aquifer should be set using the RUNSPEC section keyword [AQUDIMS](#). For more information, see [Fetkovich \(1971\)](#).

SUMMARY file quantities for the aquifer may be obtained using AAQR (influx rate), AAQT (cumulative influx) and AAQP (aquifer average pressure) in the SUMMARY section.

If the initial aquifer pressure is defaulted, it is calculated from equation 2.48 such that the aquifer is in initial equilibrium with the reservoir.

For Tracer Tracking runs, the initial concentrations of water phase tracers in the aquifer may be supplied using the keyword [AQANTRC](#).

For the Brine Tracking option, the initial salt concentration may be supplied using [AQUFET](#) (Eclipse 100) or [AQUFETP](#).

Constant flux aquifer

For a constant flux aquifer you specify its water influx rate directly by the user, instead of it being calculated by an analytic aquifer model. For the purpose of dimensioning, among other things, it is classed with the other analytic aquifer models.

One use of a constant flux aquifer is to model rainfall influx for environmental applications.

The water flow rate into a grid block from a constant flux aquifer is given by:

$$Q_{ai} = F_a \cdot A_i \cdot m \quad \text{Eq. 2.53}$$

where

Q_{ai} is the inflow rate from the aquifer to the connecting grid block i

F_a is the aquifer flux, which you specify

A_i is the area of the connecting face of grid block i , which is calculated directly from the cell geometry

m is the aquifer influx multiplier, which you specify.

The aquifer should be defined using the [AQUFLUX](#) keyword in the `SOLUTION` section, and the aquifer connections to one or more faces of the reservoir should be made via the keywords [AQUANCON](#) and [AQANCONL](#). The latter two keywords help to distinguish between connections to a global grid cell or a local grid cell.

To achieve time-dependency, the flux may be modified during simulation by reentering the [AQUFLUX](#) keyword in the `SCHEDULE` section.

In order to use the facility, the parameters `NANAQU` (defining the maximum number of analytic aquifers) and `NCAMAX` (defining the maximum number of grid block connected to any single analytic aquifer) should be set using the `RUNSPEC` section keyword [AQUDIMS](#).

`SUMMARY` file quantities for the aquifer may be obtained using `AAQR` (influx rate) and `AAQT` (cumulative influx) in the `SUMMARY` section.

For Tracer Tracking runs, the concentrations of water phase tracers in the aquifer may be supplied using the keyword [AQANTRC](#).

The keyword [RAINFALL](#) can be used to model a monthly varying flux rate for a constant flux aquifer.

Constant head aquifers

A constant head aquifer can be defined in the `SOLUTION` section by specifying the [AQUCHWAT](#) keyword for a water aquifer, and by the [AQUCHGAS](#) keyword for a gas aquifer.

The aquifer connections to one or more faces of the reservoir should be made via the keywords [AQUANCON](#) and [AQANCONL](#). These keywords help to distinguish between connections to a global grid cell or a local grid cell.

To achieve time-dependency, the conditions may be modified during simulation by reentering the [AQUCHWAT](#) or [AQUCHGAS](#) keywords in the `SCHEDULE` section.

The aquifer inflow equations are given in the description of the [AQUCHWAT](#) and [AQUCHGAS](#) keywords respectively.

In order to use the facility, the parameters `NANAQU` (defining the maximum number of analytic aquifers) and `NCAMAX` (defining the maximum number of grid block connected to any single analytic aquifer) should be set using the `RUNSPEC` section keyword [AQUDIMS](#).

`SUMMARY` file quantities for the aquifer may be obtained using `AAQR/AAQRG` (influx rate) and `AAQT / AAQTG` (cumulative influx) in the `SUMMARY` section.

For tracer tracking runs, the concentrations of water- or gas-phase tracers in the aquifer may be supplied using the keyword [AQANTRC](#).

Multi-component analytic aquifers

Eclipse 300

Constant head gas aquifers defined by the [AQUCHGAS](#) keyword can be composed of any components that can be present in the gas phase.

Refer to the [AQUCHGAS](#) keyword description for more information.

When using one of the [CO2SOL](#), [CO2STORE](#), [COMPW](#), [GASWAT](#) options, any water analytic aquifers can be composed of any component that can be present in the water phase.

Refer to:

- "[Analytic water aquifers](#)" for the [CO2STORE](#) and [GASWAT](#) options,
- "[Aquifer properties](#)" for the [CO2SOL](#) option,
- "[Aquifer properties](#)" for the [COMPW](#) option.

For the multi-component aquifers listed above, the summary keywords [AACMR](#) and [AACMT](#) allow you to access the component molar influx rates and totals.

Summary of keywords

RUNSPEC

- [AQUDIMS](#) sets dimensions for aquifers

GRID

Restriction	Keyword	Description
	AQUCON	specifies connection data for numerical aquifers
	AQUCT	specifies the property data for Carter-Tracy aquifers
Eclipse 100	AQUNNC	set non-neighbor connection values explicitly for numerical aquifers.
	AQUNUM	assigns a numerical aquifer to a block.
	INIT	send aquifer connection data to the INIT file: Aquifer connection arrays AQUIFERN , AQUIFERA and AQUIFERG are written to the INIT file depending on whether numerical or analytic aquifers are present.
	RPTGRID	controls output from the GRID section to the print file: Use AQUCON and AQUNUM to output numerical aquifer connections and definitions respectively

PROPS

Restriction	Keyword	Description
Eclipse 300	AQUCT	specifies the property data for Carter-Tracy aquifers
	AQUATAB	influence function tables for Carter-Tracy aquifers.
Eclipse 100	RPTPROPS	controls output from the PROPS section to the print file: AQUATAB outputs Carter-Tracy influence tables.

SOLUTION

Restriction	Keyword	Description
	AQANCONL	specifies local grid refinement connection data for analytic aquifers
Eclipse 100	AQANNC	sets connection values explicitly for analytic aquifers.
	AQANTRC	sets initial tracer concentrations for analytic aquifers.
Eclipse 300	AQSTREAM	sets the composition of the injected gas stream for an AQUCHGAS aquifer; or sets the composition of the water analytic aquifers AQUCHWAT , AQUCT , AQUFETP or AQUFLUX for the GASWAT and CO2STORE options.
Eclipse 300	AQSTREAW	sets the composition of a multi-component water analytic aquifer
Eclipse 100	AQUALIST	sets up lists of aquifers for output
	AQUANCON	specifies connection data for analytic aquifers
	AQUCHGAS	specifies the property data for constant head/pressure gas aquifers
	AQUCHWAT	specifies the property data for constant head/pressure water aquifers
	AQUCT	specifies the property data for Carter-Tracy aquifers
Eclipse 100	AQUFET	specification data for Fetkovich aquifers
	AQUFETP	specifies the property data for Fetkovich aquifers
	AQUFLUX	specifies a constant flux aquifer
Eclipse 100	RAINFALL	specifies a monthly varying flux rate for a constant flux aquifer.
	RPTSOL	controls output from the SOLUTION section to the print file: To output analytic aquifer data such as Fetkovich, Carter-Tracy, Constant Flux or Constant Head use the AQUFET, AQUFETP, AQUCT, AQUFLUX, AQUANCON or AQANCONL arguments. Eclipse 100: If set to 2: data for each aquifer-grid block connection will also be printed (input using AQUANCON). Eclipse 300: Data for each aquifer-grid block connection will be printed using the argument. Note: AQUFET applies only for Eclipse 100. Note: AQANCONL applies only for Eclipse 300

SUMMARY

Refer to the following sections in the *Eclipse Reference Manual*.

- [Analytic aquifer quantities](#)
- [Numerical aquifer quantities](#)

SCHEDULE

Restriction	Keyword	Description
	AQSTREAM	sets the composition of the injected gas stream for an AQUCHGAS aquifer; sets the composition of an AQUCHGAS aquifer; or sets the composition of a GASWAT water analytic aquifer

Restriction	Keyword	Description
Eclipse 300	AQSTREAW	sets the composition of a multi-component water analytic aquifer.
	AQUCHGAS	specifies the property data for constant head/pressure gas aquifers
	AQUCHWAT	specifies the property data for constant head/pressure water aquifers
	AQUCT	specifies the property data for Carter-Tracy aquifers
Eclipse 100	AQUCWFAC	changes the property for all constant head aquifers
	AQUFETP	specifies the property data for Fetkovich aquifers
	AQUFLUX	specifies a constant flux aquifer
Eclipse 100	RAINFALL	specifies a monthly varying flux rate for a constant flux aquifer.
Eclipse 300	RPTPRINT	controls output of printed reports. Use item 13 of keyword to report aquifer flows.
	RPTSCHED	controls output from the SCHEDULE section to the print file: Use AQUCT to output the Carter-Tracy aquifer status or AQUFET, AQUFETP to output the Fetkovich aquifer status. Use AQUCHWAT, AQUCHGAS and AQUFLUX to output constant head/pressure or flux aquifer information. Note: AQUFET applies only to Eclipse 100.

Dual porosity

x	Eclipse 100
x	Eclipse 300

In a naturally fractured (for example carbonate) reservoir, fluids exist in two interconnected systems:

- The rock matrix, which usually provides the bulk of the reservoir volume
- The highly permeable rock fractures.

To model such systems in Eclipse, two simulation cells are associated with each block in the geometric grid, representing the matrix and fracture volumes of the cell. In Eclipse, the porosity, permeability, depth etc. of these may be independently defined. A matrix-fracture coupling transmissibility is constructed automatically by Eclipse to simulate flow between the two systems due to fluid expansion, gravity drainage, capillary pressure etc. This procedure is referred to as “dual porosity” modeling.

If the matrix blocks are linked only through the fracture system, this is considered to be a dual-porosity, single-permeability system, since fluid flow through the reservoir takes place only in the fracture network, with the matrix blocks acting as sources. However, if there is the possibility of flow directly between neighboring matrix blocks, this is conventionally considered to be a dual-porosity, dual-permeability system. Dual porosity runs are specified by the keyword `DUALPORO` in `RUNSPEC`, while dual permeability requires the `DUALPERM` keyword. It should be noted that dual porosity, dual permeability runs are computationally more expensive than dual-porosity, single-permeability runs.

In a dual porosity or dual permeability run of Eclipse, the number of layers in the Z-direction should be doubled. Eclipse associates the first half of the grid (the first `NDIVIZ / 2` layers) with the matrix blocks, and the second half with the fractures. In such runs `NDIVIZ` must therefore be even; Eclipse checks that this is the case.

For local grid refinements, the `NDIVIZ / 2` rule is applied to the number of refined cells (item 10 in `CARFIN`) in the LGR. The position of the LGR in the host grid (`CARFIN` items 6 and 7) is specified by its location in the matrix; it will automatically be duplicated in the fracture.

Eclipse 300

Note that for Eclipse 300, if the multiporosity option is used (see “Multi porosity” and keyword `NMATRIX`) then the first `NDIVIZ/N` layers will be the matrix blocks, where `N` is the number of porosities. References to `NDIVIZ / 2` in this chapter should be checked against the multi porosity documentation if there is more than one matrix porosity.

Eclipse 100

However, for the “Discretized matrix model (Eclipse 100)” (“Russian Doll model”) the additional blocks are all internal to the Eclipse calculation, and the matrix in the grid is always `NDIVIZ/2` layers regardless of the number of matrix subdivisions. This is because the properties of the matrix sub-cells cannot be specified independently, but are all copies of the same matrix cell data.

Transmissibility calculations

The matrix-fracture coupling transmissibility terms that exist between each cell of the matrix grid and the corresponding cell in the fracture grid are proportional to the cell bulk volume, being of the form:

$$TR = CDARCY \cdot K \cdot V \cdot \sigma \quad \text{Eq. 2.54}$$

where by default:

K is taken as the X-direction permeability of the matrix blocks (unless `PERMMF` is also specified; see below),

V is the matrix cell bulk volume (note that this is not the pore volume, having no porosity factor), and

σ is a factor of dimensionality LENGTH^{-2} , to account for the matrix/fracture interface area per unit volume, that is the size of the blocks in the matrix volume.

The default use of the X-direction permeability can be overridden by using the matrix-fracture permeability specified by the `PERMMF` keyword. Note that for one-dimensional (Y- or Z-direction) cases in the global grid with X-direction extension in an LGR, the X-direction permeability will be set to zero in all grids and `PERMMF` must be used to enable dual porosity NNCs to be generated. (This is unlikely in real simulations, but many test problems are affected by this.)

Kazemi (1976) has proposed the following form for σ :

$$\sigma = 4 \left(\frac{1}{l_x^2} + \frac{1}{l_y^2} + \frac{1}{l_z^2} \right) \quad \text{Eq. 2.55}$$

where l_x , l_y and l_z are typical X, Y and Z dimensions of the blocks of material making up the matrix volume. (l_x , l_y and l_z are thus not related to the simulation grid dimensions). Alternatively, as σ acts as a multiplier on the matrix-fracture coupling, it may simply be treated as a history matching parameter.

σ can be specified as a single value for the whole field using the keyword `SIGMA` or on a cell by cell basis using the keyword `SIGMAV`. If σ is defined on a cell by cell basis then the values corresponding to the first `NDIVIZ / 2` layers are used. If σ is printed out (`RPTGRID` keyword) then the values in the first `NDIVIZ / 2` layers are copied in to the lower `NDIVIZ / 2` layers.

The specified values of the permeabilities of cells in the fracture system are multiplied by the fracture porosities to yield effective fracture permeabilities. If the `NODPPM` keyword is used then the specified permeability is used directly, rather than the effective permeability.

That is, when `NODPPM` is **not** specified, then within the fracture cells, the input `PERMX`, `PERMY` and `PERMZ` are modified using $PERMX(fr) = PERMX(fr) \times PORO(fr)$.

Eclipse 100

An alternative form of the above transmissibility is available using the `LTOSIGMA` keyword if the matrix block dimensions are input explicitly using the `LX`, `LY` and `LZ` keywords. In this case the transmissibility is given by:

$$TR = CDARCY \cdot V \cdot \left(\frac{f_x \cdot K_x}{l_x^2} + \frac{f_y \cdot K_y}{l_y^2} + \frac{f_z \cdot K_z}{l_z^2} \right) \quad \text{Eq. 2.56}$$

where:

$f_x f_y f_z$ are input in `LTOSIGMA`

$K_x K_y K_z$ are the matrix permeability values

$l_x l_y l_z$ are the matrix block dimensions input using `LX`, `LY` and `LZ`.

If the dual porosity but not the dual permeability option is selected, the matrix blocks have no transmissibilities between them. If dual porosity and dual permeability is chosen, the matrix blocks have their normal transmissibilities.

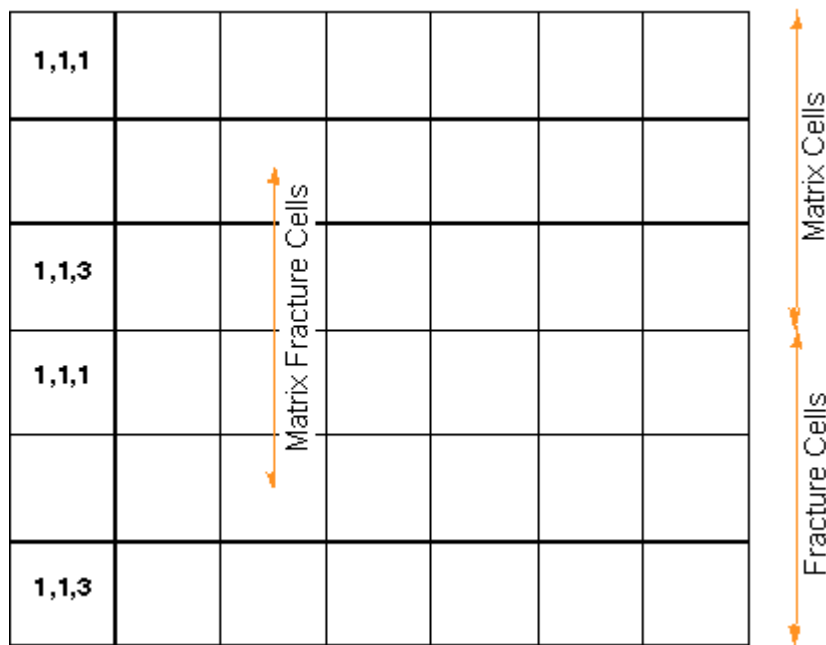


Figure 2.15. A simple, dual porosity, dual permeability system.

Block to block transmissibility calculations

It is now also possible to represent re-imbibition from a fracture cell by the matrix cell below it. In this case the transmissibility between the matrix and fracture cells is a conventional spatial one, since the two cells are not superimposed on one another, and the properties of the two cells contribute to the transmissibility in exactly the same way as for the transmissibility between two neighboring fracture cells, or two neighboring matrix cells. See "[Transmissibility calculations](#)" for the details of this calculation. This modeling is most appropriate when the physical matrix block size (l_z as used in the Kazemi formula for the usual matrix-fracture transmissibility multiplier represented by the [SIGMA](#) and [SIGMAV](#) keywords) is comparable to the size of the grid cell rather than significantly different from it, so that the environment of a lower matrix physical block also comprises neighboring fracture cells rather than just its own fracture cell.

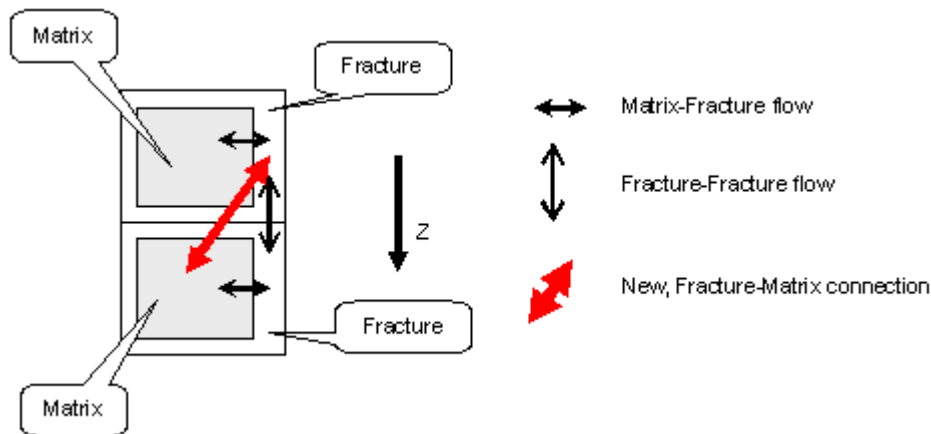


Figure 2.16. Physical view of the matrix and fracture cells for the block to block connection

Figure 2.17 shows this new transmissibility on a picture of the two physical cells (that is the matrix and fracture cells occupy the same physical location.) This figure appears to show the lower matrix disconnected from the upper fracture by its own surrounding fracture, but this is only for presentation purposes: there will always be some contact between the upper fracture and lower matrix. This is schematically represented in figure 2.18, where the lower matrix and fracture are shown as single adjacent blocks, but the upper matrix is split in two, one half abutting the lower matrix (a disconnection in dual porosity but a connection in dual permeability) and one half abutting the lower fracture (not represented in the model, but conceptually possible). The upper fracture contacts both lower fracture and lower matrix.

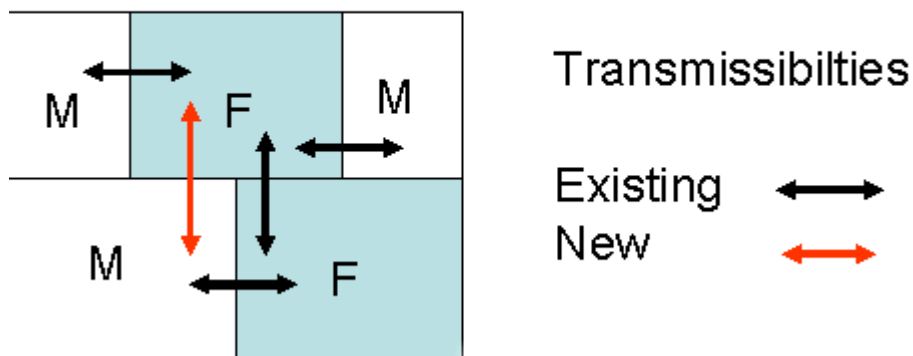


Figure 2.17. Schematic view of the contacts between the upper and lower cells of the two porosity systems

The extent of the contact between the upper fracture and lower matrix is a variable of the model, and hence a new multiplier is applied to the transmissibility between these cells, to represent this effective contact area between the matrix and the fracture, which will be different from the contact area between the two vertically neighboring fracture cells, or two vertically neighboring matrix cells. This contact area multiplier is specified by the keywords `BTOTALFA` or `BTOTALFV`; the former specifies a single value for the whole grid, while the latter specifies it cell by cell. The presence of these keywords also activates these additional transmissibilities, which will otherwise not be present.

Eclipse 100

In Eclipse 100, block-to-block connections between LGRs and their parents or adjacent LGRs are only supported for in-place LGRs.

Note: Block to block connections cannot be made when a multi porosity model is specified (`NMATRIX` or `TRPLPORO`).

Recovery mechanisms

In a dual porosity system the majority of the oil is contained in the matrix system, but the production of oil to the wells is through the high permeability fracture system. In such a system an injected fluid does not sweep out oil from the matrix block. Production from the matrix blocks can be associated with various physical mechanisms including:

- Oil expansion
- Imbibition
- Gravity imbibition/drainage

- Diffusion
- Viscous displacement.

Oil expansion

As the pressure drops in the fracture system, oil flows from the matrix to equilibrate the matrix pressure with the surrounding fracture pressure. This production mechanism can be thought of as expansion of the oil within the matrix block, either above the bubble point or by solution gas drive below the bubble point.

Imbibition

In a typical water wet system the matrix rock has a positive water-oil capillary pressure. If water is introduced into the fracture, the water flows under capillary forces into the matrix system, displacing oil. Note that if the gravity drainage model (see below) is not active, then the production continues until the oil saturation in the matrix has reached residual oil.

The water imbibition process is modeled in Eclipse by specifying different saturation table numbers for the matrix and fracture cells respectively. The matrix cells typically have a water-oil capillary pressure, while the fracture cells usually have zero capillary pressure.

In gas-oil systems the oil is the wetting phase and tends to imbibe into the matrix. In practice this means that if the gravity drainage model is not active then no oil production occurs from a matrix block when the associated fracture block is full of gas.

Gravity imbibition/drainage

Fluid exchange between the fracture and matrix due to gravity is modeled in one of two ways. One method is to consider each of the matrix and fracture cells as separately in vertical equilibrium, and then to calculate additional potential due to differences in contact heights between the matrix and fracture. There are two options for this, which are activated by including either of the keywords [GRAVDR](#) or [GRAVDRM](#) (described in the next two sections) in the RUNSPEC section. This method leaves the number of porosities in the grid as 2.

The other method (described in the section following) uses a modification of the Discrete Matrix model (Eclipse 100) or multi porosity model (Eclipse 300), and uses N matrix porosities, where N is user-defined, to create a vertical stack of finely spaced matrix cells which describe the distribution of properties within a single block of matrix material (with height as defined by the [DZMTRX](#) keyword). This option is not available with dual permeability runs, which should use a more detailed discretization of the whole grid; the advantage of discretizing only the matrix is that its equations can be eliminated from the solution before the linear solution is performed, so that no additional time is spent in that part of the code.

The [DZMTRX](#) keyword is also required for the [GRAVDR](#) or [GRAVDRM](#) models, and a non-zero value should be given for it in the GRID section in all gravity drainage cases.

Standard gravity drainage model (GRAVDR)

The illustration shows a typical block of matrix material containing oil and water. The fractional height of the water table in the fracture is X_W and the fractional height of the water displacement front in the block of matrix material is X_w .

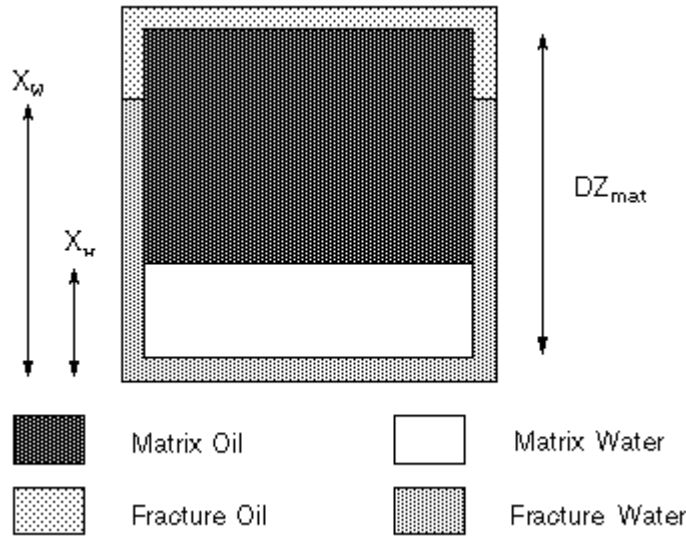


Figure 2.18. A typical block of matrix material containing oil and water

The pressure difference due to gravity alone is

$$\Delta P = DZ_{mat}(X_W - X_w)(\rho_w - \rho_o)g \quad \text{Eq. 2.57}$$

where

ρ_w is the density of water

ρ_o is the density of oil at reservoir conditions.

DZ_{mat} is the same as l_z in Kazemi's formula.

Integrating ΔP over all blocks of matrix material within a reservoir simulation cell leads to the same formula for ΔP , except that it now represents the gravitational pressure difference between reservoir simulation fracture and matrix cells, and X_W and X_w are the partial volumes of mobile water in the simulation fracture and matrix cells. In Eclipse, the effect is modeled by introducing additional pseudo capillary pressures, P_S

$$P_{Scow} = DZ_{mat} \cdot X_W(\rho_o - \rho_w)g \text{ (fracture cells)}$$

$$P_{Scow} = DZ_{mat} \cdot X_w(\rho_o - \rho_w)g \text{ (matrix cells)}$$

$$P_{Scog} = DZ_{mat} \cdot X_G(\rho_o - \rho_g)g \text{ (fracture cells)}$$

$$P_{Scog} = DZ_{mat} \cdot X_g(\rho_o - \rho_g)g \text{ (matrix cells)}$$

$$P_{Scwg} = DZ_{mat} \cdot X_G(\rho_w - \rho_g)g \text{ (fracture cells)}$$

$$P_{Scwg} = DZ_{mat} \cdot X_g(\rho_w - \rho_g)g \text{ (matrix cells)}$$

Thus, the flow of oil and gas from a fracture to a matrix cell in a gas oil system is computed as

$$F_g = TR \cdot GMOB \times \left(P_{of} - P_{om} + d_{fm} \rho_g g + P_{cogf} - P_{cogm} + \frac{DZ_{mat}(X_G - X_g)(\rho_o - \rho_g)g}{2} \right) \quad \text{Eq. 2.58}$$

$$F_o = TR \cdot OMOB \left(P_{of} - P_{om} + d_{fm} \rho_o g - \frac{DZ_{mat}(X_G - X_g)(\rho_o - \rho_g)g}{2} \right) \quad \text{Eq. 2.59}$$

where

TR is the transmissibility between the fracture and matrix cells,

GMOB is the gas mobility in the (upstream) fracture cell,

OMOB is the oil mobility in the (upstream) fracture cell,

P_{of} is the oil phase pressure in the fracture cell,

P_{om} is the oil phase pressure in the matrix cell,

d_{fm} is the difference in depth between the fracture and matrix cells (usually zero),

ρ_g is the density of gas at reservoir conditions,

ρ_o is the density of oil at reservoir conditions,

g is the acceleration due to gravity,

P_{cogf} is the capillary pressure of gas in the fracture cell (normally zero)

P_{cogm} is the capillary pressure of gas in the matrix cell.

Note: The gravity drainage head $DZ_{mat}(X_G - X_g)(\rho_o - \rho_g)g$ has been split between the two phases.

In dual porosity / dual permeability models in which the gravity imbibition / drainage mechanism is active, the calculation of the initial reservoir state (see ["Initializing the study"](#)) occurs independently of the value of DZ_{mat} that is the reservoir is initialized without regard for the gravity imbibition / drainage forces acting between the matrix and fracture cells. When the simulation begins, the gravity imbibition / drainage forces may cause fluid movement between matrix cells and their corresponding fracture cells due to the different fluid levels in the matrix / fracture cells. If the redistribution of fluids causes a significant transient when the simulation is started, you can overcome this by activating [item 11](#) of the `OPTIONS` keyword. If this switch is activated, the effect is to cause modifications to the phase pressures to make the initial solution a true steady state. These phase pressure modifications are applied for the duration of the run.

Eclipse 300

In Eclipse 300, a different value of the σ coupling may be specified for gravity drainage by using the `SIGMAGD` keyword. If the values entered with `SIGMA` and `SIGMAGD`, [denoted σ and σ_{gd} respectively] differ, the program uses an interpolated σ value for each phase which depends on the relative strengths of the capillary pressure and gravity drainage terms:

$$\sigma_p^{\text{eff}} = f_p \cdot \sigma + (1 - f_p) \cdot \sigma_{gd} \quad \text{Eq. 2.60}$$

where p = oil, gas, water, and

$$f_p = \frac{|\Delta P_p|}{|\Delta P_p| + |\Delta P_{pgd}|} \quad \text{Eq. 2.61}$$

ΔP_{pij} is the usual potential difference term between two cells i and j :

$$\Delta P_{pij} = P_{pi} - P_{pj} + P_{pci} - P_{pcj} - g\bar{\rho}_p \Delta d \quad \text{Eq. 2.62}$$

where

(P_{pi}, P_{pj}) are the cell pressures,

(P_{pci}, P_{pcj}) are the cell capillary pressures,

Δd is the height difference between the cells,

g is the acceleration due to gravity

$\bar{\rho}_p$ is the saturation weighted average density of phase p

ΔP_{gd} is the gravity drainage term, which depends on the phase:

$$\Delta P_o = -\frac{1}{2} (\Delta P_{gd}^{og} + \Delta P_{gd}^{ow}) \quad \text{Eq. 2.63}$$

$$\Delta P_g = \frac{1}{2} \Delta P_{gd}^{og} \quad \text{Eq. 2.64}$$

$$\Delta P_w = \frac{1}{2} \Delta P_{gd}^{ow} \quad \text{Eq. 2.65}$$

$$\Delta P_{gd}^{og} = g \times DZ_{mat} (\bar{\rho}_o - \bar{\rho}_g) (X_{ig} - X_{jg}) \quad \text{Eq. 2.66}$$

$$\Delta P_{gd}^{ow} = g \times DZ_{mat} (\bar{\rho}_w - \bar{\rho}_o) (X_{iw} - X_{jw}) \quad \text{Eq. 2.67}$$

where X_g and X_w are the fractions of the cell containing mobile gas and water respectively. These are estimated from the cell saturation using a vertical equilibrium model. If the gravity drainage term is dominant, $f_p \rightarrow 0$ and $\sigma_{\text{eff}} \rightarrow \sigma_{gd}$.

Eclipse 100

Note that in Eclipse 100, the SIGMAGD keyword has a different function, and is used to differentiate oil-gas and oil-water mechanisms. See ["Variation of the sigma factor with drainage process"](#).

Alternative gravity drainage model (GRAVDRM)

An alternative gravity drainage model is available based on the formulation proposed by [Quandalle and Sabathier \(1989\)](#).

The matrix-fracture flow is taken to be the sum of three flows from the center of the matrix to the fracture system: one horizontal, one vertically upwards and one vertically downwards. This formulation allows for different flow paths and hence transmissibilities in the vertical and horizontal directions. In some circumstances, typically in mixed wettability systems, the two transmissibilities give a better match to a finely gridded matrix block model.

The matrix fracture flow is given by:

$$F = F_h + F_{up} + F_{down} \quad \text{Eq. 2.68}$$

as shown in the following figure:

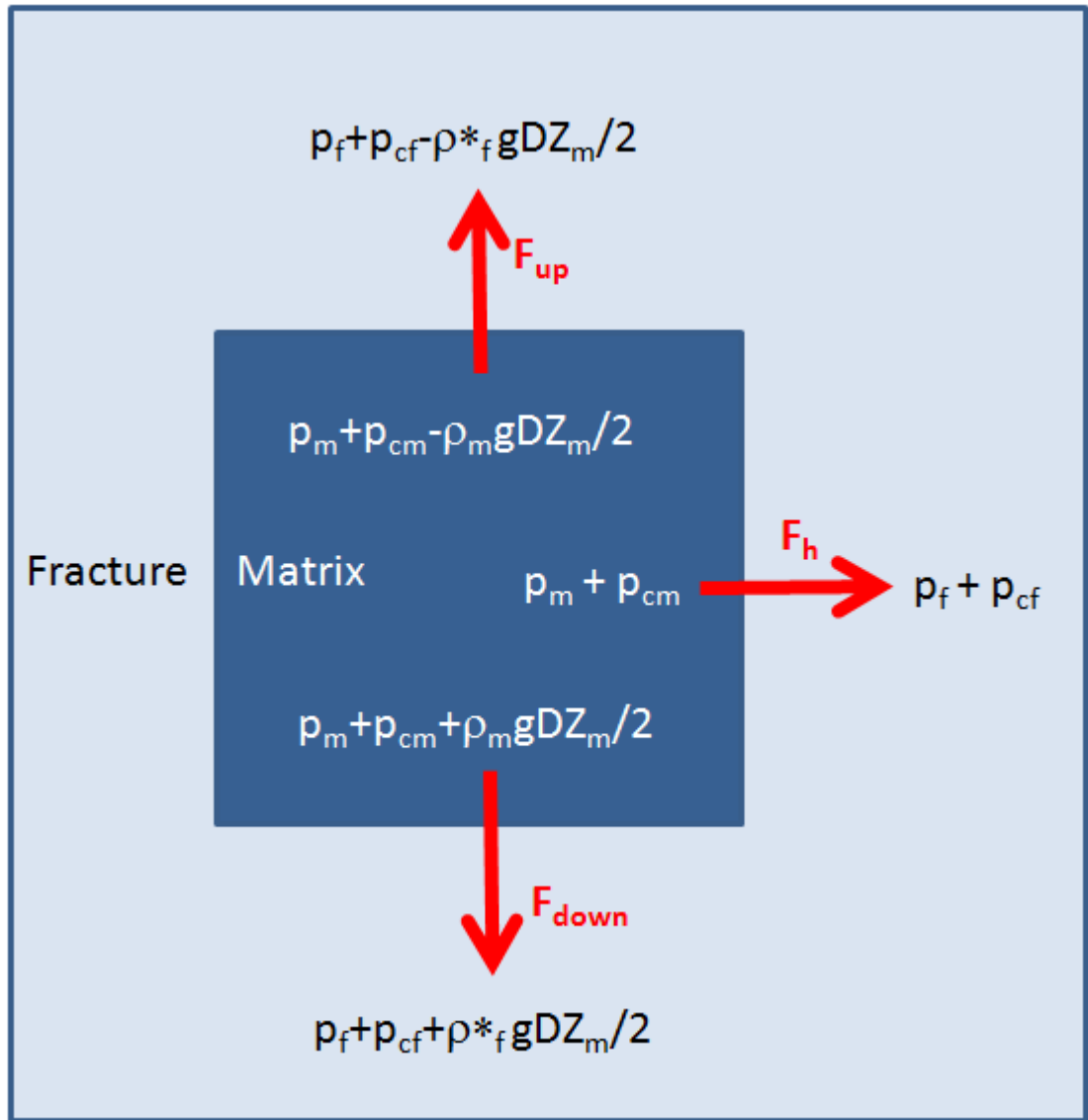


Figure 2.19. Flows and potentials in the GRAVDRM model

The horizontal flow is:

$$F_h = TR_{sigma} \cdot MOB_h \cdot (P_f + P_{cf} - P_m - P_{cm}) \quad \text{Eq. 2.69}$$

where:

TR_{sigma} is the transmissibility based on the [SIGMA](#) keyword (that is defined by equation 2.54 with σ values from the [SIGMA](#) keyword, and K described as in the text following equation 2.54). Note that there is no switching between the [SIGMA](#) and [SIGMAGD](#) keywords in this model: there is no effective σ as in equation 2.60 for Eclipse 300, or switching between [SIGMA](#) and [SIGMAGD](#) as described for Eclipse 100 in the [SIGMAGD](#) keyword description.

MOB_h is the phase (upstream) mobility. See MOB_v below for the definition of upstream.

P_f, P_m are the oil phase pressures in the fracture and matrix cells

P_{cf}, P_{cm} are the capillary pressures.

The vertical flows are:

$$F_{down} = TR_{sigmagd} \cdot MOB_{down} \cdot (P_f + P_{cf} - P_m - P_{cm} + (\rho_m - \rho_f^{star}) \cdot g \cdot (DZ_{mat})/2) \quad \text{Eq. 2.70}$$

$$F_{up} = TR_{sigmagd} \cdot MOB_{up} \cdot (P_f + P_{cf} - P_m - P_{cm} - (\rho_m - \rho_f^{star}) \cdot g \cdot (DZ_{mat})/2) \quad \text{Eq. 2.71}$$

where:

$TR_{sigmagd}$ is the transmissibility based on the `SIGMAGD` keyword

MOB_v (where $v = up \text{ or } down$) is the mobility of the fluid in the upstream cell for the relevant phase. The upstream cell is determined from the sign of the potential difference (which is the whole parenthesized expression in either equation 2.69, equation 2.70 or equation 2.71: this includes the gravitational potential energy term. The upstream cell has the higher total potential).

DZ_{mat} is the matrix block height given in the `DZMATRIX` keyword

g is the acceleration due to gravity

ρ_m is the phase density in the matrix

ρ_f^{star} is given by $\rho_f^{star} = S_w \cdot \rho_w + S_o \cdot \rho_o + S_g \cdot \rho_g$

One drawback with this formulation is that the final recovery from a block can become a function of the transmissibility in the case when the final recovery is given by $F = 0$ when the vertical flows balance the horizontal $F_h = -F_{up} - F_{down}$. This effect can be avoided (in the case of oil) by invoking an option to prevent the re-infiltration of oil; see the `GRAVDRM` keyword in the `RUNSPEC` section.

Eclipse 100

Note that the `SIGMA` and `SIGMAGD` values can be calculated from the block dimensions by using the `LTOSIGMA` keyword in conjunction with the `LX`, `LY` and `LZ` keywords.

Vertical discrete matrix gravity drainage model

The third alternative gravity drainage model uses the discrete matrix model (Eclipse 100) or multi porosity model (Eclipse 300), both of which provide a number of matrix porosities connected together in a chain, with the outermost connected to the fracture. If the `NMATOPTS` keyword is set to the value `VERTICAL`, then this chain of matrix porosities will be interpreted as a vertical stack of matrix cells connected within a single block of matrix material of height `DZMATRIX`. This value of the keyword will also connect all submatrix porosities to the fracture. The number of porosities used will still be defined by the `NMATRIX` keyword.

When these options are activated, the physical picture of the fracture cell and its associated matrix cells is as shown in figure 2.21. Each matrix cell sees the same fracture cell conditions. To prevent this limitation reducing the accuracy of the solution unacceptably, the properties of the fracture cell are altered for each NNC describing the matrix-fracture links to agree better with the property variations with height which would be experienced if the fracture were similarly discretized. This alteration takes into account the change with height of gravitational potential and any phase discontinuities (which are described in the same way as for the `GRAVDR` model).

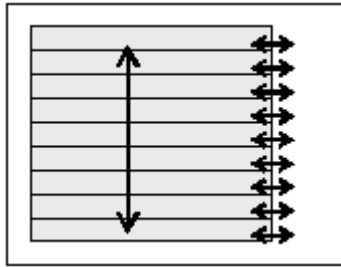


Figure 2.20. The discretized matrix material block within a fracture cell

Note: The connections to the fracture are shown explicitly; the discrete matrix model connections are shown as a single arrow.

Eclipse 300

In Eclipse 300, the input requirements for multi porosity give greater flexibility in the variation of properties at different heights of the matrix, but it is recommended that all the porosities are given the same property values.

Diffusion

Molecular diffusion of gas and oil between the matrix and fracture may be a significant production mechanism from the matrix. The Eclipse diffusion option is described in "Diffusion".

Viscous displacement (Eclipse 100)

Viscous displacement of a fluid is simply the movement of that fluid when a pressure differential is applied. In a dual porosity system there is a pressure gradient in the fracture system moving the fluid through the fracture, towards the production wells. In many cases this pressure gradient is small as the fracture system has a very high effective permeability. In these cases it is reasonable to ignore the viscous displacement of fluids from the matrix by the fracture pressure gradient. However, if the fracture system has a more moderate permeability then flow to and from the matrix caused by the fracture pressure gradient may be expected to act as a significant production mechanism.

The viscous displacement option that models this effect is activated using the `VISCD` keyword in the `RUNSPEC` section. The representative matrix block sizes are input using the `LX`, `LY` and `LZ` keywords in the `GRID` section.

Provided that the representative matrix block sizes have been input using the `LX`, `LY` and `LZ` keywords, it is possible to calculate the `SIGMA` factor from these matrix block dimensions using the `LTOSIGMA` keyword. Note that if `SIGMA` is calculated in this way then any input `SIGMA` data will be ignored.

The Eclipse viscous displacement model is based on a technique described by *Gilman and Kazemi (1988)*.

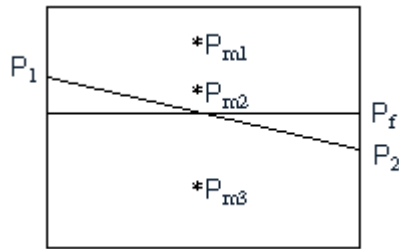


Figure 2.21. A representative matrix block in a fracture potential gradient

Consider a representative matrix block in a fracture potential gradient. The average fracture potential is given by P_f , with an effective pressure drop across the block of $P_1 - P_2$.

If the matrix pressure is P_{m1} , which is greater than both P_1 and P_2 then clearly the flow will be from matrix to fracture,

$$F = \frac{T}{2} \cdot M_m (P_{m1} - P_1) + \frac{T}{2} \cdot M_m (P_{m1} - P_2) \quad \text{Eq. 2.72}$$

where:

T is the matrix-fracture transmissibility

M_m is the matrix mobility.

Hence

$$F = T \cdot M_m (P_{m1} - P_f) \quad \text{Eq. 2.73}$$

that is the usual matrix-fracture flow since $P_f = (P_1 + P_2)/2$.

If the matrix pressure is lower than both P_1 and P_2 (for example P_{m3}) then the flow is from fracture to matrix, and as above can be expressed as:

$$F = T \cdot M_f (P_f - P_{m3}) \quad \text{Eq. 2.74}$$

where:

M_f is the fracture mobility.

But in the case where the matrix pressure is P_{m2} , where $P_1 > P_{m2} > P_2$, flow will occur from fracture to matrix at one end and from matrix to fracture at the other. In this case the flow is given by

$$F = \frac{T}{2} \cdot M_f \cdot (P_1 - P_{m2}) - \frac{T}{2} \cdot M_m \cdot (P_{m2} - P_2) \quad \text{Eq. 2.75}$$

This can be written as

$$F = \frac{T}{2} \cdot M_f \cdot \left(P_f + \frac{(P_1 - P_2)}{2} - P_{m2} \right) - \frac{T}{2} \cdot M_m \cdot \left(P_{m2} - P_f + \frac{(P_1 - P_2)}{2} \right) \quad \text{Eq. 2.76}$$

P_1 and P_2 are not known directly in the simulator, but an estimate of the fracture potential gradient can be made from the adjacent cell pressures. Now

$$P_1 - P_2 = G \cdot L$$

where:

G is the fracture pressure gradient

L is the representative dimension of the matrix block in the appropriate direction, input using the `LX`, `LY` and `LZ` keywords.

Equation 2.76 can be written as a standard matrix-fracture flow with modified up-winding and an extra term associated with the viscous displacement

$$F = \frac{T}{2} \cdot M_f \cdot (P_f - P_{m2}) - \frac{T}{2} \cdot M_m \cdot (P_{m2} - P_f) + \frac{T}{4} \cdot G \cdot L \cdot (M_f - M_m) \quad \text{Eq. 2.77}$$

The matrix-fracture flows are modified in a similar manner for all three directions.

The fracture potential gradient, G , is estimated from the potential differences to the surrounding cells.

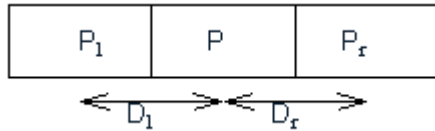


Figure 2.22. Estimating the potential differences to the surrounding cells

$$G = \frac{|P_l - P|}{2D_l} + \frac{|P - P_r|}{2D_r} \quad \text{Eq. 2.78}$$

where:

$|P_l - P|$ and $|P - P_r|$ are the absolute values of the potential difference to adjacent cells

D_l and D_r are the distance between the cell centers.

In this version of Eclipse the fracture potential gradients are calculated from the potential differences at the previous timestep.

Facilities specific to dual porosity runs

Fracture permeabilities

It is necessary to be clear about what the input fracture permeability represents. By default Eclipse multiplies the input fracture permeabilities by the fracture porosity to generate an effective permeability. If the input fracture permeabilities themselves represent the effective value, then the `NODPPM` keyword should be used in the `GRID` section.

Matrix-fracture relative permeabilities

Usually two sets of saturation functions (relative permeabilities and capillary pressures) are supplied, one for the matrix cells and one for the fracture cells. The `SATNUM` keyword in the `REGIONS` section is used to associate the tables with the appropriate grid blocks. As Eclipse ‘upwinds’ the flows, a phase flow from matrix to fracture takes the relative permeability calculated from the phase saturation in the matrix using the matrix table. Flow from the fracture to the matrix uses the fracture saturation and table.

Eclipse 100

If a separate relative permeability table is required for the matrix/fracture flow, this can be specified using the `KRNUMMF` keyword in the `REGIONS` section. If the Hysteresis option is active, then a corresponding keyword `IMBNUMMF` can be used to specify the imbibition relative permeability table.

When flow occurs from fracture to matrix it may be more reasonable to regard the maximum relative permeability as the matrix relative permeability at the residual saturation of the displaced phase. This effect can be easily achieved without specifying separate matrix-fracture tables (`KRNUMMF`), by scaling the fracture relative permeability. This scaling can be activated by using the third data item in the `DPKRMOD` keyword.

Simplified grid input

The keyword `DPGRID` may be used to simplify the construction of grids for dual porosity runs. This keyword enables grid data to be entered for the matrix cells only (the first `NDIVIZ / 2` layers), the missing values for the remaining fracture layers being obtained from the corresponding matrix cell.

Note: Only certain keywords can be copied to the fracture in this way. Check the `DPGRID` keyword for the complete list

Use of FLUXNUM or MULTNUM or OPERNUM

The use of these keywords differs between Eclipse 100 and Eclipse 300.

Eclipse 100

Only the matrix cells need to be input. Eclipse 100 will copy these values to the fracture cells automatically. If both matrix and fracture cells are given, a warning message would be output complaining about too much data. If you want to specify matrix cells and fracture cells separately, then set [item 117](#) of the `OPTIONS` keyword to be greater than 1.

Eclipse 300

Both matrix and fracture cells should be provided. `FLUXNUM` and `MULTNUM` use with `MULTREGT` or `MULTREGP`, will automatically copy all matrix cells to the corresponding fracture cells, so dummy fracture values could be input. For `OPERNUM` use, and other `FLUXNUM` and `MULTNUM` uses, there is no copying of data from the matrix to fracture cells.

Partially fractured reservoirs

In a dual-porosity single-permeability run, the simulation model may be specified to be dual porosity only in part of the reservoir, using the keyword `DPNUM`. Eclipse treats the rest of the reservoir as in a conventional single porosity run. In this case, the single porosity cells are taken to be matrix cells for input purposes, and any data input for fracture cells in the single porosity region is ignored.

Diffusion

The Molecular Diffusion option can be limited to calculate only the matrix/fracture diffusive terms, by using the `DIFFDP` keyword. The assumption is that diffusion in the fracture system is negligible compared to the fracture flows. The matrix/fracture diffusivity can be modified using the `DIFFMMF` keyword in the `GRID` and `SCHEDULE` sections.

Modifying the recovery versus time response

Eclipse 100

The shape of the recovery versus time response can be modified by changing the matrix relative permeability curves. Eclipse provides a simplified method of changing the input relative permeabilities using only a single parameter. Typically this is useful when matching a dual porosity cell to a finely gridded

single porosity model. The facility is described in more detail in a subsequent section and is activated with the `DPKRMOD` keyword.

Integrated capillary pressure option

This option automatically generates a modified capillary pressure function with the aim of improving the prediction of final recovery from a matrix cell. The option is described in detail in a subsequent section and is activated with the `INTPC` keyword.

Calculating SIGMA from the matrix dimensions

If the Viscous Displacement option has been used, the representative matrix block sizes will have to be entered using the `LX`, `LY` and `LZ` keywords. It is possible to calculate the `SIGMA` value from these block dimensions by specifying the `LTOSIGMA` keyword. Note that if this keyword is used then any input `SIGMA` values are ignored.

Restrictions on dual porosity runs

The following restrictions apply to dual porosity (`DUALPORO`) runs (except in single porosity regions specified using `DPNUM`), but **not** to dual porosity, dual permeability (`DUALPERM`) runs.

- Wells connect only to fracture cells - not to matrix cells.
- Non-neighbor connections (keyword `NNC`) may not be used with matrix cells. The internal connection of each matrix cell to its appropriate fracture cell is made automatically by Eclipse.
- Each active matrix cell must connect with an active fracture cell.

In single porosity regions, there are no active fracture cells. Within these regions, data is only required for the matrix cells; any data for the fracture cells are ignored. Wells should connect to matrix blocks within single porosity regions.

For a coal bed methane model, the `DUALPERM` option should only be used if the instant adsorption model in Eclipse 300 is activated (`CBMOPTS`) or if the time-dependent adsorption model is used with Multi-Porosity. With the time-dependent sorption model, one of the submatrix porosities will then represent the pore volume of the matrix having conductivity to neighbor matrix grid blocks, while grid cells assigned to be coal will get a zero transmissibility.

Keyword summary

Restriction	Keyword	Description
GRID keyword	<code>BTOBALFA</code>	Initiates block to block connections and specifies a single contact area multiplier.
GRID keyword	<code>BTOBALFV</code>	Initiates block to block connections and specifies contact area multipliers.
Switch in RUNSPEC	<code>DUALPORO</code>	Initiates the dual porosity model.
Switch in RUNSPEC	<code>DUALPERM</code>	Initiates the dual permeability model.
Switch in RUNSPEC	<code>GRAVDR</code>	Requests alternative gravity drainage model
Switch in RUNSPEC	<code>GRAVDRM</code>	Requests alternative gravity drainage model

Restriction	Keyword	Description
RUNSPEC keyword	NMATRIX	In Eclipse 100, requests the discretized matrix model. In Eclipse 300, requests the equivalent discretized matrix if the NMATOPTS option is used - otherwise see " Multi porosity ".
SIGMA or SIGMAV required	SIGMA	Matrix-to-fracture coupling factor for the entire grid.
SIGMA or SIGMAV required	SIGMAV	Matrix-to-fracture coupling factor for the current box.
SIGMA or SIGMAV required	MULTSIG	Matrix-to-fracture coupling multiplier for the entire grid.
SIGMA or SIGMAV required	MULTSIGV	Matrix-to-fracture coupling multiplier for the current box.
DZMTRX or DZMTRXV required if gravity drainage is active.	DZMTRX	Specifies the typical block height for gravity drainage for the entire grid.
DZMTRX or DZMTRXV required if gravity drainage is active	DZMTRXV	Specifies the typical block height for gravity drainage for the current box.
Eclipse 100 Optional when the viscous displacement option is active	LX	Specifies the representative X dimension of the matrix block.
Eclipse 100 Optional when the viscous displacement option is active	LY	Specifies the representative Y dimension of the matrix block.
Eclipse 100 Optional when the viscous displacement option is active	LZ	Specifies the representative Z dimension of the matrix block.
Eclipse 100 Optional when the viscous displacement option is active	DIFFDP	Restricts the molecular diffusion calculation to just the matrix-fracture flows.
Optional	DPGRID	Enables you to enter grid data for the matrix cells only.
Optional	DPNUM	Identifies the extent of the regions in which the single porosity model may be applied in a dual porosity run.
Eclipse 100 Optional if the hysteresis option is active	IMBNUMMF	Imbibition table region for matrix-fracture flows.
Eclipse 100 Optional	KRNUMMF	K_r region for matrix-fracture flows.
Optional	NODPPM	Specifies that permeabilities in the fracture cells are not to be multiplied by the fracture porosity.
Recommended	SATNUM	To specify different saturation region to the matrix and fracture cells.

Restriction	Keyword	Description
Optional	SIGMAGD	In Eclipse 100: Matrix-to-fracture coupling for oil-gas gravity drainage, for the entire grid. If this is used, SIGMA is the coupling for oil-water gravity drainage In Eclipse 300: Alternative matrix-fracture coupling for all phases for gravity-drainage dominated potentials.
Optional	SIGMAGDV	SIGMAGD by cell for the current box.
Optional	MULSGGD	Matrix-to-fracture multiplier for SIGMAGD, for the entire grid.
Optional	MULSGGDV	Matrix-to-fracture multiplier for SIGMAGD by cell, for the current box.
Optional	INTPC	Invokes the integrated capillary pressure option
Eclipse 100 Optional	DPKRMOD	Modifies oil K_r curve for matching fine grid single porosity recovery vs. time, and allows scaling of the fracture relative permeability.
Eclipse 100 Optional if the viscous displacement option is active	LTOSIGMA	Requests that the SIGMA value is to be calculated from the LX, LY and LZ data.
Optional if the discretized matrix option is active	NMATOPTS	Specifies the geometry type of the matrix and the size of the outer matrix cell.
Eclipse 300 Optional	PERMMF	Permeability for Matrix-Fracture Coupling.
Eclipse 300 Optional	MULTMF	Multiplier for Matrix-Fracture Permeability.
Optional	DIFFMMF	Multiplier for Matrix-Fracture Diffusivities.

Table 2.1: Summary of dual porosity keywords

Transfer functions

Transfer functions are curves representing the oil expelled from a matrix element as a function of time. To simulate a gas/oil transfer function, set up a reservoir grid as shown below. Make the bottom matrix cell inactive using keyword [ACTNUM](#). Fill the fracture cells, F1 and F2, with gas and the matrix cell, M1, with oil and connate water using the appropriate keywords in the SOLUTION section. Include keyword [GRAVDR](#) in the RUNSPEC section. Record the oil saturation in cell F2 using BOSAT in the SUMMARY section. Run the simulation for a few years. Display the transfer function using post-processing software. Adjust [DZMTRX](#) and/or P_{cogm} to match the observed data or match to a finely gridded single porosity model.

M1	F1
M2	F2

Figure 2.23. A simple reservoir grid used to simulate a gas/oil transfer function

Modifying the recovery versus time response (Eclipse 100)

The eventual recovery of oil from a matrix block is governed by the final balance between the gravity head and the capillary pressure. The initial rate of drainage is governed by the transmissibility between the matrix and fracture, which in turn is controlled by the shape factor SIGMA. The ‘shape’ of the recovery curve between the end points can be modified by changing the relative permeability function for the matrix-fracture flow.

This can be done by modifying the relative permeability curves directly. However, when matching a finely gridded single porosity model or experimental results it is often simpler to modify a single parameter to change the ‘shape’ of the recovery versus time response. The `DPKRMOD` keyword applies a quadratic modification to the oil relative permeability to achieve this aim.

The modification factor is not a physical quantity and should be regarded only as a tuning parameter to match a dual porosity block with a finely gridded single porosity model.

The scaling for oil-in-water relative permeability is as follows.

For the case where m_w (the modification factor set in `DPKRMOD`) is greater than 0.0:

$$K_r = \bar{K}_r + M(K_r(s) - \bar{K}_r) \quad \text{Eq. 2.79}$$

where

$\bar{K}_r$ is the relative permeability at $\bar{s} = (\text{SOWCR} + 1.0 - \text{SWCO})/2$

$K_r(s)$ is the input table relative permeability

SOWCR is the critical oil to water saturation

SWCO is the connate water saturation

M is the modification function, which is given by $M = 4.0m_w K_r(s)^2 - 4.0m_w K_r(s) + 1$

For the case where m_w is less than 0.0, then the same scaling is applied but with the X and Y axes ($s, K_r(s)$) reversed.

Given a saturation s , a new saturation s' is calculated such that:

$$\bar{s} + M(s' - \bar{s}) = s \quad \text{Eq. 2.80}$$

where

$\bar{s}$ is $(\text{SOWCR} + 1.0 - \text{SWCO})/2$

M is the modification function, which is given by $M = 4.0m_wK_r(s')^2 - 4.0m_wK_r(s') + 1$

K_r is then taken as $K_r(s')$.

Scaling the fracture relative permeability (Eclipse 100)

By default, both the fracture-to-fracture flow and the fracture-to-matrix flow uses the fracture relative permeability table. Typically this will have a maximum relative permeability to water of 1.0, whereas the water relative permeability in the matrix at residual oil saturation is likely to be lower than 1.0. In some cases the matrix relative permeability may provide a better description of the fracture-to-matrix flow.

The fracture scaling option in the [DPKRMOD](#) keyword modifies the fracture relative permeability for the fracture-to-matrix flow while leaving the fracture-to-fracture relative permeability alone.

For the water phase the fracture relative permeability is scaled as follows:

$$K_{rf}(S_w) = K_{rf}^{\text{tab}}(S_w) \cdot K_{rm \text{ max}} / K_{rf \text{ max}} \quad \text{Eq. 2.81}$$

where

$K_{rf}^{\text{tab}}(S_w)$ is the fracture relative permeability looked up using the input table

$K_{rm \text{ max}}$ is the matrix relative permeability at $S_w = 1.0 - S_{owcr}$

$K_{rf \text{ max}}$ is the maximum fracture relative permeability

S_{owcr} is the critical oil saturation to water.

The other phases are treated in an analogous manner. In the gas case the relative permeability is scaled to the matrix value at $S_g = 1.0 - S_{ogcr}$ and in the oil case at $S_o = 1.0 - \max(S_{crg}, S_{crw})$.

Solution of the linear equations

The linear equations associated with dual porosity/permeability simulations can be written in the form

$$Ax + Uy = R_m$$

$$Lx + By = R_f$$

where

A is the usual banded matrix for the matrix cells

B is the banded matrix for the fracture cells

R_m is the matrix residual

R_f is the fracture residual

L and U are diagonal matrices coupling the fracture and matrix systems

x and y are the solutions (pressure and saturation changes) in the matrix and fracture cells respectively.

In the dual-porosity, single-permeability case, there is no direct communication between neighboring matrix cells and the matrix A is diagonal. This greatly enhances the efficiency of the solution procedure used by Eclipse because the equations can now be simplified to the form

$$(B - L \cdot A^{-1}U)y = R_f - L \cdot A^{-1}R_m \quad \text{Eq. 2.82}$$

$$x = A^{-1}(R_m - Uy) \quad \text{Eq. 2.83}$$

where A^{-1} is the inverse of A .

Because $L \cdot A^{-1}U$ is diagonal it does not alter the banded structure of B . The problem size is effectively halved because the equation for y only involves fracture cells. The equations are solved sequentially, first for the fracture solution, y , and then for the matrix solution, x . In the more expensive dual porosity/dual permeability case above simplification does not apply and the equations are solved simultaneously.

Data for single porosity regions in a dual porosity run is copied internally to the fracture cells, so that the above simplification still applies.

Eclipse 100

In dual permeability simulations, the linear equations can be solved more efficiently by pairing together the corresponding matrix and fracture cells. For a three-phase run, the diagonal entries of the Jacobian matrix are then 6×6 matrices of the form.

$$D = \begin{bmatrix} D_m & U \\ L & D_f \end{bmatrix} \quad \text{Eq. 2.84}$$

where D_m and D_f are the diagonal terms of the matrix and fracture Jacobian matrices. To account for the extra dimension, a fourth nesting has been used in the Nested Factorization preconditioner (see "[Solution of the linear equations](#)"). Using the notation of that chapter, an approximation, B , is constructed to the full Jacobian matrix.

$$A = D + L1 + U1 + L2 + U2 + L3 + U3 \quad \text{Eq. 2.85}$$

where the band terms $L1$ and $U1$ and so on represent flows between matrix or fracture cells. The following nested sequence of factorizations is used

$$\begin{aligned} B &= (P + L3)P^{-1}(P + U3) \\ P &= (T + L2)T^{-1}(T + U2) \\ T &= (G + L1)G^{-1}(G + U1) \\ G &= (\Gamma + L)\Gamma^{-1}(\Gamma + U) \end{aligned} \quad \text{Eq. 2.86}$$

where Γ is a diagonal matrix (whose elements are all 3×3 matrices in the three-phase case), and G has the same structure as D .

Eclipse 100

If [item 60](#) of the `OPTIONS` keyword is set greater than zero, the less efficient dual permeability solver for pre-97A versions of Eclipse is restored. If you encounter problems with your simulation run you may also wish to use this option.

Calculation of fractional volumes for gravity drainage

The initial fractional volume of water below the water contact, X_{wi} , is

$$X_{wi} = \frac{S_{wi} - S_{wco}}{1 - S_{cohy} - S_{wco}} \quad \text{Eq. 2.87}$$

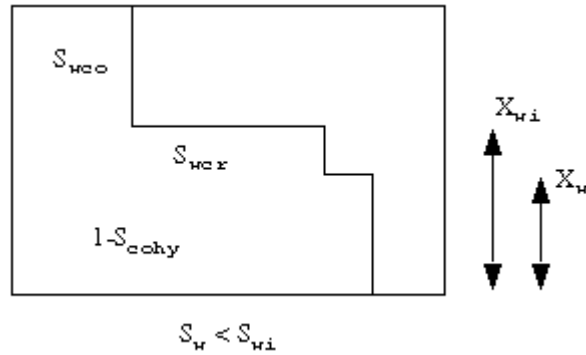


Figure 2.24. Initial fractional volume of water below the water contact

where

S_{wi} is the initial water saturation

S_{wcr} is the critical water saturation

S_{wco} is the connate water saturation

S_{cohy} is the connate hydrocarbon saturation

= S_{ocow} (oil/water systems)

= $S_{ocow} + S_{gco}$ (oil/water/gas systems)

= S_{gco} (gas/water systems)

S_{crhy} is the critical hydrocarbon saturation.

If $S_w > S_{wi}$ then

$X_w = 1$ when

$$S_w = S_{w \max} = X_{wi}(1 - S_{cohy}) + (1 - X_{wi})(1 - S_{crhy})$$

$$X_w = (S_w - X_{wi}(S_{crhy} - S_{cohy}) - S_{wco}) / (1 - S_{crhy} - S_{wco}) \text{ when } S_w < S_{w \max}$$

If $S_w < S_{wi}$ then

$$X_w = 0 \text{ when } S_w = S_{w \min} = X_{wi}S_{wcr} + (1 - X_{wi})S_{wco}$$

$$X_w = (S_w - X_{wi}(S_{wcr} - S_{wco}) - S_{wco}) / (1 - S_{cohy} - S_{wcr}) \text{ when } S_w > S_{w \min}$$

The initial fractional volume of gas above the gas contact, X_{gi} , is

$$X_{gi} = \frac{S_{gi} - S_{gco}}{1 - S_{lco} - S_{gco}} \quad \text{Eq. 2.88}$$

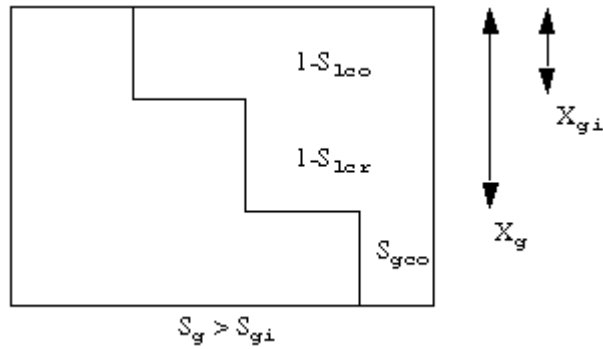


Figure 2.25. Initial fractional volume of gas above the gas contact

where

S_{gi} is the initial gas saturation

S_{gco} is the connate gas saturation

S_{gcr} is the critical gas saturation

S_{lco} is the connate liquid saturation

= S_{ogow} (oil/gas systems)

= $S_{ogow} + S_{wco}$ (oil/water/gas systems)

= S_{wco} (gas/water systems)

S_{lcr} is the critical liquid saturation

If $S_g > S_{gi}$ then

$$X_g = 1 \text{ when } S_g = S_{g \max} = X_{gi}(1 - S_{lco}) + (1 - X_{gi})(1 - S_{lcr})$$

$$X_g = (S_g - X_{gi}(S_{lcr} - S_{lco}) - S_{gco}) / (1 - S_{lcr} - S_{gco}) \text{ when } S_g < S_{g \max}$$

If $S_g < S_{gi}$ then

$$X_g = 0 \text{ when } S_g = S_{g \min} = X_{gi}S_{gcr} + (1 - X_{gi})S_{gco}$$

$$X_g = (S_g - X_{gi}(S_{gcr} - S_{gco}) - S_{gco}) / (1 - S_{lco} - S_{gcr}) \text{ when } S_g > S_{g \min}$$

Variation of the sigma factor with drainage process

In a typical water-wet system, production of oil from the matrix will be dominated by two different processes in different regions of the reservoir, depending on the displacing fluid. In the water invaded zone the production will be by water imbibition (assisted by gravity drainage), and in the gas invaded zone by gravity drainage (retarded by capillary effects).

A single sigma factor may not be sufficient to model both these cases. Typically, gravity drainage is a slower process than imbibition because the displaced fluid flows mainly in one direction (vertically).

Eclipse 100

The [SIGMAGD](#) keyword is available in three-phase dual-porosity runs to input a second sigma value for gas-oil recovery processes. The matrix-fracture transmissibility based on [SIGMAGD](#) will be used to calculate the oil flow when the following conditions are satisfied:

- The gravity drainage model is active,
- Oil flow is from the matrix to the fracture,
- The gravity drainage head associated with gas in the fracture is greater than the head associated with water.

Eclipse 300

Note that in Eclipse 300, the [SIGMAGD](#) keyword has a different function, as described in "[Calculation of fractional volumes for gravity drainage](#)".

Rock volume and cell activation in dual porosity thermal calculations

Rock volume

In single porosity calculations, the rock volume is simply the cell volume less the pore volume. In dual porosity calculations, this leads to a double counting of rock volume, as the cell volume is in reality common to all the cells of different porosity sharing the same location. To prevent this, the rock volume, defined as the average cell volume for all porosities less the total pore volume in all cells in that location, is shared between the porosities. Each cell receives the same rock volume as its pore volume. Any remaining rock volume is given to the matrix. If there is not enough rock volume to achieve this, the rock volume is shared in proportion to the cell pore volume.

As a special case, if a cell in one porosity has [ACTNUM](#)=2 (see below), meaning that the cell is rock only, then the remaining rock is shared equally among all the porosities for that cell location.

As a special case, if a cell in one porosity has [ACTNUM](#)=3 (see below), meaning that the cell is pore-volume only, then the pore volume assumed is the physical volume divided by the number of porosities. If an asymmetric pore volume distribution is required, the net-to-gross value can be adjusted as a weighting factor.

If this option does not achieve the desired rock distribution, you can override it with the [ROCKSPLV](#) keyword.

ACTNUM

The [ACTNUM](#) keyword allows you to deactivate cells. In thermal calculations, you can also deactivate either the fluids or the rock to give an energy-only or flow-only calculation in that cell.

In dual porosity, total deactivation of the matrix causes deactivation of the fracture, and the other way round. But rock-only or flow-only deactivation only affects the porosity specified. The specification of rock-only or flow-only cells does not affect the distribution of rock among the porosities, as described above; however, the rock or pore volume calculated may be set to zero by the [ACTNUM](#) flag.

There is no automatic default for synchronizing [ACTNUM](#) rock-only or flow-only deactivations with the rock distribution. It is suggested that you take advantage of the possibility to deactivate using porosity (keyword [PORO](#)) or pore volume (keyword [PORV](#)) and [ROCKSPLV](#) to control deactivation through the volume sizes on a per cell, per porosity basis.

If a cell is rock-only ([ACTNUM](#)=2) then the rules for apportioning rock volume are modified; see above. If a cell is pore-volume only ([ACTNUM](#)=3) then the rules for apportioning pore volume are modified; see above.

Integrated capillary pressure option

When a gravity drainage option is in use, the final recovery from the matrix is determined by the balance between the capillary pressure and gravity forces. The final saturation distribution in the matrix block can be thought of as a transition zone where at each height the capillary pressure is $P_c = \Delta \rho \cdot g \cdot h$.

Within the dual porosity matrix representation, it is desirable that the final recovery should be equal to the final recovery from the continuous matrix medium. This can be achieved by modifying the capillary pressure curves in the matrix to take account of the fact that the rock P_c curves need to be integrated over the matrix block height to calculate the average saturation.

The integrated P_c option is activated by using the `INTPC` keyword in the `PROPS` section. The appropriate modified capillary pressure curves are written to the `PRINT` file for inspection if either the `SWFN` or `SGFN` mnemonics are used in the `RPTPROPS` keyword.

The integration procedure for the gas-oil capillary pressure is as follows, starting with a piecewise linear table:

$$p_c = P_c(S_g) \text{ and } S_g = P_c^{-1}(p_c)$$

where P_c^{-1} is the inverse capillary pressure function.

Now consider a matrix block of height h with a contact ($p_c = 0$) at the bottom of the block and a capillary pressure P at the top.

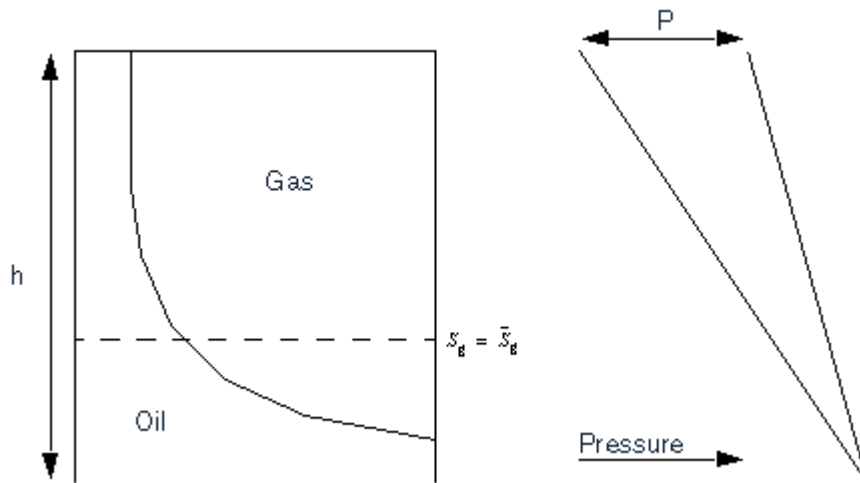


Figure 2.26. Integrated capillary pressure

The average gas saturation $\overline{S_g}$ in the matrix block up to the height h equivalent to $p_c = P$ is given by

$$\overline{S_g} = \frac{1}{P} \cdot \int_0^P P_c^{-1}(p) dp \quad \text{Eq. 2.89}$$

This can be used to set up a modified capillary pressure function such that the capillary pressure at a saturation $\overline{S_g}$ is equal to the gravity drainage head for a block of height h and saturation $\overline{S_g}$.

The gravity drainage model accounts for the gravity head within a grid block, by assuming vertical equilibrium. (Note that if the gravity drainage option is not used the gravity head within a grid block is ignored.)

For the gravity drainage model selected by [GRAVDR](#), the gravity head P_{gdr} is proportional to the mobile oil saturation, assuming the fracture is filled with gas:

$$P_{gdr} = \frac{S_t - \bar{S}_g}{S_t} \cdot \Delta\rho \cdot g \cdot h \quad \text{Eq. 2.90}$$

where

S_t is 1 – SWCO – SOGCR

SWCO is the connate water saturation

SOGCR is the critical oil-in-gas saturation

$\Delta\rho$ is the difference between the gas and oil densities

g is the acceleration due to gravity

h is the block height.

Now the pressure at the top of the matrix block P is given by

$$P = \Delta\rho \cdot g \cdot h$$

From this a new capillary pressure function can be constructed, such that at a saturation $\bar{S}_g$ the capillary pressure is exactly equal to P_{gdr} and hence the equilibrium saturation will be at (and only at) $S_g = \bar{S}_g$.

The constructed capillary pressure function P'_c should have $P'_c(\bar{S}_g) = P_{gdr}$, and thus

$$P'_c(\bar{S}_g) = \frac{S_t - \bar{S}_g}{S_t} \cdot P \quad \text{Eq. 2.91}$$

where P is the true capillary pressure at the top of the block when the equilibrium saturation is $\bar{S}_g$. This can be calculated by looking up the table of $\bar{S}_g$ versus P given by Equation 2.89, which is stored in Eclipse as a table.

For the alternative gravity drainage model selected by [GRAVDRM](#), the gravity head P_{gdr} is proportional to $P/2$, assuming the fracture is filled with gas. This condition requires that oil is not re-infiltrating into the matrix, and this possibility should be excluded using the GRAVDRM keyword.

Notes

- By construction, for any block height h the ‘correct’ final saturation is achieved by running the simulator to a stable state when:

$$P'_c(S) = \text{gravity drainage head,}$$

$$S = \bar{S} \text{ for that block height.}$$

- At other saturations ($S \neq \bar{S}$) a potential difference generates the usual flow to and from the block.
- During equilibration the modified capillary pressure is accounted for. Note however that the equilibrium saturation is not the same as in the equivalent drained block, as the fluids above the contact are assumed to be only connate water + gas, where a drained block includes the residual oil

Discretized matrix model (Eclipse 100)

Traditional dual porosity models assume that the matrix to fracture flow is in steady state, and thus the matrix cell can be regarded as a single cell. In some cases, however, for example a well test in a dual porosity system, the transient nature of the matrix to fracture flow can be important. To model these systems a discretized matrix model is available, which subdivides each matrix cell into a series of nested sub-cells, allowing the simulator to predict the transient behavior.

The matrix cells are still only connected to their corresponding fracture cell, but flow to the matrix surface is supplied from a one dimensional grid system. Each matrix grid cell could be regarded as a “Russian Doll” of nested matrix sub-cells.

Matrix sub-grid

The matrix subdivision is set up automatically by Eclipse once you have specified the number of sub-cells to split each matrix cell into, using the `NMATRIX` keyword in the `RUNSPEC` section. The differing properties across the field are specified in the usual way by using the standard `GRID` section keywords including `SIGMAV` to specify the matrix-fracture transmissibility.

The generated grid is logarithmic away from the fracture wall and can represent a linear (1D), radial (2D) or spherical (3D) geometry. You can control the size of the outer matrix sub-cell (as a fraction of the corresponding fracture cell) and the geometry type using the `NMATOPTS` keyword. Once the outer sub-cell size and the number of sub-cells required are known, the growth factor for each individual matrix cell can be calculated. The maximum and minimum growth factors across the field are reported in the `PRINT` file. If `NMATOPTS` is omitted, then provided `NMATRIX` is present, the default values from `NMATOPTS` will be assumed.

First an overview of the discretization is given for the linear and non-linear geometry types, and then a more detailed view of the volume and transmissibility calculations will be provided.

Overview of Linear geometry

If the geometry is assumed to be 1D then the `SIGMA` factor and hence the matrix-fracture transmissibility are assumed to represent 1D flow.

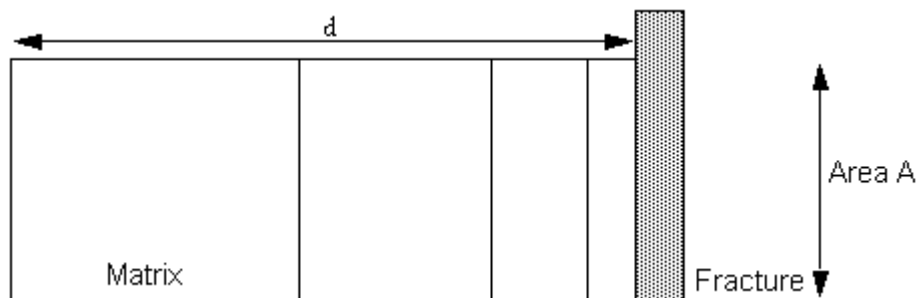


Figure 2.27. Nested matrix sub-cells in linear geometry

The matrix-fracture transmissibility is assumed to be the transmissibility associated with flow over the distance d , through area A . The sub-cell transmissibilities are then calculated assuming a 1D logarithmic grid over the same matrix volume.

Overview of cylindrical and spherical geometries

In the 2D (Cylindrical) and 3D (Spherical) flow options the geometry is taken to be Cartesian to enable simple mapping from the input matrix-fracture transmissibility.

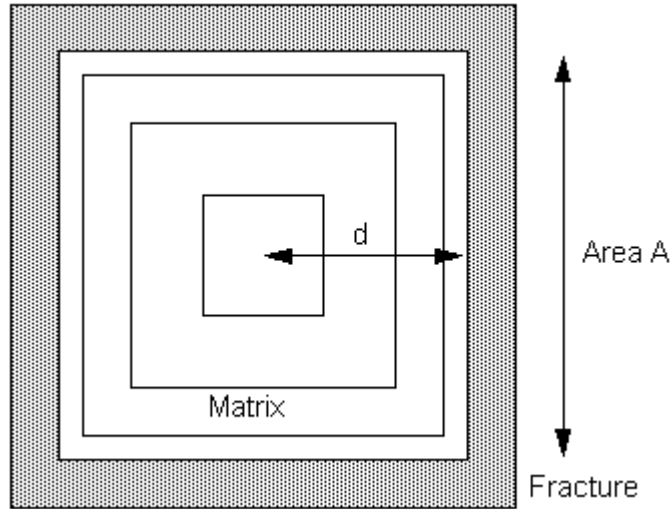


Figure 2.28. Nested matrix sub-cells in cylindrical and spherical geometry

In the 2D case the diagram represents a square cylinder with the matrix-fracture transmissibility assumed to represent four times the flow over the distance d through area A . In the 3D case the diagram represents a cube with the transmissibility taken as sum of six flows out of the six faces.

In each case the sub-cell transmissibilities are calculated assuming a logarithmic distribution of sub-cell sizes away from the fracture walls.

Volume and transmissibility calculations details

Let the n^{th} sub-matrix cell have volume v_n , where $n=1$ is the sub-cell nearest the fracture, and v_N is the sub-cell furthest away. N is the value input by keyword `NMATRIX`. Then:

$$V_m = \sum_n v_n$$

where V_m is the volume of the whole matrix region.

The outermost sub-cell volume v_1 is determined by the fraction of the fracture pore volume specified by item 2 of the `NMATOPTS` keyword. The sizes of the remaining sub-cells are then functions of the factor

$$f = \left(\frac{X}{\xi_1} \right)^{\frac{1}{N-1}}$$

where

X is the distance of the center of the matrix from the fracture surface, and

ξ_1 is the width of the outermost sub-cell.

For a LINEAR assumed geometry (NMATOPTS item 1), the distances X and ξ_1 are assumed proportional to the volumes V_m and v_1 respectively. For CYLINDRICAL or SPHERICAL geometries, these distances are calculated from the geometry of a cylindrical or spherical shell with inner and outer radii X and $X-\xi_1$, using the known volumes V_m and v_1 for the whole cylinder/sphere and the shell.

The factor f gives the distance ξ_n from the matrix-fracture surface to the inner surface of the remaining ($n > 1$) interior sub-cells by

$$\xi_n = f^{n-1} \xi_1$$

that is, the logarithms of the ξ_n scale linearly.

As with ξ_1 , when the assumed geometry is CYLINDRICAL or SPHERICAL, the sub-cells are cylindrical or spherical shells around the center of the matrix, and $X-\xi_n$ is the inner radius of the shell. The shell volume is determined from the inner and outer radii, with the same constant of proportionality as used to determine X and ξ_1 in the first place. For a LINEAR assumed geometry the sub-cells are assumed to be slabs parallel to the matrix-fracture surface, arranged from the surface to the center of the matrix, and the volumes are assumed proportional to the distances. The overall effect of this is to obey the relationship

$$\frac{(X^D - (X - \xi_1)^D)}{X^D} = \frac{v_1}{V_m}$$

The transmissibilities between the sub-cells are then determined by the area of the surface between them and the distance between the sub-cell centers (where for CYLINDRICAL and SPHERICAL geometries, cell center means the radius halfway between the cell's two surfaces, and for LINEAR geometries, the position halfway between the cell's two surfaces). Thus for any geometry, the center of cell n ($n > 1$) is at

$$c_n = \frac{1}{2}(\xi_{n-1} + \xi_n)$$

and the area of the interface between cells n and $n+1$ is

$$A_n = Z(X - \xi_n)^{D-1}$$

where D is the dimensionality (LINEAR=1, CYLINDRICAL=2, SPHERICAL=3) and Z is a constant for all the sub-cells n . For LINEAR this is the constant area equivalent to the outer area of the matrix; for CYLINDRICAL it is 2π times the height of the matrix cylinder and for SPHERICAL it is 4π .

Then the transmissibility of the original matrix-fracture connection is scaled to the sub-cell to sub-cell connection by the factor

$$G_n = \left(\frac{A_n}{A_X} \right) \left(\frac{2X}{c_n - c_{n-1}} \right)$$

where $A_X = ZX^{D-1}$ is the outer area associated with overall matrix size X . Note that the factor Z cancels out in the expression as only ratios of the A factors are used.

For the transmissibility between the outer matrix sub-cell and the fracture, the factor G_1 is $2X/\xi_1$

Initialization, solution method and efficiency

The internal sub-matrix cells cannot be explicitly initialized in the way that the normal matrix cells can, and so the **EQUIL** keyword must be used to perform an initialization.

The matrix sub-grid forms a series of separate 1D flow systems connected through the outer sub-cells only to the corresponding fracture cells. In the solution matrix structure these flows lead to a set of tri-diagonals connected to the fracture system. This structure is used in the pre-conditioning step by eliminating the tri-diagonals before the fracture system is solved, effectively reducing the problem to a set of flow equations for the fracture only. Once the fracture equations have been solved a back substitution gives the matrix sub-grid solution.

Clearly, discretizing the matrix increases the total number of cells in the model. However, the efficiency of the above procedure ensures that increasing the number matrix sub-cells will have a significantly smaller effect on the run time than a similar increase in the number of simulation grid blocks.

Reporting the matrix solution

The solution for separate sub-cells (or rings) of the matrix can be output as arrays over the simulation grid by setting any of the first 5 data items in `RPTSCHEG` to 2. If an individual sub-cell solution is required as a function of time, the following summary keywords are available:

BPR, BRS, BRV, BOSAT and BWSAT.

The matrix sub-cell is specified by appending its ring number to the end of the keyword, for example:

```
BOSAT7  
  1  1  2  /  
  /
```

The ring number starts from 1 for the outer sub-cell and increases towards the center of the matrix cell. Note that ring 1 is equivalent to the keyword without a ring number and represents the matrix sub-cell adjacent to the fracture.

Restrictions

The option cannot at present be used with the following:

- gravity drainage models (`GRAVDPR`, `GRAVDRM`)
- Local grid refinement option
- Dual permeability model
- Parallel option

and must use the following to initialize the sub-matrix cells

- `EQUIL`

Discretized matrix model (Eclipse 300)

The discretized matrix model can be invoked in Eclipse 300 in a similar way to Eclipse 100. However there are some differences in the setup as the Eclipse 300 model is based on the multi-porosity options described in "Multi porosity". The `DIMENS` keyword needs to include all porosities in the Z-direction.

The model generates new `SIGMAV` values for the connections between the matrix sub-grid cells, following the geometric partitioning as described for Eclipse 100. The input primary (outer) matrix block volumes, pore volumes and rock volumes, are partitioned between the matrix sub-grid cells. Both transmissibilities and diffusivities are then generated from the new `SIGMA` values, the block volumes, pore volumes and rock volumes (the rock volumes are only used if the rock contains adsorbed gas). Permeabilities, for the

transmissibilities as given by [PERMX](#) or [PERMMF](#), are by default copied from the primary matrix grid cells if unspecified.

The geometrical partitioning is done in the same way as for Eclipse 100 for `LINEAR`, `RADIAL` and `SPHERICAL` discretization methods. In addition the `UNIFORM` and `VERTICAL` methods are available in Eclipse 300. For information, see the [NMATOPTS](#) keyword.

The volumes of the partitioned outer matrix grid cells, that connect to the fracture grid cells, can be set either

- to match a fraction of the fracture pore volume using `NMATOPTS` item 3 set to `FPORV`, or
- to match a fraction of the matrix bulk volume using `NMATOPTS` item 3 set to `MBLKV`.

When the feature is used with coal or shale gas, the definition of volumes with adsorbed gas needs to be considered. For information on this, see ["Shale Gas"](#). It is recommended that the keyword [ROCKFRAC](#) is used to set the input rock fractions and that the [PORO](#) keyword is used to set the input porosity fractions. This makes it easier to switch between the instant adsorption model and the time-dependent sorption model, since the inputs for setting rock and pore volumes are compatible. Only the input from the primary matrix grid cells are used to generate the partitioned matrix volumes.

Further information about the Eclipse 300 matrix discretization model can be found in ["Multi porosity"](#).

Single medium conductive fractures (Eclipse 300)

An alternative dual porosity formulation in Eclipse 300 is the Single Medium Conductive Fractures development. This is based on a generalization of the technique described in [van Lingen et al \(2001\)](#).

The technique allows the incorporation of the effects of conductive fractures into a single medium model (that is not explicitly using a dual system of porosities or permeabilities) by modification of grid properties and the generation of pseudo tables. It is focused on modeling fractures on scales that are too large to be homogenized using the traditional approach.

For further details of the technique, please refer to [van Lingen et al \(2001\)](#). The sections below assume that the reader has familiarity with the paper, and outline the approach taken in Eclipse 300, including extensions and generalizations on the technique described in the paper, and how to use the Eclipse 300 development.

The single medium approach

For a given input fracture swarm distribution map, and associated fracture properties, there are three subsequent main areas of implementation:

- Grid property modifications
- Pseudo generation
- Well PI modifications

The implementation of the technique in these areas is described in the following paragraphs.

Grid property modifications

Firstly grid block properties are modified to take account of the physical void introduced by the fractures. The porosities are increased by proportional volume averaging, and permeabilities flow-averaged using Darcy's Law, either arithmetically (parallel to fracture) or harmonically (perpendicular to the fracture).

The approach taken in Eclipse 300 differed to that in the paper in

- the full use of all cell dimensions (rather than assuming square cells)
- the calculation of directional permeability enhancements (parallel and perpendicular components)
- allowing for vertical permeability enhancement
- allowing for fracture volumes in porosity modifications.

One further modification is to recognize fractures being defined in a zigzag rasterized format. When the path of a fracture changes direction by 90 degrees within one cell, the effective length of the fracture is halved. This attempts to model the intention that the fracture does not traverse the cell in question twice, but rather somehow ‘turns a corner’.

Pseudo generation

The saturation curves of the fractures and affected grid blocks are combined under the assumption that oil is displaced from fractures by either gas or water before any oil is displaced from the matrix volume of a grid block, which requires that any capillary imbibition is dominated by viscous and gravity effects. Under this assumption, the saturation tables are averaged by determining end point saturations from mass conservation, and end point relative permeabilities by Darcy's Law arithmetic averaging of effective phase flows. The original fracture tables are then used in a scaled form during the fracture invasion by gas or water, and scaled matrix tables are used for matrix invasion. Notice that (as in the paper) the fracture tables are assumed straight lines; no attempt is made to combine the full shape of the curve with that of the matrix.

Since the permeabilities and saturation tables used potentially vary on a grid block basis, the above could feasibly result in one set of pseudo tables for each affected cell. To reduce the dimensionality, a grouping technique is used that averages the pseudo end points used over cells with common attributes, these being saturation region, permeability value, and the number of contained fractures. This extended the paper's treatment which assumed that layers represented the saturation regions.

If velocity dependent relative permeability effects are being modeled, the values used for the internally generated tables for the `VDKRG`, `VDKRO` and `VDKRGK` keywords will be taken from the associated input matrix tables. For example, if input saturation function table sets 1 and 2 correspond to the input matrix and fracture properties respectively of a grid block required to model a single medium conductive fracture using the internally generated table set 3, the `VDKRG`, `VDKRO` and `VDKRGK` properties for table 3 will be taken from those specified for table 1. The `VDKRG`, `VDKRO` and `VDKRGK` properties assigned to the internal tables can be reported using the `RPTPROPS` keyword. Prior to 2016.1, these values were not defined for the internally generated tables. Pre-2016.1 behavior can be restored by setting [item 342](#) of the `OPTIONS3` keyword to 1.

Note: Capillary curves are not included in this treatment, and are merely extrapolated at constant value from the matrix end-point values.

Well PI

By default, wells completed in grid blocks containing conductive fractures will experience enhanced flows due to the increased effective permeability, due to both the permeability and relative permeability changes. [Van Lingen et al \(2001\)](#) argues that this only arises if the fracture actually cuts the well bore in a connection cell. Because of this, you are required to specify whether or not a well connection cell is to be modified by this treatment by indicating whether it is cut by fractures or not. If not, then the well PI is modified so as to reverse the changes in permeability, and the connection cell saturation table reverts to that of the original matrix block. You are also given the chance to enter a PI multiplier to indicate perhaps some degree of connection with conductive faults.

Using the conductive fractures model

The conductive fractures model is activated by specification of the [SCFDIMS](#) keyword in the RUNSPEC section, which also sets up dimensioning defaults for reading in the fracture details, and also defaults for the pseudo grouping algorithms used.

The conductive fractures themselves should be specified using the [CONDFRAC](#) keyword, one for each fracture being modeled. This specifies the path of fracture planar segments through the grid blocks, the effective aperture, fracture permeability, and saturation table number to use. The format is similar to the [FAULTS](#) keyword, but note that the fractures are assumed to pass through a block, whereas faults run alongside blocks. For this reason, when a ‘corner’ is detected in the fracture, the contribution from the two intersecting segments is halved, to reflect the fact that the corner is formed notionally at the center of the cell, rather than around the edges.

The well intersection behavior is controlled using the [WELLCF](#) keyword.

Compatibility

The feature has been made explicitly compatible with the restart mechanism, and will run in parallel mode. It is currently restricted to family (ii) saturation table keywords - SWFN, SGFN and SOF2/SOF3.

The development is known not to be compatible with the following options:

- radial grids
- analytic saturation functions
- MPFA
- Up-front memory
- LGRs
- The Leverett J-function option per saturation function [JFUNC](#)

Multi porosity

x	Eclipse 100
x	Eclipse 300

In Eclipse 300 you can define a multi porosity model as an extension to the dual porosity model to model the fracture system and a number of matrix pore systems.

There are two ways to supply the grid properties for the matrix sub cells:

- Using the keyword [NMAOPTS](#) the matrix sub cells are assigned pore volumes as described in the "[Discretized matrix model \(Eclipse 100\)](#)". The grid is geometrically partitioned as a “Russian Doll” of nested matrix sub-cells. This enables grid data to be supplied only for the dual porosity part of the grid and the nested matrix sub-grid cells data are generated automatically. New [SIGMAV](#) factors are automatically generated from the input [SIGMA](#)/[SIGMAV](#) factors from the primary matrix. You can still set other data such as permeabilities, although this is not recommended. By default, if these are not specified, the input primary matrix values are used to populate the sub-cell values.
- Input over the grid. The data is supplied for the entire simulation grid, similarly to the dual porosity model, but with grid data such as permeabilities also for the matrix-matrix connections.

In either case the number of pore systems is input using the [NMATRIX](#) keyword. This general setup gives a variety of possibilities for studying the transient behavior in the matrix. In particular the option can be used for.

- Shale gas reservoirs. See "[Shale gas](#)".
- Coal bed methane. See "[Adsorption models in Eclipse 300](#)"

The special case of triple porosity systems to represent vuggy carbonates uses the same underlying multi porosity model, but restricts the number of porosities to 3 (fracture, matrix, vugs with fracture and matrix connections) or 4 (with additional “isolated” vugs connected only to the matrix). This model is activated using the [TRPLPORO](#) keyword, which also specifies if isolated vugs are to be used. The data are supplied for the entire simulation grid, as with the multi porosity option (2) above. This model is described in more detail in the Triple Porosity section below. Triple porosity is also available in Eclipse 100 but the general multi-porosity feature is **not** available.

Using the multi porosity option

RUNSPEC section

The number of simulation cells in the Z-direction, as specified by the [DIMENS](#) keyword, needs to be a multiple of the number of pore systems. The number of pore systems is equal to the number of matrix pore systems, set by the [NMATRIX](#) keyword, increased by one to account for the fracture system ($NPOROS = NMATRIX + 1$).

The [DUALPORO](#) keyword is also required. If a dual permeability calculation is to be made where matrix to matrix connections are also used, then the [DUALPERM](#) keyword is also required. For multi-porosity, this will create a multi-permeability model - the sub-matrices will also be connected together. In most cases, this may not be the required solution, as the primary job of the sub-matrices is usually to model transients in the matrix-to-fracture flow.

Note: It is important to note that the Eclipse 300 model requires the full grid dimensions to be set by [DIMENS](#), unlike the discretized matrix model in Eclipse 100.

GRID section

In order to be compatible with standard dual porosity input, the first `NDIVIZ` / `NPOROS` layers are associated with the outermost matrix cells connecting to the fractures. The next layers are associated with the fracture cells. Thereafter the matrix sub cells follow as illustrated in the figure below.

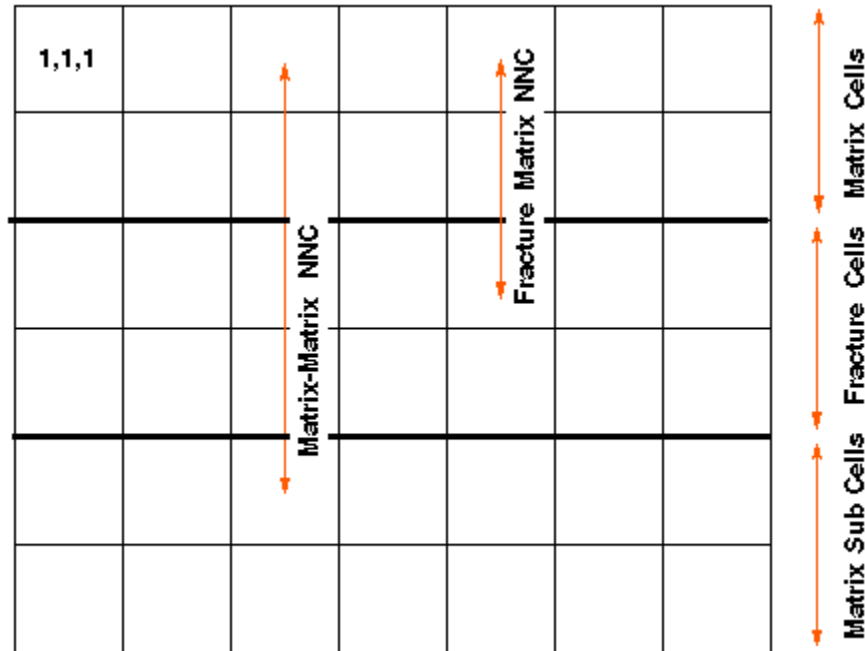


Figure 2.29. Multi porosity example having three pore systems.

The connections between the porosity systems are automatically generated. It is possible to supply the grid for the outermost primary matrix grid cells only, and using the keyword `DPGRID` the fracture and sub matrix grid are copied from the outermost matrix grid if they are not specified. Please note that `SIGMAV` values should be specified for all the matrix subgrid cells, including when `DPGRID` is used.

By default, properties such as porosities and permeabilities together with `SIGMAV` should be input for the sub-matrix grid-cells. In a multi-permeability model, it is possible to alter the transmissibility between the sub-grid cells using the `MULTX`, `MULTY` and `MULTZ` multipliers. This can, for example, be useful in order to disable communication between some of the matrix sub-grid cells.

Matrix-matrix transmissibility and diffusivity

Each pore system is connected as described above.

The transmissibilities between the sub-grid cells are then calculated similar to the matrix-fracture transmissibilities as described in "Dual Porosity". Note that the innermost matrix grid cell properties are used to set up the transmissibilities and diffusivities.

The permeability factor used between the subgrid porosities can be set, similarly to dual porosity, by the keyword `PERMMF`.

For two grid cells where inner matrix cell I connects to the outer grid cell J (which could be matrix or fracture), the transmissibility is given as

$$TR_{IJ} = CDARCY \cdot K_I \cdot VB_I \cdot \sigma_K$$

where

K_I is taken as the X-direction permeability of the matrix blocks (unless PERMMF is also specified),

VB_I is the matrix cell bulk volume (note that this is not the pore volume, having no porosity factor), and

σ_K is a factor of dimensionality LENGTH^{-2} , to account for the matrix-matrix interface area per unit volume.

Note that the cell index for σ_K has been written as K rather than I . When I and J are both matrix cells, then K is the *outer* matrix cell J . However, when outer cell J is the fracture, then K is the inner cell: that is, K is the outermost matrix. The result of this is to have all the sigma values in contiguous locations in the first $N-1$ porosity locations in the SIGMAV keyword input. The innermost SIGMAV porosity (porosity $N=\text{NMATRIX}+1$) is not used.

The diffusivity is similarly given as

$$\text{DIFF}_{IJ} = \text{DIFFMMF}_I \cdot \text{VOL}_I \cdot \sigma_K$$

where

DIFFMMF_I is the diffusivity multiplier, and

VOL_I is the matrix cell pore volume or coal volume (coal bed methane option)

Matrix discretization (NMATOPTS)

When the NMATOPTS option is invoked, the matrix sub-grid cell properties are copied from the outer primary matrix grid-cells. Only the bulk volume, porosities and/or rock fraction of the primary matrix grid cells are used to generate the matrix sub-grid cells' new volumes. The pore volume and transmissibility/diffusivity follow what is described in the "[Discretized matrix model \(Eclipse 100\)](#)". However Eclipse 300 does not generate the transmissibilities directly, but generates new sigma (SIGMAV) values following the geometric partitioning of the matrix block, and updates the block volumes, pore volumes and rock volumes of the matrix grid cells. In addition, the Eclipse 300 model can be used with UNIFORM partitioning. The pore volumes are evenly distributed between the matrix sub-grid cells and the transmissibilities are calculated following a 1D computation where the distance DX of the matrix cells is assumed to be partitioned as $\text{DX} / \text{NMATRIX}$.

DUAL PORO

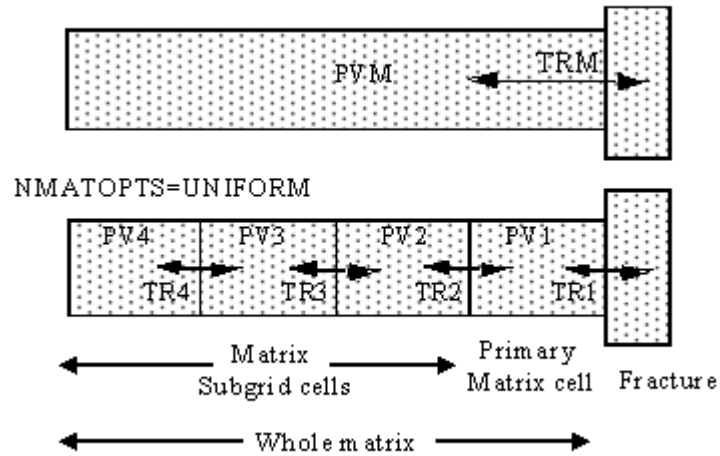


Figure 2.30. Structure of the matrix discretization and nomenclature

In the figure above, the uniform partitioning is used with $NMATRIX=4$. Each block is partitioned uniformly and sigma values generated for the connections. The pore volume of the primary matrix cell is divided into $PV1 = PV2 = PV3 = PV4 = \frac{PVM}{4}$. The assumptions for the UNIFORM partitioning is the same as for the LINEAR except that each matrix block is uniformly partitioned. The available methods with $NMATOPTS$ are:

NMATOPTS	Assumption for initial matrix- fracture transmissibility/diffusivity	Partitioning method
UNIFORM	1D flow over the distance d and through area A	Partitions the distance d uniformly
LINEAR	1D flow over the distance d and through area A	Logarithmic growth factor of the distances
RADIAL	2D flow, four times flow, over a distance d and through area A	Logarithmic growth factor of the distances
SPHERICAL	3D flow, six times flow, over a distance d the and through area A	Logarithmic growth factor of the distances
VERTICAL	For gravity drainage. A depth assigned to the sub-matrix grid cells.	Partitioned uniformly (vertically)

Table 2.2: Available methods with $NMATOPTS$

Based on the method, a new internal grid is generated, representing the initial matrix-fracture connection. The distance “d” is partitioned resulting in new bulk volumes, pore volumes and rock volumes. Based on the initial sigma factor assigned to the primary matrix grid, new sigma factors are calculated and populated in the matrix grid.

As an example consider the UNIFORM partitioning above and then how to calculate new sigma values.

For a unit m^3 cube, $BV=1m^3$. The distance $d=1m$ and the area $A=1 m^2$. Partitioning d into $1/4m$ gives the flow over $1/4m$ for each connection through an area A. Since “d” and “A” are already embedded in the

initial sigma value, multiplication gives $(\sigma \cdot 4)$ for the connection. This needs to be scaled to apply to the new block volumes of the cells and so this gives the factor $(\sigma \cdot 4) \cdot BV / (BV/4)$ for the new sigmas, that is $(\sigma \cdot 16)$.

For more information about the discretization, see the ["Discretized matrix model \(Eclipse 100\)"](#).

Coal in multi porosity

When using the [NMATOPTS](#) matrix discretization feature with coal or shale gas, the definition of coal volumes needs to be considered. See ["Coal bed methane model"](#) and for the partitioning into the matrix subgrid cells see ["Shale gas"](#). Only the input of the primary matrix sub-grid cells is needed when generating the sub-cells' coal and pore volumes.

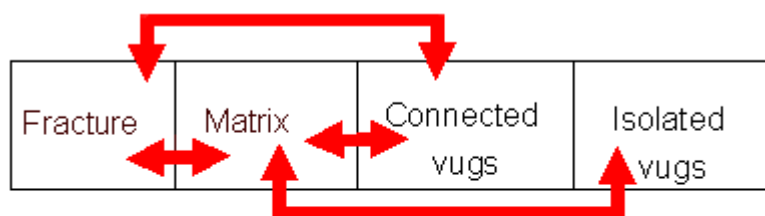
In general it is recommended to have all sub-grid cells defined as coal and/or use the instant sorption method when doing a matrix discretization. Having nonhomogeneous properties means that the number of matrix cells and the placement of pore, coal volume becomes important when considering the response from the matrix. For specialized models, the general multi-porosity option can be used with user-defined sigma values within the sub-matrix grid.

Triple porosity

When the [TRPLPORO](#) keyword is used, only 3 or 4 porosities are allowed. The matrix and fracture porosities are specified as porosities 1 and 2 as usual. Porosity 3 is used to represent vugs (larger voids in the rock which are not part of the fracture system) which are connected to the matrix in the same way as the fracture is connected to the matrix, and additionally to the fracture through micro fracture connections. If used, porosity 4 represents additional vugs with connection to the matrix only (isolated vugs).

Unlike the standard multi porosity connections, a porosity in triple porosity may be connected to more than 2 other porosities, as the matrix can be connected to the fracture and two different vug porosities (isolated and connected). There is also a cyclic path in the porosity system, formed by the matrix, fracture and connected vug porosities. These relationships are illustrated in the figure below. Note that the logical order, with the fracture as the outer porosity, is shown, but in the grid description the fracture is still the second porosity.

As for the multi porosity model, the [DUALPORO](#) and, where needed, the [DUALPERM](#), keywords are required. The input allows for the possibility of a triple permeability model, where the vug porosities form a connected continuum. However, in most if not all cases, only a dual permeability model will be required for triple porosity. To make the vug cells connected only to the local matrix and, for connected vugs, to the local fracture, you should zero the permeability or add permeability multipliers of size zero to the appropriate permeability for the other vug-matrix, vug-fracture or vug-vug connection. The convention for assigning cell permeabilities to triple porosity connections is noted in the Transmissibilities section below.



Triple porosity for fractured matrix with vugs
(including cyclic flow paths)

Figure 2.31. Triple porosity connectivity

Note: `NMATRIX` and `NMATOPTS` should not be used with `TRPLPORO`

Transmissibilities

In the current version of the input for this model, transmissibilities between vugs porosities and the matrix and fracture porosities are still described by Kazemi-type multipliers using the `SIGMA` and `SIGMAV` keywords. However, the `LTOSIGMA` keyword cannot be used and the interpretation of the `SIGMA` multiplier as an inverse squared matrix length is not applicable. In particular the connection between fracture and connected vugs is through micro fractures, and the connection between the matrix and vugs is through the typical inter-vug distance in the matrix and the size of the vugs. The `SIGMA` values used for all connections to vugs, whether through micro fractures from the fracture, or through matrix rock from the matrix, should honor this difference from standard matrix-fracture connection values.

An exact Kazemi-type equivalent to the `LTOSIGMA` input cannot be provided. However, for matrix-vug sigmas, a useful rule of thumb is that the product $V\sigma$ can be replaced by $\beta A/L$ where A is the cross-sectional area of the vug, L is half the distance between two vugs, and β is the fraction of total volume occupied by the vugs. There is no simple formula to represent the connected-vug/fracture sigma, and you should consider general transmissibility theory for the micro fractures characterizing this connection.

Input values for `SIGMAV` are applied to porosities in the following way:

- matrix/fracture connection sigma values are input through the matrix cells;
- fracture/connected-vug sigma values are input through the fracture cells;
- matrix/connected-vug sigma values are input through the connected-vug cells (third porosity);
- matrix/isolated-vug sigma values are input through the isolated-vug cells (fourth porosity, if present).

PROPS, REGION and SOLUTION sections

This follows the input as for a standard single or dual porosity model. However, simulation grid cell data must be specified on a cell-by-cell basis, regardless of the use of `NMATOPTS`.

An exception is the regional data such as `SATNUM`, `EQLNUM` and `FIPNUM`. If no value is supplied these are copied from the primary matrix grid cell.

Output

Summary output follows the usual syntax, where block solutions can be obtained by specifying a cell index corresponding to the matrix sub-cell.

On the restart file the sub-cell matrix solutions are only output if a flexible restart file is requested.

Output to the PRT file using the [RPTGRID](#), [RPTSOL](#) and [RPTSCHED](#) keywords will output the requested quantity for all cells.

Examples

The following data sets come with the Eclipse installation:

File	Description
MPORO . DATA	Multi porosity example, gas-water case
SHALEGAS1 . DATA	Shale gas

Table 2.3: Multi porosity example files

Limitations

The multi porosity option cannot currently be used with:

- Coarsening
- Block to block connections ([BTOBALFA](#) or [BTOBALFV](#))
- The triple porosity option cannot currently be used with parallel in Eclipse 100.
- Local grid refinements when using the discretized matrix model.

Eclipse 100

Eclipse 100

Independent reservoir regions

x	Eclipse 100
x	Eclipse 300

Using the ISOLNUM option (Eclipse 100 only)

The independent reservoir region facility allows the efficient simulation of models containing two or more reservoirs that only communicate with each other through the well controls.

If there are no flow connections between the two reservoirs (through normal transmissibility, non-neighbor connections, wells, or aquifers), the linear equations solved by Eclipse 100 for the two reservoirs are independent. The solution of the two sets of linear equations in general require different numbers of linear iterations.

The Independent reservoir option allows Eclipse 100 to utilize this potential efficiency gain.

Note: This option is not compatible with the parallel option.

1. Set `NRFREG` (the maximum number of independent reservoir regions) in item 3 of the `RUNSPEC` section keyword `REGDIMS`.

2. Use the `ISOLNUM` keyword in the `GRID` section to establish the separate regions.

There must be at least one active cell allocated to each reservoir region between 1 and `NRFREG`. Any cells given a reservoir region number of 0 or $> \text{NRFREG}$ are treated as inactive.

The `ISOLNUM` data can be written out to the debug file for an existing model by using the `RPTISOL` keyword in the `GRID` section. The `ISOLNUM` array can be subsequently added to the data set.

3. The `ISOLNUM` array can be output to the `PRINT` file using the mnemonic `ISOLNUM` in the `RPTGRID` keyword.
4. If the mnemonic `NEWTON` in the `RPTSCHEG` keyword is activated, a separate summary line for each reservoir is printed at each Newton iteration.

Merging two or more corner point grids (Eclipse 100 and Eclipse 300)

Corner point grids are usually supplied for the entire grid from a pre-processing program such as Petrel. However, if a number of separate models have been run independently and are later required to be run as a single model, there are a number of facilities to enable the corner point grids to be input separately.

If the separate grids are to be placed side by side in the X or Y directions, the `COORD` and `ZCORN` keywords can be used in conjunction with a `BOX` keyword to input the data for a portion of the field. Note that the grids should be separated by a row of inactive cells to prevent the coordinate lines overlapping. Also any inactive parts of the merged model should have dummy `COORD` and `ZCORN` values input; the cells can then be made inactive using the `ACTNUM` keyword. See "Example 1".

If the separate grids are to be 'stacked' in the Z direction, either the grids must use the same coordinate lines or the multiple reservoir option must be employed (set `NUMRES > 1` in the `RUNSPEC` section keyword `NUMRES`). If `NUMRES` is set > 1 , you must identify which layers belong to which reservoir with the keyword `COORDSYS`. To combine the grid data for each reservoir into a single model, the `RESVNUM` keyword can be used. Data is then expected for a single reservoir at a time. A layer of inactive cells should be left between the separate grids, or a zero-transmissibility multiplier should be applied, to stop communication between the reservoirs. See "Example 2".

Examples

Example 1

To merge a 2*4*1 grid with a 3*2*1 grid side by side in the X direction.

```
RUNSPEC section:
DIMENS
 6  4  1  /
GRID section:
BOX
 1  2  1  4  1  1  /
COORD....   for the first grid.
ZCORN....
BOX
 4  6  1  2  1  1  /
COORD....   for the second grid.
ZCORN....
EQUALS
'ACTNUM' 0 3 3 1 4 1 1 /   inactivate the row between the grids.
'ACTNUM' 0 4 6 3 4 1 1 /   inactivate the redundant region of
/                           the grid.
```

Example 2

To merge a 2*4*1 grid with a 3*2*1 grid, 'stacked' in the Z direction.

```
RUNSPEC section:
DIMENS
 3  4  3  /
NUMRES
 2  /
GRID section:
COORDSYS
 1  2  /
 3  3  /
RESVNUM
 1  /
BOX
 1  2  1  4  1  1  /
COORD....   for the first grid.
ZCORN....
RESVNUM
 2  /
BOX
 1  3  1  2  3  3  /
COORD....   for the second grid.
ZCORN....
EQUALS
'ACTNUM' 0 1 3 1 4 2 2 / inactivate the layer between the grids.
'ACTNUM' 0 3 3 1 4 1 1 / inactivate the redundant region of
'ACTNUM' 0 1 3 3 4 3 3 / the grid.
/
```

Vertical equilibrium

x	Eclipse 100
	Eclipse 300

This chapter describes the Eclipse 100 vertical equilibrium option.

Vertical equilibrium (VE) is the assumption that equilibrium is established in a cell in a time short compared to that characteristic of horizontal flows. This implies that the phase hydrostatic potential is independent of depth within a cell. In the absence of capillary pressure effects, the saturation distribution is then a step function depending on the fluid contact depths. This saturation distribution is taken account of when Eclipse calculates relative permeability values for the cell faces. In the case shown (figure 2.33), for example, water may flow out of the bottom of the cell, but not out of the top, at which no mobile water exists. The water flow through the sides reflects the fraction of the cell faces over which a mobile saturation exists. If the VE assumption of approximate hydrostatic equilibrium is valid, such a segregated saturation distribution may provide a more accurate description of fluid flow than a default dispersed option in which the fluids are assumed to be evenly distributed over the grid block.

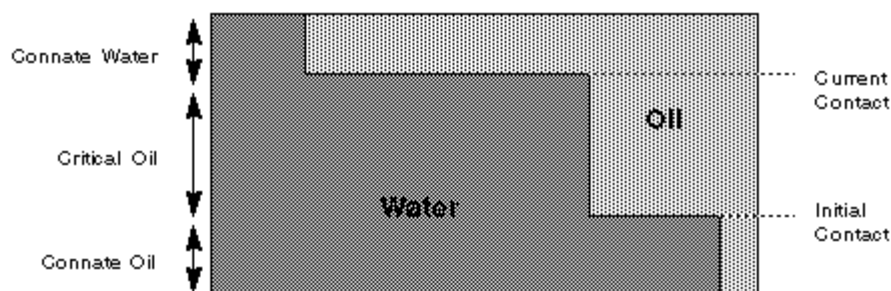


Figure 2.32. A cell with approximate hydrostatic equilibrium

The effect of corner point geometry upon the VE model is to introduce different depths for cell faces. For example, water may only flow out of an up-dip cell face if the contact level is sufficiently high.

In such a model hysteresis can occur if, for example, oil invades the water zone and retreats, leaving a residual oil saturation.

An option exists in the Eclipse VE implementation that enables you to specify the fractions of distributed and segregated saturation distribution functions which are to be used.

The use and formulation of the Eclipse VE option is discussed in more detail below.

Using the VE option

The vertical equilibrium option in Eclipse 100 is enabled by including the keyword `VE` in the RUNSPEC section.

There is a set of four keywords (`VEFRAC`, `VEFRACV`, `VEFRACP`, `VEFRACPV`) which are specific to VE. These keywords enable you to specify either dispersed (rock curve) or segregated (VE) saturation functions, or a weighted average of the two, for calculating relative permeabilities and capillary pressures. `VEFRAC` gives the VE relative permeability fraction, and `VEFRACP` the VE capillary pressure fraction, for the whole field. `VEFRACV` and `VEFRACPV` allow the specification of `VEFRAC` and `VEFRACP` values on a cell-by-cell basis. The values of `VEFRAC` and `VEFRACP` should lie in the range 0.0 to 1.0. For example, using the following:

```
VEFRAC
0.8 /
```

```
VEFRACP
0.8 /
```

will specify 80% VE and 20% rock curves for both relative permeability and capillary pressure. The default is a VEFrac and VEFracP value of 1.0. If the VE option is not required, omit the keyword VE from RUNSPEC, rather than use a VEFrac value of 0.0 everywhere.

If VE is specified, and the ninth argument of the [EQUIL](#) keyword data is set to zero, then a method of equilibration specific to VE is used, which ensures that the initial contact depths are exactly matched by the initial saturations, and which yields a static initial solution. If this ninth argument is not zero, then a normal Eclipse fine grid equilibration is performed using the rock curves.

The latter method is only likely to be useful if a non-default VEFrac value is used; for a pure VE run, the former option is best.

To obtain a printed output of the contact depths at each report step, use the arguments OWC and/or GOC in the [RPTSCHEd](#) keyword. These control the output of the oil water and gas oil contact depths respectively.

The vertical equilibrium option cannot be used in combination with the Hysteresis option in the same run.

The endpoint scaling option/initial mobile fluid end point correction should not be used with the vertical equilibrium option.

The cell saturation distribution

The saturation distribution in a cell assumed by the VE model is based on the assumption that the phase hydrostatic potential is independent of depth. In the absence of capillary pressure effects, the saturation distribution becomes a step function. The form of this depends on the phase option used.

The oil-water case

The distribution is derived from the water saturation value, S_w . If Z_c is the current oil water contact depth, and Z_l is the lowest contact depth which has occurred in the run, the distribution is as shown below. This has three depth intervals, as follows:

Region 1

Above the current oil water contact. The water saturation takes the connate value, S_{wco} , and the oil saturation is $1-S_{wco}$.

Region 2

Above the lowest oil water contact, but below the current. In this region water has displaced oil, so the critical oil saturation remains, S_{ocr} , whilst the water saturation is $1-S_{ocr}$.

Region 3

Below the lowest oil water contact. Only the connate oil saturation exists, S_{oco} , which is usually zero, and the water saturation is $1-S_{oco}$.

If the current contact attempts to fall below Z_l , oil invading the water zone, then Z_l is set to the new Z_c value. If the contact again rises Z_l retains its value, so that a critical oil saturation remains in the invaded region.

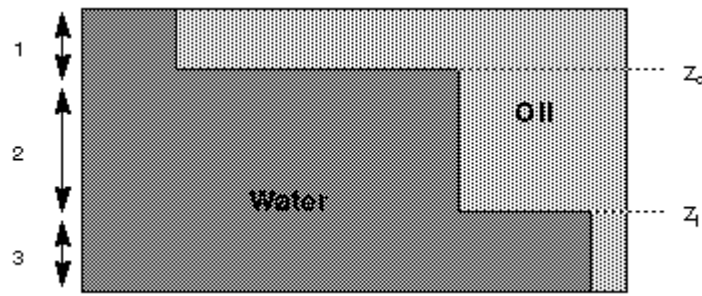


Figure 2.33. Oil-water vertical equilibrium

The oil-gas case

The distribution is derived from the gas saturation value, S_g . If Z_c is the current oil gas contact depth, and Z_h is the highest contact depth that has occurred in the run, the distribution is as shown below. This has three depth intervals, as follows:

Region 1

Above the highest oil gas contact. The connate oil saturation exists, S_{oco} , which is often zero, and the gas saturation is $1-S_{oco}$.

Region 2

Below the highest oil gas contact, but above the current. In this region gas has displaced oil, so the critical oil saturation remains, S_{ocr} , whilst the gas saturation is $1-S_{ocr}$.

Region 3

Below the current oil gas contact. The gas saturation is the connate value, S_{gco} , which is usually zero, and the oil saturation is $1-S_{gco}$.

If the current contact attempts to rise above Z_h , oil invading the gas zone, then Z_h is set to the new Z_c value. If the contact again falls Z_h retains its value, so that a critical oil saturation remains in the invaded region.

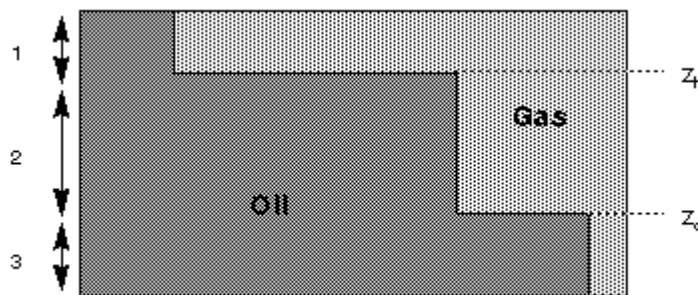


Figure 2.34. Oil-gas vertical equilibrium

The gas-water case

The distribution is derived from the gas saturation value, S_w . If Z_c is the current water gas contact depth, and Z_l is the lowest contact depth that has occurred in the run, the distribution is as shown below. This also has three depth intervals, as follows:

Region 1

Above the current water gas contact. The water saturation has the connate value, S_{wco} , and the gas saturation is $1 - S_{wco}$.

Region 2

Above the lowest water gas contact, but below the current. In this region water has displaced gas, so the critical gas saturation remains, S_{gcr} , whilst the water saturation is $1 - S_{wcr}$.

Region 3

Below the lowest water gas contact. The gas saturation is the connate value, S_{gco} , which is usually zero, and the water saturation is $1 - S_{gco}$.

If the current contact attempts to fall below Z_l , gas invading the water zone, then Z_l is set to the new Z_c value. If the contact again rises Z_l retains its value, so that a critical water saturation remains in the invaded region.

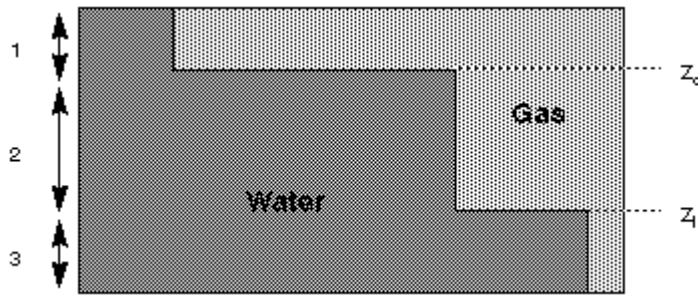


Figure 2.35. Gas-water vertical equilibrium

The three phase case

In this case S_w and S_g are distinct variables. The usual distribution is shown in figure 2.37. There are five depth intervals, as follows:

Region 1

Above the highest gas oil contact. The water saturation has the connate value, S_{wco} , and the oil saturation is the connate with respect to gas, S_{ocog} . The gas saturation is thus $1 - S_{wco} - S_{ocog}$.

Region 2

Below the highest oil gas contact, but above the current. Gas has displaced oil, so the critical oil saturation with respect to gas remains, S_{ogcr} , with $S_w = S_{wco}$ and $S_g = 1 - S_{ogcr} - S_{wco}$.

Region 3

Below the current oil gas contact and above the current oil water contact. The water and gas saturations take their connate values, (S_{wco}, S_{gco}) , and the oil saturation is $1 - S_{wco} - S_{gco}$.

Region 4

Below the current oil water contact, but above the lowest oil water contact. Water has displaced oil, so the critical oil saturation with respect to water remains, S_{owcr} , with $S_w = 1 - S_{owcr} - S_{gco}$.

Region 5

Below the lowest oil water contact. The gas saturation is connate, S_{gco} , and the oil saturation is the connate value with respect to water, S_{ocow} . The water saturation is $1 - S_{ocow} - S_{gco}$.

In any of the phase combinations, one or more of the regions may not exist, depending on the conditions in a cell.

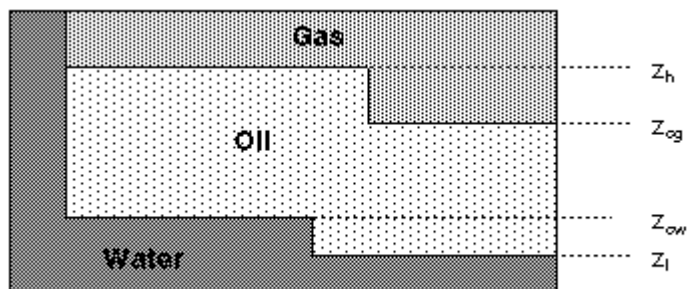


Figure 2.36. Three-phase vertical equilibrium

In the 85/7 version of Eclipse the oil-water contact was simply not permitted to be above the oil-gas contact. In later versions the treatment of the three-phase VE case was extended to cover the case in which the oil zone becomes immobilized. When this occurs, the oil zone becomes a residual saturation over a certain depth interval, and the water may rise above this zone, leading to a gas-water contact.

The immobilized oil state can occur with the gas-water contact above, below or in the immobilized oil. The first situation would arise if the water contact rose up into the original oil zone. In the normal three phase oil case, the residual oil saturation would then have to exist from the original contact depth to the current. This is possible as long as sufficient oil saturation exists. If, however, the contact continues to rise, a point will be reached at which all the oil exists at residual saturation. If the contact rises further, the following immobilized oil distribution will exist (assuming connate oil and gas saturations are zero):

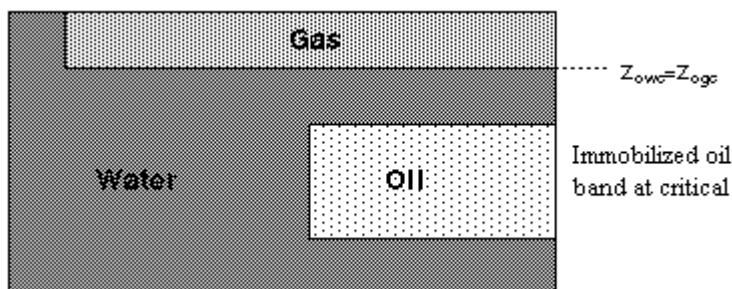


Figure 2.37. Three phase equilibrium with immobilized oil

Eclipse now models this immobilized oil state correctly, and returns to the mobile oil state once a sufficient oil saturation exists.

The various connate and critical saturations used in the above are obtained from the PROPS section data with the appropriate saturation table number, as follows:

Saturation	Description
Connate water saturation, S_{wco}	The smallest saturation value in the SWFN table.
Critical water saturation, S_{wcr}	The smallest saturation value for which a non-zero k_{rw} value exists.

Saturation	Description
Connate gas saturation, S_{gco}	The smallest saturation value in the SGFN table.
Critical gas saturation, S_{gcr}	The smallest saturation value for which a non-zero k_{rg} value exists.
Connate oil saturation in water, S_{ocow}	This is $1 - S_{wmax} - S_{gco}$, where S_{wmax} is the maximum saturation value in the SWFN table.
Connate oil saturation in gas, S_{ocog}	This is $1 - S_{gmax} - S_{wco}$, where S_{gmax} is the maximum saturation value in the SGFN table.
Critical oil saturation in water, S_{owcr}	This is the lowest oil saturation for which a non-zero k_{ro} value exists, with $S_g = S_{gco}$ (that is, from the second column of the SOF3 table in the three phase case or from SOF2 in the two phase case).
Critical oil saturation in gas, S_{ogcr}	This is the lowest oil saturation for which a non-zero k_{ro} value exists, with $S_w = S_{wco}$ (that is from the third column of the SOF3 table in the three phase case or from SOF2 in the two phase case).

VE relative permeabilities and capillary pressures

In the Eclipse VE algorithm, the current best solution to the non-linear fluid flow equations is held as a set of cell saturations. The current contact levels are calculated from the saturations using the models for the segregated fluid distributions described in the previous section. (The lowest and highest levels, Z_l and Z_h , are stored as arrays, and will affect the relationship between the saturation and current contact values). Once the contact depths have been obtained, the relative permeability values over the cell faces and a ‘pseudo capillary pressure’ can be evaluated.

These calculations require a knowledge of the cell geometry, obtained by an analysis at the start of the run. For each cell a table of fractional volumes below each distinct depth is stored, a distinct depth being one at which one or more cell corners exist. These fractional volumes are obtained using a linear interpolation of the horizontal cross-sectional areas of the cell at each distinct depth. A similar table of fractional areas below depths is kept for each cell face. With these tables it is possible to obtain effective relative permeability values by integration over depth, weighting the appropriate relative permeability values implied by the saturation distributions in each interval. The different depths of block faces are thus correctly accounted for in corner point geometry.

The ‘pseudo capillary pressure’ is due to the difference in hydrostatic head between two phases, and is a function of the distance between the cell center and the current contact depth. In the oil water case it is given by:

$$P_{cow} = g(\rho_w - \rho_o)(Z_{owc} - Z_{center}) \quad \text{Eq. 2.92}$$

where

ρ_w and ρ_o are the phase densities

g is the acceleration due to gravity constant

Similarly, for the gas phase, the pseudo capillary pressure is given by:

$$P_{cog} = g(\rho_o - \rho_g)(Z_{goc} - Z_{center})$$

The densities are calculated at the end of each timestep and then held constant over the next timestep. If water is not present in a cell, or has a saturation value lower than connate, then the water-oil contact Z_{owc} is set to the lowest depth in the cell. This effectively introduces a cutoff in the pseudo capillary pressure, although in cases where this occurs, the water would be immobile. In a similar manner, if gas is absent, the contact Z_{goc} is set to the top of the cell.

Residual flow model

When water sweeps through a cell containing oil, a residual oil saturation as defined in the [SATNUM](#) saturation table will remain immobile in the cell. In some cases this ‘residual’ oil may in fact be slightly mobile, and over a period of time some of this oil will flow, reducing the oil saturation to a new residual value. This situation can be modeled in a very simple manner using the residual flow option.

You specify a second set of saturation tables, using the [RESIDNUM](#) keyword in the [REGIONS](#) section. An extra component of relative permeability is then added to the flows, which is calculated by looking up the relative permeability on the [RESIDNUM](#) tables using the prevailing cell saturation.

$$K_{r \text{ tot}} = K_{r \text{ VE}} + K_{rr}(S_o) \quad \text{Eq. 2.93}$$

where

$K_{r \text{ tot}}$ is the total relative permeability

$K_{r \text{ VE}}$ is the VE relative permeability calculated from the contact depth

$K_{rr}(S_o)$ is the relative permeability from the [RESIDNUM](#) table

S_o is the oil saturation.

The [RESIDNUM](#) tables will typically have very small oil K_r values, with zero K_r values for phases with no residual flow.

For example, consider a system with the following relative permeability behavior:

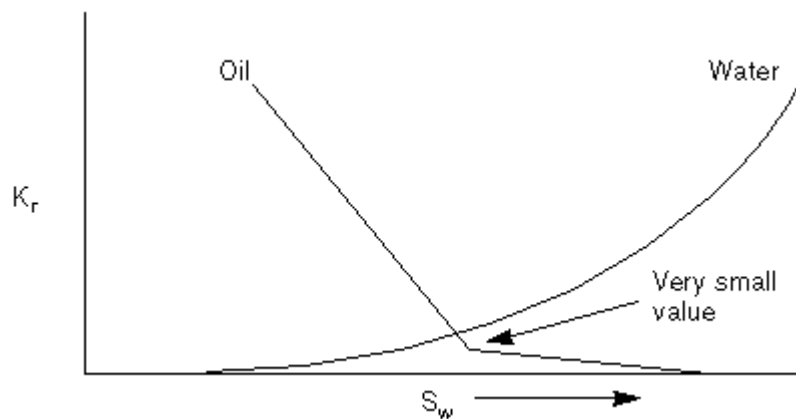


Figure 2.38. Relative permeability behavior for a residual flow model

This relative permeability curve could be partitioned into a curve for VE (specified using [SATNUM](#)) and a curve for the residual flow specified using [RESIDNUM](#).

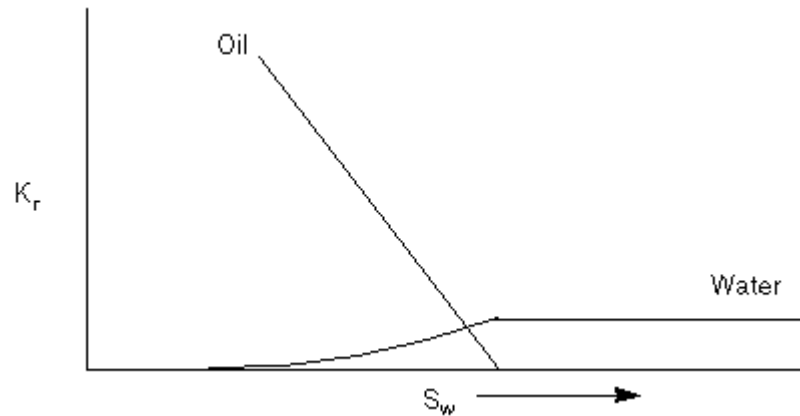


Figure 2.39. VE Curves (SATNUM)

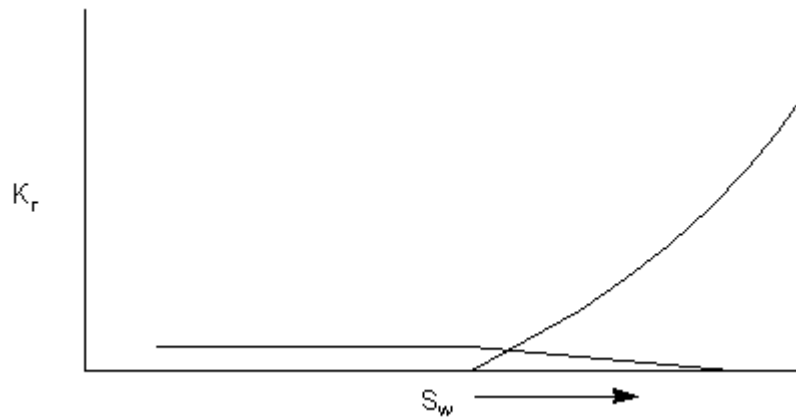


Figure 2.40. Residual flow model

Miscellaneous points

RESTART files produced by VE runs include the lowest and highest contact depths, as required by the phase combination used. It is not possible to restart a VE run from a non-VE run.

Although the saturation distributions assumed by the VE algorithm are based on zero capillary pressures, non-zero values may be entered in the PROPS data section, to be used if the VEFrac mixing parameter is not unity; the VE fraction will ignore rock capillary pressures.

The VE model described above assumes a three zone saturation distribution. It is possible to extend this model to allow four zones. In such a case, considering the oil water phase combination, there would be a critical water saturation up to a highest contact depth, and a (possibly different) connate saturation above this. This extension to a 'four cycle' VE model could be provided in future if demand exists.

If twisted sloping faults are used, the intersection of a cell face with a plane of constant depth becomes slightly curved. This curvature is ignored when setting up the cell geometry data for VE. However, the fractional volumes below distinct depths are then re-normalized to ensure that the volume fraction below the highest distinct depth is 1.

If the reservoir geometry is entered using TOPS, DX, DY etc., rather than with corner point geometry (COORD, ZCORN etc.), the top and bottom of the cells will be flat. In this case an unrealistic step function

change in the relative permeability would be obtained when the fluid contact reaches the top of a cell, which would cause problems in solving the non-linear flow equations. To avoid this, the cell top is 'spread' slightly, over 10% of the cell depth, so that the flow relative permeability rises from zero to its maximum value as the fluid contact traverses this interval.

In a dipping reservoir specified using TOPS, DX etc. the VE algorithm will prevent flow to the adjacent cell if the water level is below the bottom of the adjacent cell. A better method of specifying a dipping reservoir would be to use corner point geometry (COORD, ZCORN etc.). However the full face of each cell can be specified for flow by using the OPTIONS keyword, setting OPTIONS(19)=1. A second option can be used by specifying OPTIONS(19)=2, in which case the flows are calculated assuming a corner point approximation to the original grid. The pore volume and transmissibilities are preserved, however the VE flow calculations are performed assuming the cells to be part of a continuous layer. This corner point approximation will be output to the GRID file and hence can be plotted using a pre-processor such as Petrel. This latter option is used in Local Grid Refinement cases if the grid is entered as a block centered grid.

Consider a reservoir specified using corner point geometry. When there is a gap of non-zero thickness between a pair of adjacent active cells in a column which are connected together (such as for a pinchout of non-zero thickness due to use of MINPV), contact depths cannot be resolved in this dead space. In this case, setting OPTIONS(52)=2 instructs Eclipse to modify corner-point depths on the bottom face of the topmost cell in the pair to be equal to those for the top face of the bottommost cell. If the Local Grid Refinement option is used, Eclipse always modifies the corner point depths in local grids to avoid gaps between local and global grids where there is a connection.

In the assumed segregated fluid distributions described previously, the connate saturation of a phase was always assumed to be present throughout a cell. As soon as this is exceeded, a contact forms, and flow is generally possible through the top or bottom of the cell. This will usually occur at a lower saturation than the critical value. For example, the connate gas saturation is often zero. If any gas saturation exists in a cell it will be assumed to be concentrated at the top, and thus be mobile through the top of the cell, even if this is below the critical gas saturation. This is a straightforward consequence of the assumption of vertical equilibrium, but may not be appropriate in some cases. For example, if gas appears in a cell through bubble point crossing, it would be evenly distributed through the cell, and could not concentrate at the top until the critical, rather than the connate, saturation is reached. Arguments 4 and 5 of the OPTIONS keyword (used in the SCHEDULE section) enable the critical water and gas saturation values to replace the connate in the simulation if required. Note, however, that this may result in the VE equilibration option not being static, as this always uses connate saturations.

Compressed vertical equilibrium option

The compressed vertical equilibrium option is a facility to allow the modeling of a reservoir in which the entire reservoir section is assumed to be in vertical equilibrium. If this assumption holds, then gravity forces dominate the viscous forces and we can assume that the phases will be distributed with the less dense phases above the denser phases.

The option takes full account of the reservoir heterogeneity by allowing the engineer to input a full three-dimensional model of the reservoir, which may include many geological layers with different porosities, permeabilities and relative permeabilities. The individual layers are automatically merged in the Z-direction into compressed VE cells, which use internal tables to account for the variation in rock properties across the width of the merged layers.

The physical model assumed in the compressed VE option contrasts with the standard (uncompressed) VE description, in that vertical equilibrium is assumed to exist across the entire section, rather than within each original cell (representing a geological layer or sub-layer) individually. Note that if the assumption that the

entire reservoir section is in VE holds, then the two models should give similar descriptions of the initial fluid distribution.

Features such as shale breaks and non-VE layers can be modeled. In these cases the merging of layers is only carried out over particular sections of the reservoir, rather than across the whole width. Non-neighbor connections are generated automatically to allow the modeling, for example, of intermittent shales.

Using the option

The option is initiated by setting item 1 to 'COMPRESS' in keyword **VE** in the RUNSPEC section. The 3D model will automatically be modified into a compressed VE grid having a number of larger merged cells. In the absence of any shale breaks (zero Z-transmissibilities or inactive cells between layers), a three dimensional run will reduce to a two dimensional areal model.

The **COLLAPSE** keyword is available in the GRID section to allow you to prevent certain cells being merged into compressed VE cells. Also, the **CRITPERM** keyword can be used to specify a minimum value of the Z-direction permeability below which a cell will not be merged with its lower neighbor in the column.

The top cell of each column of merged cells is taken as the representative cell. In the printed output these representative cells are output with a "T" in place of the decimal point; the original cells below the top cell are output with a "V".

Breaking the compressed VE cells

Within a column of cells, each cell will be merged with the next active cell below, provided that none of the following situations occur:

- You have given the cell a collapse integer of zero in the **COLLAPSE** keyword, to prevent it from being merged.
- $TRANZ = 0$, representing an impermeable barrier. The assumption of vertical equilibrium breaks down if the transmissibility is zero.
- $PERMZ \leq$ the critical value entered in **CRITPERM**, if this keyword is present. If the **CRITPERM** keyword is absent, the critical permeability defaults to 0.
- There is an inactive cell with $DZ > 0$ between the two active cells. If an inactive cell has $DZ > 0$ then it must form an impermeable barrier.
- Either cell has been refined using **RADFIN** or **CARFIN** (LGR option).

Any of these situations will prevent the column being merged into a single collapsed VE cell. The column will then be represented by two or more collapsed VE cells. Non-neighbor connections will be generated wherever required.

Tabular description of the cell

Internal tables are set up to describe the compressed VE cells. These tables consist of a set of data items tabulated against distinct depths. The distinct depths are the corner points of the original cells that make up the compressed VE cell. If too many of these distinct depths occur together, some will be removed to improve efficiency.

There are tables for the cell volume and for each face. The tabulated data is as follows:

Cell volume table:

Keyword	Description
DEPTH	
VOLUME	Normalized volume tables, including porosity and NTG values
SOWCR	Oil saturation at critical water (residual oil to water)
SOGCR	Oil saturation at critical gas (residual oil to gas)
SWCR	Critical water saturation
SGCR	Critical gas saturation
SOWMX	Oil saturation at maximum water saturation
SOGMX	Oil saturation at maximum gas saturation
SWCO	Connate water saturation
SGCO	Connate gas saturation
KRO1	K_{ro} at SWCO, SGCR
KRO2	K_{ro} at SWCO, SGCO
KRO3	K_{ro} at SWCR, SGCR
KRW1	K_{rw} at SOWCR, SGCO
KRW2	K_{rw} at SOWMX, SGCO
KRG1	K_{rg} at SOGMX, SWCO
KRG2	K_{rg} at SOGCR, SWCO

Table 2.4: Cell volume table

Keyword	Description
DEPTH	Depth
AREA	Normalized surface area tables
KRO2	K_{ro} at SWCO, SGCO
KRW1	K_{rw} at SOWCR, SGCO
KRW2	K_{rw} at SOWMX, SGCO
KRG1	K_{rg} at SOGMX, SWCO
KRG2	K_{rg} at SOGCR, SWCO

Table 2.5: Cell face tables

These internal tables can be examined by using the [VEDEBUG](#) keyword. For each cell requested, they can be output as tables to the debug file.

Use with local grid refinement

The compressed VE option can be used in conjunction with Local Grid Refinement. Any cells that have been refined are not placed in compressed VE merged cells, as a more detailed description of the reservoir is required in these areas. Compressed VE cells will be generated adjacent to the refinement and will be automatically connected by non-neighbor connections.